Multi-Verifier Keyed-Verification Anonymous Credentials

Jan Bobolz[1], Emad Heydari Beni[2,3], Anja Lehmann[4], Omid Mirzamohammadi[2], Cavit Özbay[4], and Mahdi Sedaghat[2]

[1] University of Edinburgh, Edinburgh, UK
jan.bobolz@ed.ac.uk
[2] COSIC, KU Leuven, Leuven, Belgium
{emad.heydaribeni, omid.mirzamohammadi, ssedagha}@esat.kuleuven.be
[3] Nokia Bell Labs
[4] Hasso Plattner Institute, University of Potsdam, Germany
{anja.lehmann, cavit.oezbay}@hpi.de

Abstract. Keyed-Verification anonymous credentials (KVAC) enable privacy-preserving authentication and can be seen as the symmetric primitive of conventional anonymous credentials: issuance and verification of credentials requires a shared secret key. The core advantage of KVACs is that they can be realized without pairings, which still appears to be a significant bottleneck when it comes to real-world adoption. KVACs provide all the benefits from anonymous credentials, in particular multi-show unlinkability, but only work in the setting where the issuer and verifier are the same entity, limiting the applications they can be used in. In this work we extend the idea of keyed-verification credential to a setting where again *multiple verifiers* are supported, each sharing an individual secret key with the issuer. We formally introduce this as multi-verifier keyed-verification anonymous credentials (mKVACs). While users must now get verifier-specific credentials, each credential still provides multi-show unlinkability. In terms of security, mKVAC naturally strengthens the single-verifier variant, as it guarantees that corruption of any verifier does not impact unforgeability guarantees for other verifiers. The main challenge therein is to not trade this added flexibility for privacy, and hide the verifier's identity in the credential issuance. We provide formal definitions of all desired security and privacy features and propose a provably secure and pairing-free construction. Along the way, we develop a new KVAC-like primitive that authenticates group elements and offers statistical privacy, solving the open problem of combining multi-verifier support and pairing-freeness. Finally, we demonstrate practicality of our protocol via implementation benchmarks.

1 Introduction

Anonymous credentials (ACs) [Cha85] let users prove certified attributes, such as age or membership, in an unlinkable manner and revealing only the minimal amount of information necessary for every authentication. Efficient protocols for such anonymous credentials are well-known in the academic community

for more than 20 years. While initial schemes relied on RSA-based constructions [CL01, CL03, CL02], the most efficient ones all rely on pairing-based cryptography [BBS04, CL04, FHS19, TZ23, MBS+25]. Pairing-based systems are elegant and highly efficient. In particular they are performant enough to be used on high-throughput or resource-limited environments like mobile phones – at least in theory.

Unfortunately, pairing-friendly curves are not recognized by national standardization bodies, and thus also not widely supported in hardware and face regulatory hurdles [SKSW22]. This hinders the real-world deployment of anonymous credentials, despite seeing a massive interest in these techniques lately. Particularly noteworthy is the EUDI wallet, a digital identity app currently developed by the EU and expected to be launched to 500 million EU citizens by the end of 2026 [Eur25]. The EUDI wallet must satisfy the security and privacy requirements laid out in the eIDAS 2.0 regulation [Eur24], which explicitly demands unlinkability and selective disclosure. While technically this can only be realized with anonymous credentials, the EU decided to revert to batch issuance of one-time ECDSA credentials due to the deployment challenges of pairing-based anonymous credential schemes [BBC+24, EU 25]. Apart from not satisfying the privacy features legally required, the batch issuance also comes with significant challenges in deployment and thus the interest in supporting solutions that support multi-show unlinkability remains high – but it must be compatible with secure hardware that is currently on the market, given the tight time-constraints.

KVACs: Anonymous Credentials w/o Pairings. Keyed-verification anonymous credentials (KVACs), introduced by Chase, Meiklejohn, and Zaverucha [CMZ14], offer a solution to realize multi-show unlinkable presentations *without* pairings. In this setting, the issuer and verifier share a secret key, which turns verification of credential into a keyed operation. This reliance on a shared secret key enables to easily turn pairing-based anonymous credential schemes into pairing-free variants [Orr24]. Essentially, they replace the signatures with algebraic message authentication codes (MACs), with the most prominent constructions being CMZ [CMZ14] and BBS [BBS04] MACs. As KVACs can be implemented over standard elliptic curves, they have already been used in practical systems such as Signal's private group messaging [CPZ20] and Tor's bridge distribution [TG23].

More recently, KVACs also see strong interest from major tech providers, such as Google and Apple, aiming to improve their systems that currently rely on Privacy Pass [DGS+18, HIP+24]. Privacy Pass is a protocol suite standardized by the IETF that enables privacy-preserving rate limiting and is currently built from either OPRFs or blind RSA signatures, and has seen massive adoption in real-world applications. Both instantiations rely on batch issuance of many one-time tokens, that users can later redeem in an unlinkable manner. Similar as in the EUDI context, securely realizing batch-issuance for millions of users is a significant deployment challenge. Therefore, both Google and Apple have proposed KVAC-based solutions for so-called Anonymous Rate-Limited Credentials (ARC), that enable multi-show unlinkable presentations from a sin-

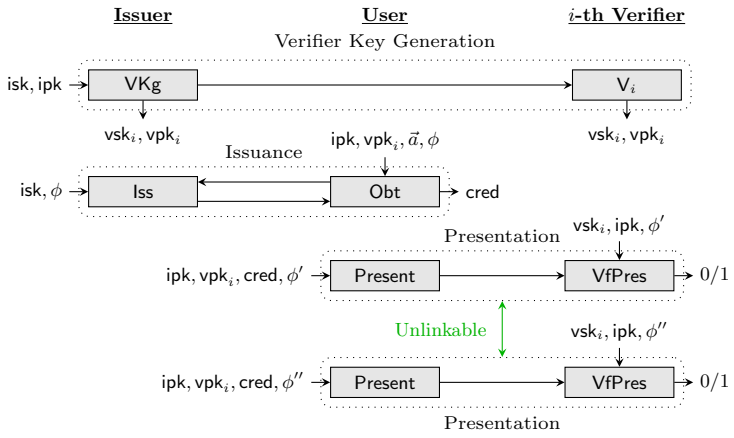

Fig. 1. Illustration of our mKVAC setting, supporting multiple verifiers and enabling multi-show unlinkable credential presentation.

gle credential. Both ARC proposals already undergo standardization in the IETF [YW25, FS25].

Clearly, the reliance of a shared secret key between the verifier and issuer significantly limits the scenarios where KVACs can be deployed, and makes them unsuitable in open settings such as eID systems. In fact, even in closed enterprise environments as the Google and Apple deployments, relying on a secret key that needs to be highly replicated among all verifying entities does not seem ideal from a security perspective. The high availability needed for verification puts the secret key at risk that provides the foundation for the unforgeability of the entire authentication system.

KVACs for Multiple Verifiers. The fundamental drawback of KVACs is that the issuer and the verifier need to share a secret key. This raises the question of how to handle settings with *multiple* verifiers, e.g., in an eID scheme where one government agency issues credentials but different authorities and online services must verify them. A naïve approach would be for all verifiers to share the issuer's secret key. This is clearly insecure as the compromise of a single, less protected verifier would allow forging arbitrary credentials. A more reasonable alternative is for the issuer to maintain a separate KVAC key for each verifier. This way, verifiers cannot forge credentials for each other, and a leaked verifier key only impacts that one verifier. This makes credential issuance verifier-specific, i.e. whenever the user encounters a new verifier, they need to obtain a new credential specifically for this verifier. This immediately destroys a fundamental privacy property of anonymous credentials: the issuer learns the services the users wants to access, essentially defeating much of the purpose of using anonymous credentials in the first place.

1.1 Our Contributions

In this paper, we push the boundaries of anonymous credentials that can be realized without pairings by introducing *multi-verifier keyed-verification anonymous credentials* (mKVACs), essentially extending KVACs to the multi-verifier setting. Our goal is to preserve the multi-show unlinkability property of KVACs while avoiding the privacy issues associated with verifier-specific issuance outlined above. We formally define mKVAC and its desired security and privacy guarantees. We then give a pairing-free construction that can be realized from variants of well-understood primitives such as CMZ [CMZ14] and BBS [BBS04]. In more detail we provide the following contributions:

Formal Model for mKVAC*s.* We start by formally defining this new type of keyed-verification credentials, supporting multiple verifiers. The overall mKVAC setting is illustrated in Figure 1. In mKVAC the issuer still maintains a single short secret key isk. With it, the issuer can derive many verifier key pairs, $(\mathsf{vpk}_i, \mathsf{vsk}_i)$, and we imagine verifiers registering with the issuer to obtain their key. This registration process for verifiers, i.e., relying parties, is part of the eIDAS 2.0 specifications [Eur24, Article 5b], and is also a common pattern in legacy protocols such as OpenID Connect. The user can obtain a verifier-specific credential from the issuer, certifying her attributes along the verifier public key vpk_i. Importantly, the verifier key vpk_i must remain hidden towards the issuer. The user can then use such a verifier-specific credential to prove a predicate over her attested attributes to the verifier associated with vpk_i, who uses its secret key vsk_i to verify the presentation. This process can be safely repeated multiple times using the same credential, and does not require any further interaction with the issuer.

More precisely, we formalize the following security and privacy properties for mKVACs: **Unforgeability** covering the expected unforgeability of attributes and presentation. Crucially, compromise of any verifier key vsk_i does not impact the unforgeability guarantees for any verifier $i' \neq i$. **Presentation anonymity** ensuring the standard unlinkability and privacy guarantees expected from anonymous credentials, in particular *multi-show unlinkability.* **Issuance anonymity** guarantees that the issuer does not learn anything about the verifier the user requests a credential for (and hides additional attributes, as our scheme supports partially-blind issuance too). Further, we leverage the fact that we are in the keyed-verification setting to express an additional property that is desirable in the context of identity systems but usually considered out-of-scope: **Deniability**, ensuring that a verifier cannot provide convincing evidence to a third party that a given presentation was produced by using a legitimate credential.

Pairing-Free Constructions. We present two practical mKVAC constructions based on the efficient BBS and CMZ frameworks, offering different trade-offs between communication costs and privacy guarantees. We formally prove the security of our constructions under standard cryptographic assumptions. We demonstrate that they are practical and efficient for real-world applications,

through a prototypical implementation. For a credential with 8 attributes, presenting and verification can be run in less than 1.2ms for each party, and generating proofs of size 1KB.

Our constructions are given in a semi-generic manner and rely on an additional building block we formally introduce first and we believe to be of independent interest: SAGA, an algebraic MAC on vectors of group elements with a special KVAC-style presentation mechanism. This will be used to authenticate the verifiers' public key, which the user can then use to blindly obtain credentials on a previously authenticated vpk. We formalize this building block and provide instantiations relying on variants of BBS and CPZ MACs. In the following, we give a high-level idea of our constructions before presenting our formal mKVAC and SAGA models, as well as concrete instantiations for both.

1.2 Technical Outline

We now present the high-level blueprint of our mKVAC construction and explain the challenges we had to overcome. One important constraint for our construction is that the issuer should not have to do work linear in the number of verifiers every time a credential is issued. This is because issuers may be restricted in hardware (e.g., HSMs) and we want to support large numbers of verifiers. Without additional assumptions (such as non-colluding servers), this rules out approaches based on private information retrieval or similar techniques.

High-Level Idea. Instead, our approach is to build mKVAC by combining two KVAC mechanisms in a nested way. One layer is used to authenticate the verifier's public key, and the other is used to issue credentials bound to that authenticated key. To prevent the issuer from directly storing a separate secret key for every verifier, each verifier embeds an encrypted form of its signing key into its public key. The issuer holds the matching decryption key and can recover just enough information during issuance to produce a valid credential, without learning which verifier is involved. In practice, the user sends a blinded version of the verifier's public key to the issuer, the issuer applies its decryption key to obtain the necessary signing component, and then completes issuance of a blinded credential. This design keeps the issuer's role lightweight, while each verifier still holds the secret needed to check credentials.

Challenges to Overcome. However, we first need to address several challenges with this approach. First, hiding the signing key in the verifier's public key (in some randomizable/blindable form) seems to imply that the signing must be a group element. Thankfully, there is one KVAC system that supports issuance using a group element signing key: the CMZ scheme [CMZ14]. For this reason, we base our mKVAC construction on CMZ and refer to this variant as $\mathsf{mKVAC}_{\mathsf{CMZ}}$.

Second, the CMZ issuance protocol involves the user computing a Pedersen commitment C_a to their desired attributes $a_1, \ldots, a_n$ on a basis $(X_1, \ldots, X_n) \in \mathbb{G}^n$, derived from the verifier's secret key. Crucially, the integrity of this basis must be protected. If, for example, the user were to use the basis $(2X_1, \ldots, 2X_n)$

for the commitment, they can effectively receive a credential for attributes $2a_1, \ldots, 2a_n$, which may not satisfy the issuance policy. This leads to a need to authenticate the basis (alongside the other parts of the verifier's public key) and to prove in zero-knowledge that C_a is a valid commitment on *hidden* attributes $\vec{a}$ on a *hidden* basis $\vec{X}$ that is *authenticated*. Such statements (where both the basis and the message of a Pedersen commitment are hidden) are notoriously difficult to prove using efficient Sigma protocols.

Need for a new primitive: SAGA. We solve this problem by introducing *symmetric anonymous group element authenticator*s (SAGA), an algebraic MAC on vectors of group elements with a special KVAC-style presentation mechanism. SAGA is used to authenticate the verifier's commitment basis $\vec{X}$ and its presentation is designed to hide $\vec{X}$ while enabling efficient Sigma protocol proofs such as the commitment proof for C_a above. However, constructing SAGA presents another challenge: while there are MACs on group elements with a suitable presentation mechanism to instantiate SAGA (for example the CPZ MAC [CPZ20]), their privacy property depends on computational assumptions that might be broken by future quantum computers. Ideally, we want our mKVAC construction remain secure against attackers that harvest today's traffic to attack its privacy using quantum computers in the future. Towards this goal, we design a new MAC on group elements, based on BBS signatures [BBS04, TZ23]. Note that while there are pairingless BBS-based MACs [BBDT16, Orr24] and KVACs, they only authenticate $\mathbb{Z}_p$ scalars rather than group elements. Our SAGA construction enjoys security against harvest-now-decrypt-later attacks. We prove it secure in the GGM.

Finally, we construct the issuance protocol of $\mathsf{mKVAC}_{\mathsf{CMZ}}$ by using SAGA to ensure that the user is using a valid verifier commitment basis for the CMZ issuance protocol while statistically hiding the verifier's public key. To make this construction efficient, we solve several technical and optimization problems. See Section 5 for details on our final construction.

1.3 Related Work

Our mKVAC scheme is positioned somewhere in between KVACs and publicly verifiable anonymous credentials. They preserve the pairing-freeness of KVACs, but cater for multiple (yet registered) verifiers getting closer to an open setting.

Another noteworthy recent work that aims at providing pairing-free anonymous credentials are *server-aided anonymous credentials* (SAACs) [CAHLT25]. Their approach also implicitly relies on KVACs but is entirely orthogonal to our work: they aim at public-verifiability, whereas we are still in a privately-verifiable setting, but support multiple verifiers. SAAC achieves such public verifiability by sacrificing multi-show unlinkability. That is, while the user is in possession of a KVAC credential, she also needs to get a helper token from the issuer for every unlinkable presentation she wants to derive from that credential. This token essentially transforms the private-verifiable credential into a publicly verifiable presentation with the help of an always-online helper server. Thus, in terms of

Table 1. Comparison of Anonymous Credential and Related Schemes. The number of attributes is n. Credential size excludes storage of attributes. $\mathbb{G}$: size of group elements. $\mathbb{Z}_p$: size of scalars. All security analyses assume the ROM. Numbers heuristically assume that Fiat-Shamir is straightline extractable. *: Showing requires helper server interaction which can be performed offline and batched. †: Multi-Verifier. See Table 2 for a full comparison.

Scheme	Pub. Ver.	Multi-Show	Pairing-Free	Cred.	Pres.	Assum.	Anon.
Traditional Anonymous Credentials							
CL01/03 [CL01]	✓	✓	✓	$O(1)$	$O(n)$	Strong RSA	Comp.
ACL [BL13]	✓	×	✓	$2\mathbb{G} + 6\mathbb{Z}_p$	$2\mathbb{G} + (n+8)\mathbb{Z}_p$	AGM+DL	Stat.
BBS04 [BBS04]	✓	✓	×	$1\mathbb{G}_1 + \mathbb{Z}_p$	$2\mathbb{G}_1 + (n+3)\mathbb{Z}_p$	q-SDH	Stat.
Brands [Bra95]	✓	×	✓	$O(n)$	$O(n)$	DL	Comp.
SNARK-based							
LongFellow [Fs24]	✓	✓	✓	$O(n)$	$O(\mathsf{polylog}(n))$	Sum-check	Stat.
Semaphore [Eth19]	✓	✓	×	$O(\log n)$	$O(1)$	SNARKs	Comp.
KVAC							
CMZ14 [CMZ14]	×	✓	✓	$2\mathbb{G}$	$(n+2)\mathbb{G} + (2n+2)\mathbb{Z}_p$	GGM	DDH
DGSTV18 [DGS+18]	×	✓	✓	$2\lambda + \mathbb{G}$	4λ	gap-OMCDH	Stat.
BBDT16 [BBDT16]	×	✓	✓	$2\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n+7)\mathbb{Z}_p$	Gap-q-SDH	Stat.
CDDH19 [CDDH19]	×	✓	✓	$(n+1)\mathbb{G}$	$2\mathbb{G} + (n+1)\mathbb{Z}_p$	n-SCDHI	Stat.
CPZ20 [CPZ20]	×	✓	✓	$2\mathbb{G} + \mathbb{Z}_p$	$(n+3)\mathbb{G} + 4\mathbb{Z}_p$	GGM	DDH
μCMZ [Orr24]	×	✓	✓	$2\mathbb{G}$	$(n+2)\mathbb{G} + (2n+2)\mathbb{Z}_p$	AGM+3-DL	Stat.
μBBS [Orr24]	×	✓	✓	$1\mathbb{G} + \mathbb{Z}_p$	$2\mathbb{G} + (n+4)\mathbb{Z}_p$	AGM+q-DL	Stat.
MBS+25 [MBS+25]	×	✓	✓	$(n+2)\mathbb{G}$	$2\mathbb{G}$	GGM	Stat.
SAAC							
SAAC_{BBS} [CAHLT25]	✓	×*	✓	$1\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n+8)\mathbb{Z}_p$	Gap-q-SDH	Stat.
SAAC_{DDH} [CAHLT25]	✓	×*	✓	$4\mathbb{G}$	$(n+6)\mathbb{G} + (4n+11)\mathbb{Z}_p$	DDH	DDH
Multi-Verifier KVAC (This Work)							
mKVAC	✓†	✓	✓	$2\mathbb{G}$	$(n+3)\mathbb{G} + (2n+4)\mathbb{Z}_p$	GGM	Stat.

public verifiability, SAACs are clearly more powerful than our mKVACs. However, SAAC inherits the same batch issuance challenges currently faced in the EUDI context with batch-issued ECDSA credentials and Privacy Pass. In fact, seeing that companies such as Google and Apple that deployed Privacy Pass at scale, are now replacing their batch-issuance based implementation that was publicly verifiable and had eternal privacy through a KVAC-based scheme with DL-based privacy, demonstrates the need for solutions that have multi-show unlinkability build in – which is what our solution provides.

Table 1 provides a comprehensive comparison of various anonymous credential schemes and related systems. We discuss additional related work and give a broader overview in Appendix A.

2 Preliminaries

Notation. We introduce some notation that we use throughout the paper. We notate a vector as $\vec{a}$ and the ith element of the vector as a_i. Given $\vec{a}, \vec{b} \in \mathbb{Z}_p^n$, $\vec{X}, \vec{Y} \in \mathbb{G}^n$, and $Z \in \mathbb{G}$: $\vec{a} \cdot \vec{b}$ and $\vec{a} \circ \vec{b}$ gives the inner product and the entrywise product of the two vectors, respectively. Similarly $\vec{a} \cdot \vec{X} := \sum_{i \in [n]} a_i X_i$, $\vec{a} \circ \vec{X} := (a_i X_i)_{i \in [n]}$, and $\vec{a} Z := (a_i Z)_{i \in [n]}$. In our game descriptions, **require** x is a shortcut for **if** $\neg x$: **return** $\bot$.

Groups. We rely on pairing-free prime order groups with the description $(\mathbb{G}, p, G)$ such that $\langle G \rangle = \mathbb{G}$ and $|\mathbb{G}| = p$ throughout the paper. We describe a group generator $\mathsf{GGen}(\lambda)$ that outputs a group description with $|p| > 2\lambda$. Throughout the paper we are going to rely on the two algebraic assumptions on groups which we sketch below and formally define in Appendix B: (1) the DDH Assumption: Given $(\mathbb{G}, p, G, A := aG, B := bG, C)$ for $a, b \xleftarrow{\$} \mathbb{Z}_p$, it is hard for an adversary to distinguish whether $C = abG$ or $C \xleftarrow{\$} \mathbb{G}$. (2) n-DL Given $(\mathbb{G}, p, G, (A_i := (a^i)G)_{i \in [n]})$ for $a \xleftarrow{\$} \mathbb{Z}_p$, it is hard for an adversary to output a.

NIZK PoK. Throughout the paper we will rely on Fischlin-transformed Sigma protocol which gives straightline simulation extractability in ROM [Fis05, Ks22, LR22]. Below we present the definitions for a straightline-extractable NIP by Lysyanskaya and Rosenbloom in our syntax [LR22]. The only change we make is the addition of a proof context $\mathsf{ctx}_{\mathsf{NIP}}$ as input to proving and verifying algorithms. This allows binding additional information to each proof. On the construction level, $\mathsf{ctx}_{\mathsf{NIP}}$ becomes additional input to the Fischlin hash function.

Definition 1. *A* $\mathsf{NIP}_{\mathcal{R}}$ *scheme for a relation* $\mathcal{R}$ *is a tuple of algorithms s.t.:*

$\mathsf{Setup}(\lambda) \to \mathsf{pp}_{\mathsf{NIP}}$**:** Outputs the public parameters for input security parameter.
$\mathsf{Prove}(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}}) \to \pi$**:** Given $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, outputs a proof π for the context $\mathsf{ctx}_{\mathsf{NIP}}$.
$\mathsf{Vf}(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi) \to b \in \{\mathsf{true}, \mathsf{false}\}$**:** It verifies the proof π against the statement $\mathbb{x}$ for the context $\mathsf{ctx}_{\mathsf{NIP}}$.

Below we give the informal descriptions of the NIP properties that we rely on, and present their formal definitions in Appendix B.

Completeness. $\mathsf{NIP}_{\mathcal{R}}$ has completeness if for all $\mathsf{pp}_{\mathsf{NIP}} \leftarrow \mathsf{Setup}(\lambda)$, $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, $\mathsf{ctx}_{\mathsf{NIP}} \in \{0,1\}^*$, and $\pi \leftarrow \mathsf{Prove}(\mathbb{x}, \mathbb{w})$, $\Pr[\mathsf{Vf}(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi) = \mathsf{true}] = 1$.

Zero-knowledge. $\mathsf{NIP}_{\mathcal{R}}$ is zero-knowledge if there exists a PPT simulator $\mathsf{S} := (\mathsf{S.Setup}, \mathsf{S.Prove})$, s.t. for all adversaries, it is hard to distinguish honestly generated $\mathsf{pp}_{\mathsf{NIP}}$ and π from the simulated ones through S.Setup and S.Prove.

Simulation-extractability. $\mathsf{NIP}_{\mathcal{R}}$ is simulation-extractable if there exists an extractor Extract which can extract a valid witness from all adversarially created verifying proofs even when the adversary can see simulated proofs.

$\underline{\mathsf{CMZ.Setup}(\lambda, n)}$
$(\mathbb{G}, p, G) \leftarrow \mathsf{GGen}(\lambda)$
return $\mathsf{pp} := (\mathbb{G}, p, G)$

$\underline{\mathsf{CMZ.Kg}(\mathsf{pp})}$
$x_0 \xleftarrow{\$} \mathbb{Z}_p, \vec{x} \xleftarrow{\$} \mathbb{Z}_p^n$
return $(\mathsf{sk} := (x_0, \vec{x}), \mathsf{pk} := \vec{x}G)$

$\underline{\mathsf{CMZ.Sign}(\mathsf{sk}, \vec{m})}$
parse sk **as** $(x_0, \vec{x})$
$U \xleftarrow{\$} \mathbb{G}$
$V := (x_0 + \vec{x} \cdot \vec{m})U$
return $\sigma := (U, V)$

$\underline{\mathsf{CMZ.Vf}(\mathsf{sk}, \sigma, \vec{m})}$
parse σ **as** (U, V)
parse sk **as** $(x_0, \vec{x})$
return $\begin{pmatrix} V = (x_0 + \vec{x} \cdot \vec{m})U \\ \wedge\ U \neq 0 \end{pmatrix}$

Fig. 2. Description of CMZ MAC [CMZ14].

Throughout the paper, we give the relations that we wish to be proven without immediately instantiating the exact sigma protocol for them. However, the traditional techniques can be used to write down the concrete sigma protocols for these relations. We concretely instantiate our protocols Appendix C using standard techniques. Note that, to give a straightline extractor with Fischlin transform, these sigma protocols must satisfy strong special soundness [Ks22, LR22]. We discuss this in Section C.3.

MAC Definition and CMZ *MAC.* A $\mathsf{MAC} := (\mathsf{Setup}, \mathsf{Kg}, \mathsf{Sign}, \mathsf{Vf})$ scheme is a tuple of algorithms such that $\mathsf{Setup}(\lambda) \to \mathsf{pp}$ outputs the public parameters, $\mathsf{Kg}(\mathsf{pp}) \to (\mathsf{sk}, \mathsf{pk})$ outputs a key pair, $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$ outputs a tag σ on message m, and $\mathsf{Vf}(\mathsf{sk}, \sigma, m)$ verifies whether σ is a valid tag on message m. We recap the CMZ MAC scheme by Chase et al. [CMZ14] in Figure 2 and its unforgeability game in Appendix B.

3 Definition of mKVAC

An mKVAC system has three main entities: issuer, users and verifiers. In contrast to classic keyed-verification (KVAC) schemes, where the issuer and verifier are the same entity sharing a single secret key, our mKVAC version supports *multiple* verifiers. That is, verification is still a keyed operation, but now takes a verifier-specific secret key as input. This enables a more distributed setting than classic KVAC. Issuance is still done by the single issuer, and users must acquire credentials for the targeted verifiers. The challenge therein is to hide the verifier's identity from the issuer during issuance. This is what we capture through the notation of *issuance anonymity*, that complements the standard notions of *unforgeability* and *presentation anonymity*. As mKVAC targets a more open and distributed setting than classic KVACs, but still works in a keyed-verification setting, we further leverage this keyed aspect to demand *deniability* of the presented information. We begin by specifying the syntax of mKVAC schemes, and then formalize these desired security guarantees through game-based definitions.

3.1 Syntax

We start by explaining the syntax of mKVAC along its three main phases: setup and key generation, issuance of credentials and presentations, followed by the formal definition of all algorithms.

Setup and Keys. The Setup algorithm initializes the public parameters, which also determine the set of allowed predicates ϕ used throughout the scheme. The issuer generates its key pair $(\mathsf{isk}, \mathsf{ipk})$ by running IKg. For each verifier, the issuer additionally generates a verifier key pair $(\mathsf{vsk}, \mathsf{vpk})$ by running VKg on isk. Recall that we are still in a keyed-verification setting, i.e., such a dependency from the issuer's *secret* key is needed here.

(Partially-blind) Issuance. Credential issuance is a two-move protocol between the issuer and a user, captured through the algorithms Iss run by the issuer, and Obt_1, Obt_2 run by the user. The user initiates the protocol on input the issuer and verifier public keys $\mathsf{ipk}, \mathsf{vpk}$, the attribute vector $\vec{a}$, and a predicate ϕ which is satisfiable by $\vec{a}$. The issuer's inputs are it's secret key isk and the predicate ϕ but not the complete attribute vector $\vec{a}$ and not the vpk. Upon successful completion of issuance, the user obtains a valid credential cred for $\vec{a}$ and vpk.

Note that our protocol does not only hide the verifier's identity vpk from the issuer, but also supports partially blind issuance. Our model follows the recent works [Orr24, CAHLT25], by providing the issuer only with a predicate ϕ that is satisfied by the user's attributes, instead of the attributes directly. This abstraction enables a wide range of applications and extensions. For instance, our scheme naturally supports pseudonyms, where the user obtains a credential on a secret value that later seeds a PRF, without disclosing this key to the issuer. Likewise, it enables blind attribute transfer, allowing a user to prove possession of attributes from an existing credential and obtain a new credential on the same attributes without revealing them. This is particularly useful when transferring credentials to new verifiers or updating them for existing attributes [BBDE19]. We can imagine users holding a source credential on attributes $\vec{a}$, where the issuer is the verifier. When requesting a credential for a new verifier, the user would transfer the attributes $\vec{a}$ to the new credential without revealing them to the issuer. This enables even more flexible security: the issuer is fully blind and does not learn anything about this reissuance process (neither what verifier the user wants to use nor what attributes the user has).

Presentation. To create a credential presentation pres, the user runs Present on a predicate ϕ and a presentation context ctx, that it wants to share with the verifier. This algorithm additionally takes the public keys $\mathsf{ipk}, \mathsf{vpk}$ of the issuer and targeted verifier, as well as a credential cred that is valid for ϕ and vpk as input. The presentation context ctx can contain a nonce, or any other information the presentation shall be bound to, preventing replay attacks. Finally, a verifier runs VfPres using its secret key vsk to check the validity of a presentation pres on a predicate ϕ for a presentation context ctx.

Definition 2 (Multi-Verifier KVAC). *An* mKVAC *scheme is defined through the following algorithms:*

$\mathsf{Setup}(\lambda, n) \to \mathsf{pp}$**:** For a security parameter λ and the attribute vector size n, it outputs the public parameters pp to issue credentials on attribute vectors from $\mathbb{A}^n$. We assume that pp fixes the set of allowed predicates $\phi : \mathbb{A}^n \to \{\mathsf{true}, \mathsf{false}\}$ during the issuance and presentation. We only make pp an explicit input to IKg and treat it as an implicit input to all other algorithms.

$\mathsf{IKg}(\mathsf{pp}) \to (\mathsf{isk}, \mathsf{ipk})$**:** Outputs an issuer key pair $(\mathsf{isk}, \mathsf{ipk})$.

$\mathsf{VKg}(\mathsf{isk}) \to (\mathsf{vsk}, \mathsf{vpk})$**:** Given isk, it outputs a verifier key pair $(\mathsf{vsk}, \mathsf{vpk})$.

$\mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}, \vec{a}, \phi) \to (\mathsf{creq}, \mathsf{st})$**:** For an issuer public key ipk, a verifier public key vpk, vector of attributes $\vec{a} \in \mathbb{A}^n$, and a credential predicate ϕ, it outputs a credential request creq and issuance state st.

$\mathsf{Iss}(\mathsf{isk}, \phi, \mathsf{creq}) \to \mathsf{bcred}/\perp$**:** For an issuer secret key isk, a credential request creq, and a predicate ϕ, it outputs a blinded credential bcred if the issuance is successful, and $\perp$ otherwise.

$\mathsf{Obt}_2(\mathsf{st}, \mathsf{bcred}) \to \mathsf{cred}/\perp$**:** Given an issuance state st and a blind credential bcred, it outputs the finalized credential cred if the issuance is successful, and $\perp$ otherwise.

$\mathsf{Present}(\mathsf{ipk}, \mathsf{vpk}, \mathsf{cred}, \mathsf{ctx}, \phi) \to \mathsf{pres}$**:** For an issuer and verifier public keys ipk and vpk, a credential cred, a presentation context ctx, and a credential predicate ϕ, it outputs a credential presentation pres.

$\mathsf{VfPres}(\mathsf{vsk}, \mathsf{ipk}, \mathsf{pres}, \mathsf{ctx}, \phi) \to b \in \{\mathsf{true}, \mathsf{false}\}$**:** For a verifier secret key vsk, an issuer public key ipk, a presentation pres, a presentation context ctx, and a credential predicate ϕ, it verifies the credential.

The correctness definition is given in Appendix F for completeness. Beyond correctness, we aim at the following high-level properties:

Unforgeability: An adversary interacting with an honest issuer and verifier must not be able to generate a fresh and valid credential presentation for an honest verifier under a predicate that cannot be satisfied with the adversary's corrupted credentials. This must also hold if the adversary can corrupt other verifiers in the mKVAC system.

Presentation Anonymity: Presentations must not reveal any information beyond the disclosed predicate and verifier. In particular, given two presentations for the same policy and verifier, the adversary should be unable to distinguish whether they originate from the same credential or not.

Issuance Anonymity: During credential issuance, the issuer must not learn information about the intended verifier or the attributes beyond what the predicate reveals.

Deniability: A verifier cannot provide convincing evidence to a third party that a given presentation was produced by using a legitimate credential.

3.2 Unforgeability

Unforgeability requires that the adversary must not be able to present predicates for which they do not have an underlying credential. While the issuer

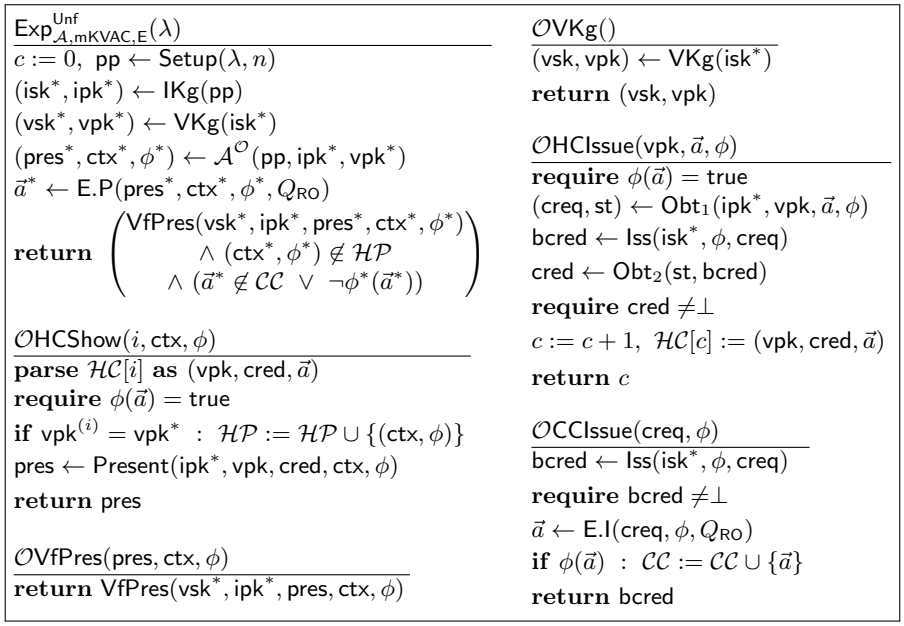

Fig. 3. Unforgeability Game for mKVAC.

is assumed to be honest for unforgeability, we aim at more granularity for the verifiers. Namely, our mKVAC supports *multiple* verifiers where all but one may be compromised. The adversary's goal is to forge a valid presentation for the remaining honest verifier. Our full definition is given in Figure 3, and closely follows the recent model by Orrú [Orr24], adapted to our multi-verifier setting. The game is parameterized through an extractor $\mathsf{E} := (\mathsf{I}, \mathsf{P})$, which we use to extract hidden attributes $\vec{a}$ during issuance and from the forged presentation. The adversary receives the public keys $\mathsf{ipk}^*, \mathsf{vpk}^*$ of the honest issuer and verifier as input, and can interact with all honest parties through a number of oracles before outputting its alleged forgery. To model the aforementioned corruption of other verifiers, the adversary can request verifier key pairs $(\mathsf{vsk}, \mathsf{vpk})$ through the oracle $\mathcal{O}\mathsf{VKg}$. For the honest challenge verifier, the adversary is granted the oracle $\mathcal{O}\mathsf{VfPres}$ that models keyed-verification with the honest vsk^*.

The adversary can obtain credentials and presentations through a number of oracles. When acting in the role of corrupt users, they can request credentials on blinded inputs through the $\mathcal{O}\mathsf{CCIssue}$ oracle. We obtain the attributes $\vec{a}$ from the blinded request creq using the extractor $\mathsf{E.I}$. The game adds the extracted $\vec{a}$ to $\mathcal{CC}$, the set of corrupted credentials, for bookkeeping of trivial wins. Note that we do *not* extract the vpk and treat all credential requests as targeting the challenge verifier. We opt for this definitional choice as having credentials designated to a verifier is a limitation of pairing-free setting rather than a feature. Honest users are simulated through two oracles. First, $\mathcal{O}\mathsf{HCIssue}$ simulates the issuance of credentials for attributes $\vec{a}$ and a verifier vpk of the adversary's choice. The

oracle internally runs the issuance, and, upon successful completion, stores the credential along with $\vec{a}$ and vpk. Unlike corrupt users, the adversary does not learn the issued credential but instead can ask for presentations thereof through the $\mathcal{O}\mathsf{HCShow}$ oracle. The adversary can request presentations for predicates and contexts of their choice, and presentations for the challenge vpk^* are stored for bookkeeping of trivial wins in $\mathcal{HP}$.

Eventually, the adversary ends by outputting a forgery $(\mathsf{pres}^*, \mathsf{ctx}^*, \phi^*)$, and wins if it is valid under vsk^* and non-trivial. For the latter, we use the extractor $\mathsf{E.P}$ to extract an attribute vector $\vec{a}^*$ from the adversaries output. The tuple $\vec{a}^*, \mathsf{ctx}^*, \phi^*$ is considered to be non-trivial if the following holds: (1) No honest presentation on ϕ^* and context ctx^* exists in $\mathcal{HP}$ (implying a forged presentation for an honest user credential). (2) Either $\vec{a}^*$ is not in the corrupted credentials set $\mathcal{CC}$ (implying a forged credential), or $\vec{a}^*$ does not satisfy the presentation predicate ϕ^* (implying a forged presentation).

Definition 3. *An* mKVAC *scheme is unforgeable for an extractor* E *if for all PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{Unf}}_{\mathsf{mKVAC},\mathcal{A},\mathsf{E}}(\lambda) := \Pr\left[\mathsf{Exp}^{\mathsf{Unf}}_{\mathcal{A},\mathsf{mKVAC},\mathsf{E}}(\lambda) = \mathsf{true}\right] \leq \mathsf{negl}(\lambda)$.

Remarks on Extractor and ROM. Our definition is explicitly given in the ROM. We assume all algorithms and the adversary have access to a random oracle as in Figure 15 in Appendix B, and do not restate its definition here. The extractors take as input the set Q_{RO} of all random oracle queries made thus far to align with straight-line extractors obtained via the Fischlin transform.

We further stress that extractor-based unforgeability is necessary for a meaningful security notion under blind issuance, and our model follows the extractor-based definitions from previous works [Orr24, CAHLT25, CKL+14]. Without extracting the blinded attributes during issuance, the adversary could obtain a credential from $\mathcal{O}\mathsf{CCIssue}$ under a trivially true predicate without revealing any information about attributes, which would render *all* presentations as trivial.

3.3 Presentation Anonymity

The core privacy property of anonymous credentials ensures that presentations do not reveal any information beyond their corresponding predicate and designated verifier. We model presentation anonymity via an indistinguishability game in Figure 4. The issuer and all verifiers are corrupt here, and the challenger represents honest users. The adversary can trigger honest users to obtain credentials for attributes of $\mathcal{A}$'s choice, through the oracles $\mathcal{O}\mathsf{Obt}_1$ and $\mathcal{O}\mathsf{Obt}_2$. Note that $\mathcal{O}\mathsf{Obt}_2$ reveals the resulting credential cred to the adversary, so presentation anonymity does not rely on credential secrecy. For registered credentials, the adversary may request presentations on arbitrary predicates through the oracle $\mathcal{O}\mathsf{Present}$. The challenge itself is provided by the left-or-right oracle $\mathcal{O}\mathsf{LoR}$: given two credential identifiers $c^{(0)}$ and $c^{(1)}$, and a predicate ϕ, the oracle returns the presentation $\mathsf{pres}^{(b)}$ derived from $c^{(b)}$. Trivial wins are excluded by checking that both credentials satisfy ϕ and they are issued for the same vpk. At the end of the game, the adversary outputs a guess for b and wins if the guess is correct.

$\underline{\mathsf{Exp}^{\mathsf{PAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda)}$
$b \leftarrow \{0,1\},\ c := 0$
$\mathsf{pp} \leftarrow \mathsf{Setup}(\lambda, n)$
$\mathsf{ipk} \leftarrow \mathcal{A}(\mathsf{pp})$
$b' \leftarrow \mathcal{A}^{\mathcal{O}}()$
return $b = b'$

$\underline{\mathcal{O}\mathsf{Obt}_1(\mathsf{vpk}, \vec{a}, \phi)}$
require $\phi(\vec{a}) = \mathsf{true}$
$c := c + 1$
$(\mathsf{creq}, \mathsf{st}) \leftarrow \mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}, \vec{a}, \phi)$
$\mathcal{HR}[c] := (\mathsf{st}, \vec{a}, \mathsf{vpk})$
return (c, creq)

$\underline{\mathcal{O}\mathsf{Obt}_2(c', \mathsf{bcred})}$
require $c' \leq c \ \wedge\ \mathcal{HC}[c'] = \perp$
parse $\mathcal{HR}[c']$ **as** $(\mathsf{st}, \vec{a}, \mathsf{vpk})$
$\mathsf{cred} \leftarrow \mathsf{Obt}_2(\mathsf{st}, \mathsf{bcred})$
require $\mathsf{cred} \neq \perp$
$\mathcal{HC}[c'] := (\mathsf{cred}, \vec{a}, \mathsf{vpk})$
return cred

$\underline{\mathcal{O}\mathsf{Present}(c', \phi)}$
require $\mathcal{HC}[c'] \neq \perp$
parse $\mathcal{HC}[c']$ **as** $(\mathsf{cred}, \vec{a}, \mathsf{vpk})$
require $\phi(\vec{a}) = \mathsf{true}$
$\mathsf{pres} \leftarrow \mathsf{Present}(\mathsf{ipk}, \mathsf{vpk}, \mathsf{cred}, \mathsf{pres})$
return pres

$\underline{\mathcal{O}\mathsf{LoR}(c^{(0)}, c^{(1)}, \phi)}$
require $\mathcal{HC}[c^{(0)}] \neq \perp \ \wedge\ \mathcal{HC}[c^{(1)}] \neq \perp$
parse $\mathcal{HC}[c^{(0)}]$ **as** $(\mathsf{cred}^{(0)}, \vec{a}^{(0)}, \mathsf{vpk}^{(0)})$
parse $\mathcal{HC}[c^{(1)}]$ **as** $(\mathsf{cred}^{(1)}, \vec{a}^{(1)}, \mathsf{vpk}^{(1)})$
require $\mathsf{vpk}^{(0)} = \mathsf{vpk}^{(1)} \ \wedge\ \phi(\vec{a}^{(0)}) \ \wedge\ \phi(\vec{a}^{(1)})$
return $\mathsf{pres}^{(b)} \leftarrow \mathsf{Present}(\mathsf{ipk}, \mathsf{vpk}^{(b)}, \mathsf{cred}^{(b)}, \phi)$

Fig. 4. Presentation Anonymity Game for mKVAC.

$\underline{\mathsf{Exp}^{\mathsf{IAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda)}$
$b \xleftarrow{\$} \{0,1\},\ \mathsf{pp} \leftarrow \mathsf{Setup}(\lambda, n)$
$\left(\mathsf{ipk}, \phi, (\mathsf{vpk}^{(i)}, \vec{a}^{(i)})_{i\in\{0,1\}}\right) \leftarrow \mathcal{A}(\mathsf{pp})$
for $i \in \{0,1\}$: $(\mathsf{creq}^{(i)}, \mathsf{st}^{(i)}) \leftarrow \mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}^{(i)}, \vec{a}^{(i)}, \phi)$
$(\mathsf{bcred}^{(b)}, \mathsf{bcred}^{(1-b)}) \leftarrow \mathcal{A}(\mathsf{creq}^{(b)}, \mathsf{creq}^{(1-b)})$
for $i \in \{0,1\}$: $\mathsf{cred}^{(i)} \leftarrow \mathsf{Obt}_2(\mathsf{st}^{(i)}, \mathsf{bcred}^{(i)})$
if $\neg\phi(\vec{a}^{(0)}) \ \vee\ \neg\phi(\vec{a}^{(1)}) \ \vee\ \perp \in \{\mathsf{creq}^{(0)}, \mathsf{creq}^{(1)}, \mathsf{cred}^{(0)}, \mathsf{cred}^{(1)}\}$: $b' \xleftarrow{\$} \{0,1\}$
else : $b' \leftarrow \mathcal{A}(\mathsf{cred}^{(0)}, \mathsf{cred}^{(1)})$
return $b = b'$

Fig. 5. Issuance Anonymity Game for mKVAC.

Definition 4. *An* mKVAC *scheme is presentation anonymous if for all PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{PAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{PAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) = \mathsf{true}\right] - 1/2\right| \leq \mathsf{negl}(\lambda)$.

Remarks. Our model provides the adversary with strong powers throughout the game, in particular it allows the adversary to choose the issuer public key (whereas e.g., Orru and Chase et al. [Orr24, CMZ14] assume honest key generation), and also explicitly reveals the generated credentials to $\mathcal{A}$, ensuring that users' privacy does not require secrecy of their credentials. We compare these choices with related work in Appendix E.

3.4 Issuance Anonymity

Our second privacy notion targets the issuance phase. Beyond the blind issuance aspect, ensuring that malicious issuers do not learn any information about $\vec{a}$

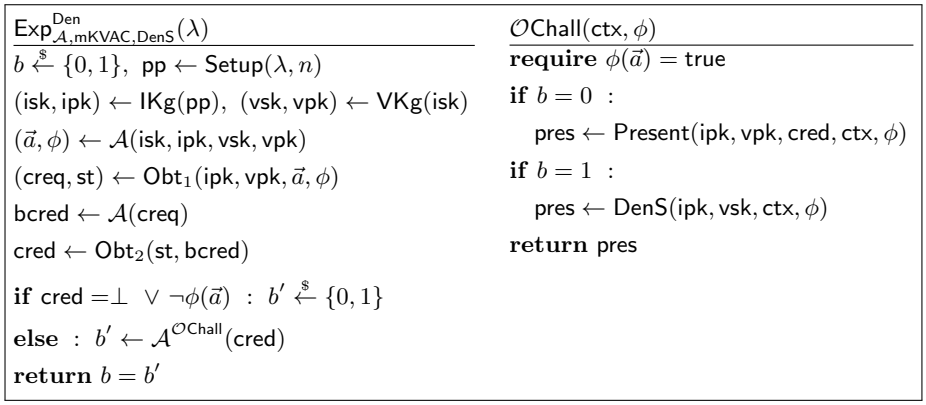

Fig. 6. Deniability Game for mKVAC.

beyond what is revealed through ϕ, it also ensures that the verifier's identity vpk remains hidden. The latter is important to not trade our increased flexibility of supporting multiple verifiers with reduced privacy for the user. In fact, without such anonymity guarantees, supporting multiple verifiers would be trivial: the issuer could simply use individual key pairs per verifier.

Figure 5 presents our indistinguishability-based issuance anonymity game where the issuers and verifiers are fully malicious. The adversary outputs an issuer public key ipk, a predicate ϕ, and two tuples $(\mathsf{vpk}^{(i)}, \vec{a}^{(i)})$ for $i \in \{0,1\}$. The challenger computes the corresponding credential requests creq values and runs the issuance protocol on the two inputs in a random order determined by a challenge bit b. The resulting credentials are returned to the adversary in the correct order. The adversary wins if it successfully guesses b, implying that it has linked credential requests to their associated attributes or verifier keys.

To prevent trivial wins, both attribute sets must satisfy the predicate ϕ, and neither credential requests nor credentials may fail. Note that in our mKVAC setting, credential requests may fail if a verifier public key is malformed.

Definition 5. *An* mKVAC *scheme is issuance anonymous if for all PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{IAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{IAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) = \mathsf{true}\right] - 1/2\right| \leq \mathsf{negl}(\lambda)$.

Remarks. Our explicit notion of issuance anonymity is stronger than previous models. Interestingly, prior works [Orr24, CAHLT25, CKL+14] model this property within their presentation anonymity game, typically through simulator-based definitions. In those definitions, a simulator interacts with the adversary using only public inputs – the issuer key and the predicate – but does not output a valid credential. Thus, in contrast to our definition, issuance anonymity therein is ensured only as long as credentials remain uncompromised.

3.5 Deniability

Our final security property is deniability, ensuring that a malicious verifier cannot convince a third party that a presentation originates from a legitimate credential held by a user. Deniability is especially desirable in identity schemes, where certified attribute presentations could otherwise enable data markets. In such markets, verifiers might sell the attested information they receive. We leverage the keyed-verification setting, which allows us to express that any presentation could be generated either via a real credential or simply by the verifier itself.

Our formal game, given in Figure 6, captures the deniability aspect through an indistinguishability argument: an honestly generated presentation from a proper credential cred is indistinguishable from a simulated one. The simulator DenS only gets the public presentation information ϕ, ctx as well as the verifier's secret key vsk as inputs. The adversary is allowed to query the challenge oracle multiple times, which either returns a proper presentation (if $b = 0$) based on an honestly generated cred or a simulated presentation via DenS ($b = 1$). The task of the adversary is to determine the randomly chosen challenge bit b. We again give the adversary strong power: $\mathcal{A}$ knows both $\mathsf{isk}, \mathsf{vsk}$, can choose the users' attributes $\vec{a}$ and predicate ϕ, and also learns the issued credential cred.

Definition 6. *An* mKVAC *is deniable for a simulator* DenS *if for all PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{Den}}_{\mathcal{A},\mathsf{mKVAC},\mathsf{DenS}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{Den}}_{\mathcal{A},\mathsf{mKVAC},\mathsf{DenS}}(\lambda) = \mathsf{true}\right] - 1/2\right| \leq \mathsf{negl}(\lambda)$.

Remark on honest key generation. Our definition honestly generates the issuer and verifier key pairs, which is unlike our two privacy games. This is because when $\mathsf{ipk}, \mathsf{vpk}$ are generated maliciously, we do not know a corresponding vsk to run the simulator with. This was not an issue in the privacy games, as we never needed the secret keys there. The recent work of Fiedler and Langrehr [FL25] overcomes this challenge by letting DenS have access to the adversary $\mathcal{A}$ and its randomness. Intuitively, this allows DenS to *extract* vsk from $\mathcal{A}$ and simulate a presentation. However, when both ipk and vpk are generated by $\mathcal{A}$, then DenS could potentially also extract isk and simulate the presentations by using isk as well. This contradicts with the notion of deniability which requires that the presentation could be generated *solely* with the information of the verifier. Thus, we opted for the simpler and more meaningful model that uses honestly generated key pairs.

4 Symmetric Anonymous Group Element Authenticator

As discussed in Section 1.2, our mKVAC construction relies on a *symmetric anonymous group element authenticator* scheme (SAGA) to ensure the integrity of mKVAC verifier keys when issuing a credential. We can think of a SAGA scheme as a KVAC-like primitive that authenticates group elements, with a special form of presentation. Namely, SAGA presentations are hardcoded to authenticate the underlying messages in randomized form as $C_i = M_i + \xi_i G_i$ for blinding terms

ξ_i and public bases G_i. Thus, SAGA setup explicitly outputs generators $\vec{G}$ and their discrete logarithms which are necessary in $\mathsf{mKVAC}_{\mathsf{CMZ}}$ proofs.

After generating a key pair, the SAGA issuer can compute a MAC tag τ on a message vector $\vec{M}$ via MAC algorithm. A presentation of a tag τ is performed via the algorithm Present which outputs a presentation $\mathsf{pres}_{\mathsf{SAGA}}$, witness $\mathsf{wit}_{\mathsf{SAGA}}$, randomized message vector $\vec{C}$ and corresponding blinding terms $\vec{\xi}$. Intuitively, $\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}$ is the public part of the SAGA presentation, whereas $\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}$ is the hidden part to ensure the privacy of a SAGA presentation.

Verification of the presentation is split into two parts: PresKeyedVf that requires the issuer secret key $\mathsf{sk}_{\mathsf{SAGA}}$ but no user secrets and PresWitVf that involves presentation secrets but not $\mathsf{sk}_{\mathsf{SAGA}}$. In practice, the verifier can validate that PresKeyedVf holds by using the public inputs and its secret key $\mathsf{sk}_{\mathsf{SAGA}}$. Furthermore, the user can convince the verifier that PresWitVf holds via a zero-knowledge proof of knowledge of $\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}$, without harming the privacy of the presentation. To disclose additional information to the verifier (who, so far, has learned nothing about $\vec{M}$), the user would extend the zero-knowledge proof of knowledge with an additional statement about $M_j := C_j - \xi_j G_j$ (where ξ_j must be accepted by PresWitVf), while keeping M_j hidden in zero-knowledge. Making C_j available to the verifier and specifically requiring $C_j = M_j + \xi_j G_j$ may seem like a failure to properly abstract from protocol details. However, it is exactly this structure that we need for our SAGA construction. We discuss this in more detail in Section 5.

Definition 7 (Symmetric anonymous group element authenticator). *A symmetric anonymous group element authenticator scheme* SAGA *for group generator* GGen *is specified by a tuple of PPT algorithms listed below:*

$\mathsf{Setup}(\lambda, \ell, \mathbb{G}, p, G) \to (\mathsf{pp}_{\mathsf{SAGA}}, \vec{G} \in \mathbb{G}^\ell, \vec{td} \in \mathbb{Z}_p^\ell)$**:** On input a security parameter λ in its unary representation and a message length parameter ℓ, outputs public parameters $\mathsf{pp}_{\mathsf{SAGA}}$, which are assumed as implicit inputs to the following algorithms. It also outputs (secret) discrete logarithms td_j and corresponding bases $G_j = td_j G$.

$\mathsf{Kg}(\mathsf{pp}_{\mathsf{SAGA}}) \to (\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pk}_{\mathsf{SAGA}})$**:** On input the public parameters $\mathsf{pp}_{\mathsf{SAGA}}$, outputs a secret key $\mathsf{sk}_{\mathsf{SAGA}}$ and a corresponding public key $\mathsf{pk}_{\mathsf{SAGA}}$.

$\mathsf{MAC}(\mathsf{sk}_{\mathsf{SAGA}}, \vec{M} \in \mathbb{G}^\ell) \to \tau$**:** On input a secret key $\mathsf{sk}_{\mathsf{SAGA}}$ and message vectors $\vec{M} \in \mathbb{G}^\ell$, outputs tag τ.

$\mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau, \vec{M}) \to b \in \{\mathsf{true}, \mathsf{false}\}$**:** A deterministic verification algorithm that checks validity of τ on $\vec{M}$ with respect to $\mathsf{sk}_{\mathsf{SAGA}}$.

$\mathsf{Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau, \vec{M}) \to (\mathsf{pres}_{\mathsf{SAGA}}, \vec{C} \in \mathbb{G}^\ell, \vec{\xi} \in \mathbb{Z}_p^\ell, \mathsf{wit}_{\mathsf{SAGA}}) / \perp$**:** On input a public key $\mathsf{pk}_{\mathsf{SAGA}}$, a tag τ, and messages $\vec{M}$, it either outputs $\perp$ if the tag is invalid, or else produces public presentation data $\mathsf{pres}_{\mathsf{SAGA}}$, blinded messages $C_j := M_j + \xi_j G_j$, and private witness data $\mathsf{wit}_{\mathsf{SAGA}}$ needed for later verification.

$\mathsf{PresWitVf}^{\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}}(\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}) \to b \in \{\mathsf{true}, \mathsf{false}\}$**:** A deterministic predicate, parameterized by the public values $(\mathsf{pres}_{\mathsf{SAGA}}, \vec{C})$, that determines the validity of the private values $(\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi})$ of a presentation.

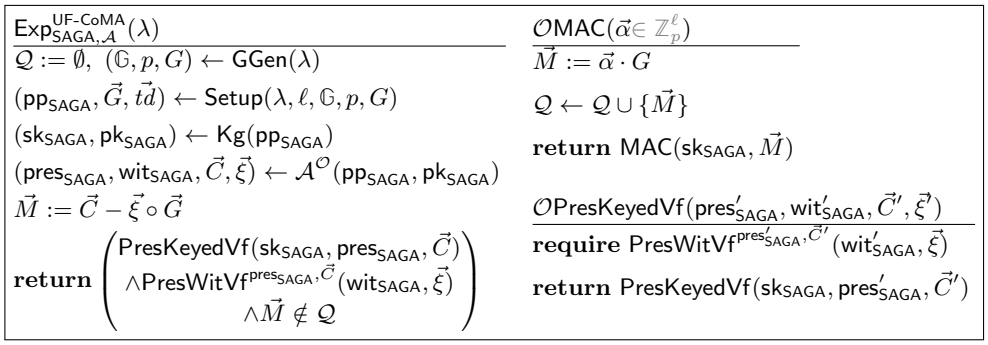

Fig. 7. Presentation unforgeability game for SAGA. The adversary has access to $\mathcal{O}$MAC and $\mathcal{O}$PresKeyedVf oracles.

$\mathsf{PresKeyedVf}(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pres}_{\mathsf{SAGA}}, \vec{C}) \to b \in \{\mathsf{true}, \mathsf{false}\}$**:** Given the secret key $\mathsf{sk}_{\mathsf{SAGA}}$ and public presentation values $(\mathsf{pres}_{\mathsf{SAGA}}, \vec{C})$, checks whether the public values of the presentation are valid.

We provide the correctness definition of SAGA in Appendix H.

4.1 Security Properties

We now formally define the security properties required of a SAGA scheme. Intuitively, we expect (1) unforgeability: it should be hard to produce a valid presentation (public and secret parts) for messages that have not been signed by the issuer, and (2) privacy: one cannot distinguish the presentations (the public parts) of two different tags.

Unforgeability. We define unforgeability under chosen open message attacks (UF-CoMA), given in Figure 7, where "open message" means that the discrete logarithms of messages queried to $\mathcal{O}$MAC must be known [FG18]. The adversary wins if it can produce a valid presentation (public and secret parts) such that the unblinded messages $M_j := C_j - \xi_j G_j$ have not been queried to $\mathcal{O}$MAC before. Looking ahead, we only consider open message attacks as this will be sufficient to ensure the security of our mKVAC construction in Section 5.

Definition 8 (SAGA unforgeability under Chosen open Message Attack). *A* SAGA *scheme satisfies (presentation) unforgeability under chosen open message attack (*UF-CoMA*) if for all PPT adversaries $\mathcal{A}$, the advantage*

$$\mathsf{Adv}^{\mathsf{UF\text{-}CoMA}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) := \Pr[\mathsf{Exp}^{\mathsf{UF\text{-}CoMA}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) = \mathsf{true}] \leq \mathsf{negl}(\lambda).$$

Privacy. The adversary plays the role of a malicious issuer/verifier in the privacy game, presented in Figure 8. The adversary outputs two message vectors and corresponding tags and is tasked to guess which tag the challenge presentation $(\mathsf{pres}_{\mathsf{SAGA}}, \vec{C})$ originates from. To prevent trivial wins, the game enforces that both $\mathsf{pres}^{(b)}_{\mathsf{SAGA}}$ values must be valid ones – not $\perp$.

$\mathsf{Exp}^{\mathsf{PRIV}\text{-}b}_{\mathsf{SAGA},\mathcal{A}}(\lambda)$ for $b \in \{0,1\}$

$(\mathbb{G}, p, G) \leftarrow [\mathsf{GGen}(\lambda)],\ (\mathsf{pp}_{\mathsf{SAGA}}, \vec{G}, \vec{td}) \leftarrow \mathsf{Setup}(\lambda, \ell, \mathbb{G}, p, G)$

$(\mathsf{pk}_{\mathsf{SAGA}}, \tau^{(0)}, \vec{M}^{(0)}, \tau^{(1)}, \vec{M}^{(1)}) \leftarrow \mathcal{A}(\mathsf{pp}_{\mathsf{SAGA}})$

for $i \in \{0,1\}$: $(\mathsf{pres}^{(i)}_{\mathsf{SAGA}}, \vec{C}^{(i)}, \vec{\xi}^{(i)}, \mathsf{wit}^{(i)}_{\mathsf{SAGA}}) \leftarrow \mathsf{Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau^{(i)}, \vec{M})$

if $\mathsf{pres}^{(0)}_{\mathsf{SAGA}} = \bot \vee \mathsf{pres}^{(1)}_{\mathsf{SAGA}} = \bot$: **return** 0 // Rule out trivial win

return $\mathcal{A}(\mathsf{pres}^{(b)}_{\mathsf{SAGA}}, \vec{C}^{(b)})$

Fig. 8. Privacy game for SAGA.

Definition 9 (Privacy). *A* SAGA *scheme satisfies* privacy *if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the advantage*

$$\mathbf{Adv}^{priv}_{\mathsf{SAGA},\mathcal{A}}(\lambda) := \left|\Pr[\mathsf{Exp}^{\mathsf{PRIV}\text{-}0}_{\mathsf{SAGA},\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Exp}^{\mathsf{PRIV}\text{-}1}_{\mathsf{SAGA},\mathcal{A}}(\lambda) = 1]\right| \leq \mathsf{negl}(\lambda).$$

4.2 Our SAGA Instantiations

We propose two instantiations of our SAGA. The first follows CPZ KVACs [CPZ20], which provide MACs on group elements together with a KVAC-style protocol to present these MACs in a privacy-preserving manner. This presentation protocol can be decomposed into the two parts required by our SAGA definition in a natural way. We describe this in Appendix L. Note that in the CPZ instantiation, the values $C_j := M_j + \xi G_j$ use the same blinding factor ξ for all messages M_j. This makes C_j perfectly binding and computationally hiding, assuming DDH is hard. As a consequence, if a future quantum computer breaks the discrete logarithms of G_j, then all past SAGA presentations retroactively lose their privacy. For this reason, we introduce a second SAGA instantiation, which we focus in this paper on due to its better privacy properties. This instantiation is inspired by BBS signatures [BBS04] and its presentations stay retroactively private even if discrete logarithms are broken in the future. In particular, the $C_j := M_j + \xi_j G_j$ are perfectly hiding.

BBS signatures have the message space $\mathbb{Z}_p$ originally. More specifically their signatures on a message $m_1 \in \mathbb{Z}_p$ are in the form of $\sigma := (A := (x+d)^{-1}(G_0 + m_1 Y_1), d)$ for a key pair $(\mathsf{sk} := x, \mathsf{pk} := (X := xG, Y_1))$ and $d \xleftarrow{\$} \mathbb{Z}_p$. The original signatures scheme relies on pairing-friendly curves to obtain public verifiability, but BBS MACs on pairing-free prime-order groups have already been proposed before [BBDT16, Orr24]. Our BBSSAGA scheme also enjoys statistical privacy just like BBS-based signatures and MACs.

BBSSAGA *Construction.* We describe our BBSSAGA scheme in Figure 9. A BBSSAGA secret key consists of the BBS MAC key x as well as an additional scalar y_j for each message in the message vector. The additional scalars are used to sign and verify tags on group elements. The public key of BBSSAGA contains $X := xG$ and $Y_j := y_j G_j$ terms to allow efficient presentation of BBSSAGA tags. Compared to BBS MAC public key, Y_j terms are computed in distinct bases which is crucial for the unforgeability of BBSSAGA (see Appendix I).

$\underline{\mathsf{Setup}(\lambda, \ell, \mathbb{G}, p, G)}$
$G_0 \xleftarrow{\$} \mathbb{G},\ \vec{td} \xleftarrow{\$} \mathbb{Z}_p^\ell,\ \vec{G} := \vec{td}G$
$\mathsf{pp}_{\mathsf{SAGA}} := (\mathbb{G}, p, G, G_0, \vec{G})$
return $(\mathsf{pp}_{\mathsf{SAGA}}, \vec{G}, \vec{td})$

$\underline{\mathsf{Kg}(\mathsf{pp}_{\mathsf{SAGA}})}$
$x \xleftarrow{\$} \mathbb{Z}_p, \vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$
$X := xG,\ \vec{Y} := \vec{y} \circ \vec{G}$
$\mathsf{sk}_{\mathsf{SAGA}} := (x, \vec{y}),\ \mathsf{pk}_{\mathsf{SAGA}} := (X, \vec{Y})$
return $(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pk}_{\mathsf{SAGA}})$

$\underline{\mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau, \vec{M})}$
parse $\mathsf{sk}_{\mathsf{SAGA}}$ **as** $(x, \vec{y})$
parse τ **as** $(A, d, \pi_{\mathsf{BBSSAGA}})$
return $(x + d)A = G_0 + \vec{y} \cdot \vec{M}$

$\underline{\mathsf{PresKeyedVf}(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pres}_{\mathsf{SAGA}}, \vec{C})}$
parse $\mathsf{sk}_{\mathsf{SAGA}}$ **as** $(x, \vec{y})$
parse $\mathsf{pres}_{\mathsf{SAGA}}$ **as** (C_A, T)
return $xC_A = G_0 + \vec{y} \cdot \vec{C} + T$

$\underline{\mathsf{MAC}(\mathsf{sk}_{\mathsf{SAGA}}, \vec{M})}$
parse $\mathsf{sk}_{\mathsf{SAGA}}$ **as** $(x, \vec{y})$
$d \xleftarrow{\$} \mathbb{Z}_p,\ A := (x + d)^{-1}(G_0 + \vec{y} \cdot \vec{M})$
$\mathbb{x}_{\mathsf{BBSSAGA}} := (A, d, \vec{M}),\ \mathbb{w}_{\mathsf{BBSSAGA}} := (x, \vec{y})$
$\pi_{\mathsf{BBSSAGA}} \leftarrow \mathsf{NIP}_{\mathsf{BBSSAGA}}.\mathsf{Prove}(\mathbb{x}_{\mathsf{BBSSAGA}}, \mathbb{w}_{\mathsf{BBSSAGA}})$
return $\tau := (A, d, \pi_{\mathsf{BBSSAGA}})$

$\underline{\mathsf{Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau, \vec{M})}$
parse $(\mathsf{pk}_{\mathsf{SAGA}}, \tau)$ **as** $\left((X, \vec{Y}), (A, d, \pi_{\mathsf{BBSSAGA}})\right)$
$\mathbb{x}_{\mathsf{BBSSAGA}} := (A, d, \vec{M})$
require $\mathsf{NIP}_{\mathsf{BBSSAGA}}.\mathsf{Vf}(\mathbb{x}_{\mathsf{BBSSAGA}}, \pi_{\mathsf{BBSSAGA}})$
$r \xleftarrow{\$} \mathbb{Z}_p,\ \vec{\xi} \xleftarrow{\$} \mathbb{Z}_p^\ell$
$\vec{C} := \vec{M} + \vec{\xi} \circ \vec{G},\ C_A := A + rG$
$T := rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}$
return $(\mathsf{pres}_{\mathsf{SAGA}} := (C_A, T), \vec{C}, \vec{\xi},\ \mathsf{wit}_{\mathsf{SAGA}} := (r, d))$

$\underline{\mathsf{PresWitVf}^{\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}}(\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi})}$
parse $(\mathsf{pres}_{\mathsf{SAGA}}, \mathsf{wit}_{\mathsf{SAGA}})$ **as** $((C_A, T), (r, d))$
return $T = rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}$

Fig. 9. BBS-based instantiation of SAGA

A BBSSAGA tag computes A term by exponentiating the commitment-like $G_0 + \vec{y} \cdot \vec{M}$ term with $(x + d)^{-1}$. Additional to (A, d), a BBSSAGA tag contains a proof of well-formedness of (A, d) according to $\mathcal{R}_{\mathsf{BBSSAGA}}$ below. The proof of well-formedness ensures the privacy of presentations against malicious signers.

$$\mathcal{R}_{\mathsf{BBSSAGA}} := \left\{ (x, \vec{y}) : X = xG \wedge\ \vec{Y} = \vec{y} \circ \vec{G}\ \wedge\ (x + d)A = G_0 + \vec{y} \cdot \vec{M} \right\}$$

Note that if the same tag is presented multiple times, re-verification is unnecessary: once validated, the core tag (A, d) can be stored and reused. For instance, in mKVAC, when a user obtains a second credential from the same verifier or updates an existing one, re-verifying this proof can be skipped.

The presentation of BBSSAGA follows the idea of BBS MAC presentations [Orr24], but is adapted to handle presenting group elements. Notice that BBSSAGA tag verification can be re-structured as $xA = G_0 - dA + \vec{y} \cdot \vec{M}$. This gives a nicely structured equation where a user can blind $A, \vec{M}$ as $C_A, \vec{C}$ as done in Present algorithm. The keyed verification PresKeyedVf then checks the tag on these blinded values together with the term T which cancels out the blinding terms in $C_A, \vec{C}$. Correspondingly, the witness verification algorithm PresWitVf ensures that T is well-formed with respect to the witness $\mathsf{wit}_{\mathsf{SAGA}}$ and $\vec{\xi}$.

Security. In the following, we state the security theorems for BBSSAGA and provide proof sketches. The full proofs are deferred to Appendix K. To prove security of this instantiation, we first establish that the MAC component of BBSSAGA is unforgeable (we formally define security of the MAC component in Appendix J). Note that while the BBSSAGA bears strong resemblance to BBS signatures, which can be proven secure under the q-SDH assumption, its message space is group elements instead of scalars. This complicates the security proof. For this reason, we offer a proof in the generic group model (GGM). We further note that we do not identify immediate attacks against the unforgeability of BBSSAGA under plain chosen-message attacks, and leave a full security analysis of it as an algebraic MAC to future work.

Theorem 1. *If* $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ *is zero-knowledge, then* BBSSAGA *(Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 17) in GGM.*

Proof (Sketch). Based on the zero-knowledge property of $\mathsf{NIP}_{\mathsf{BBSSAGA}}$, the proofs π_{BBSSAGA} can be simulated and therefore reveal no information about the secret keys. If all d values used to answer MAC queries are distinct, then by a standard GGM argument the only way for the adversary to produce a valid message-tag pair is to replay one of the pairs obtained from the MAC oracle. Since the probability that two MAC queries use the same d is negligible, it follows that our BBSSAGA construction is MAC-unforgeable in the GGM.

Building on the security of the MAC component, we then show that the presentation protocol part of BBSSAGA cannot be forged either.

Theorem 2. *If* BBSSAGA *(Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 17), then* BBSSAGA *is presentation unforgeable (Definition 8).*

Proof (Sketch). Both for the final forgery and the verification oracle queries, we are given the secret blinding terms $\mathsf{wit}_{\mathsf{SAGA}}$ and $\vec{\xi}$ and can use them to obtain a message tag pair by cancelling out the blinding terms from the presentation $\mathsf{pres}_{\mathsf{SAGA}}$. This gives us a straightforward reduction to MAC unforgeability.

Finally, we establish privacy of BBSSAGA.

Theorem 3. BBSSAGA *(Figure 9) satisfies the privacy property (Definition 9) if* $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ *is simulation-extractable.*

Proof (Sketch). By the simulation-extractability of $\mathsf{NIP}_{\mathsf{BBSSAGA}}$, the message–tag pairs obtained from the adversary are guaranteed to be well-formed. Consequently, the value T computed for $\mathsf{pres}_{\mathsf{SAGA}}$ must verify correctly under unique secret keys, regardless of whether $b = 0$ or $b = 1$. The remaining components provided to the adversary, namely $(C_A, \vec{C})$, are uniformly random group elements and therefore reveal no information about the bit b. Hence, T can equivalently be computed using uniformly random $(C_A, \vec{C})$, without relying on the tag–message pairs received from the adversary.

5 Our mKVAC Construction $\mathsf{mKVAC}_{\mathsf{CMZ}}$

In this section, we present our mKVAC construction $\mathsf{mKVAC}_{\mathsf{CMZ}}$. The main challenge of our construction is to construct a CMZ-style blind issuance protocol that hides which verifier key the issued credential is valid under. We now develop our solution step-by-step.

CMZ *Credentials.* We start by recalling basic CMZ (keyed-verification) credentials [CMZ14]. A credential is an algebraic MAC of the form

$$(uG, (x_0 + \vec{a} \cdot \vec{x})uG)$$

for messages $\vec{a} \in \mathbb{Z}_p^n$, random $u \xleftarrow{\$} \mathbb{Z}_p^*$, and secret keys $(x_0, \vec{x})$ (cf. Figure 2). The public key enabling blind issuance is $\vec{X} := (x_j G)_{j=1}^n$. Crucially, $x_0 G$ must remain secret, as it allows computing valid credentials for arbitrary messages.

To issue a CMZ credential (in the plain, known-verifier setting), the user commits to their attributes $\vec{a}$ as $C_a := \vec{a} \cdot \vec{X} + sG$ using random $s \xleftarrow{\$} \mathbb{Z}_p$ and proves in zero-knowledge that the commitment is well-formed and that the attributes fulfill the issuance policy $\phi(\vec{a})$. Note that C_a is structurally close to a valid credential that is missing the secret key x_0 term and the randomization term u. The issuer adds the missing $x_0 G$ term and applies randomization with u, i.e. the issuer returns $(uG, u(x_0 G + C_a))$. The issuer proves that they have done this correctly (preventing key parameter consistency attacks on privacy).

Removing the CMZ *key from the issuer.* As a first step towards our solution, we will enable the issuer to issue credentials without directly using the verifier's CMZ key. For this, the issuer generates decryption key e and publish the encryption key $E := eG$. We extend the verifier's CMZ public key with an Elgamal encryption $X_0 := x_0 G + \nu E, Z_0 := \nu G$ of the verifier's secret $x_0 G$. Hence the verifier's public key is now $(\vec{X}, X_0, Z_0)$. With this change, the user can hand the desired verifier's public key to the issuer, who can then decrypt (X_0, Z_0) and use the resulting $x_0 G$ to issue a credential as above. Essentially, e becomes the issuance key.

Authenticating the verifier key. A crucial weakness of the above protocol is that user may maliciously modify the verifier's public key before handing it to the issuer. This applies in particular to the verifier's commitment basis $\vec{X}$ for the commitment C_a. For example, the user could instead supply basis $(2X_1, \ldots, 2X_n)$, commit to attributes a as $C_a := \sum_{j=1}^n a_j (2X_j) + sG$, prove that a_j fulfill the issuance policy ϕ, and then reinterpret the commitment as $C_a = \sum_{j=1}^n (2a_j) X_j + sG$ and effectively receive a credential for attributes $a' := (2a_1, \ldots, 2a_n)$, which may not fulfill the issuance policy. To prevent such attacks on the integrity of verifier keys, the issuer will compute a SAGA tag τ on the verifier's key vpk and publish it alongside vpk. The user hands the desired vpk and its tag τ to the issuer, who would check τ and then run the rest of the issuance protocol assuming vpk is a valid key, preventing the attack sketched above. With this modification, the issuance protocol ensures unforgeability, but reveals the verifier key to the issuer in plain.

Hiding the Verifier. In this step, we hide all verifier-specific data from the issuer. We omit some details, which we discuss later. In order to hide all verifier-specific

$\mathsf{mKVAC_{CMZ}.Setup}(\lambda, n)$

$(\mathbb{G}, p, G) \leftarrow \mathsf{GGen}(\lambda), H \xleftarrow{\$} \mathbb{G}$

$(\mathsf{pp_{SAGA}}, \vec{G}, \vec{td}) \leftarrow \mathsf{SAGA.Setup}(\lambda, n+2, \mathbb{G}, p, G)$

return $\mathsf{pp} = (\mathbb{G}, p, G, H, \mathsf{pp_{SAGA}}, \vec{G})$

$\mathsf{mKVAC_{CMZ}.IKg}(\mathsf{pp})$

$(\mathsf{sk_{SAGA}}, \mathsf{pk_{SAGA}}) \leftarrow \mathsf{SAGA.Kg}(\mathsf{pp_{SAGA}})$

$e \xleftarrow{\$} \mathbb{Z}_p,\ E := eG$

return $(\mathsf{isk} := (\mathsf{sk_{SAGA}}, e),\ \mathsf{ipk} := (\mathsf{pk_{SAGA}}, E))$

$\mathsf{mKVAC_{CMZ}.VKg}(\mathsf{isk}, \mathsf{ipk})$

parse $(\mathsf{isk}, \mathsf{ipk})$ **as** $((\mathsf{sk_{SAGA}}, e), (\mathsf{pk_{SAGA}}, E))$

$(\mathsf{sk_{CMZ}} := (x_0, \vec{x}), \mathsf{pk_{CMZ}} := \vec{X}) \leftarrow \mathsf{CMZ.Kg}(\mathbb{G}, p, G),\ \nu \xleftarrow{\$} \mathbb{Z}_p,\ X_0 := x_0 G + \nu E,\ Z_0 := \nu G$

$\tau \leftarrow \mathsf{SAGA.MAC}(\mathsf{sk_{SAGA}}, (\vec{X}, X_0, Z_0))$

return $\Big(\mathsf{vsk} := (x_0, \vec{x}, \vec{X}), \mathsf{vpk} := (\vec{X}, X_0, Z_0, \tau)\Big)$

Fig. 10. Setup and key generation algorithms of $\mathsf{mKVAC_{CMZ}}$.

values from the previous steps, we need to hide from the issuer the Elgamal ciphertext (X_0, Z_0), the commitment basis $\vec{X}$, and the tag τ on $(\vec{X}, X_0, Z_0)$. Instead of sending the tag τ to the issuer directly, the user now executes a SAGA presentation, which safely exposes a blinded authenticated version $\vec{C}$ of the verifier key. Instead of sending the ElGamal ciphertext (X_0, Z_0), the user sends a re-randomized ciphertext $(\bar{X}_0, \bar{Z}_0)$ of a randomized plaintext $x_0 G + \bar{x}_0 G$, and proves in zero-knowledge that it is well-formed (with respect to the SAGA-blinded ciphertext C_{n+1}, C_{n+2}). Instead of sending the commitment basis $\vec{X}$, the user uses the SAGA's $C_1, \ldots, C_n$ to prove in zero-knowledge that his attribute commitment C_a is well-formed. Overall, the verifier identity is now hidden from the issuer, but the issuer can still be sure that the user's request is well-formed w.r.t. a valid verifier key.

5.1 Construction of $\mathsf{mKVAC_{CMZ}}$

$\mathsf{mKVAC_{CMZ}}$ is formally defined in Figures 10, 11, and 12. Below these procedures are explained in more detail.

Setup and Key Generation. The algorithms Setup, IKg, and VKg are described in Figure 10. Setup initializes the group, the SAGA public parameters, and a random group element H that will be used in the presentation protocol. As outlined above , the issuer's key pair consists of an Elgamal encryption key pair, used for credential issuance, and a SAGA key pair, used to compute SAGA tags on verifier keys and to validate the blinded verifier keys during issuance.

A verifier secret key is simply a CMZ secret key, syntactically extended with its corresponding CMZ public key, since verifiers need this public key when checking presentations. The verifier public key contains CMZ public key $\vec{X}$, the Elgamal encryption of $x_0 G$ to be later decrypted by the issuer, and a SAGA tag on these values, enabling the issuer to check integrity of a verifier key.

Issuance Protocol. The issuance protocol algorithms Obt_1, Iss, and Obt_2 are described in Figure 11. At its core, the issuance protocol proceeds as follows: The user commits to the desired attributes as $C_a := \vec{a} \cdot \vec{X} + \underline{s} G$ using the verifier's CMZ public key $\vec{X}$. The user furthermore sends a randomized Elgamal

$\mathsf{mKVAC}_{\mathsf{CMZ}}.\mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}, \vec{a}, \phi)$

parse $(\mathsf{ipk}, \mathsf{vpk})$ **as** $\Big((\mathsf{pk}_{\mathsf{SAGA}}, E), (\vec{X}, X_0, Z_0, \tau)\Big)$

$(\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}, \vec{\xi}, \mathsf{wit}_{\mathsf{SAGA}}) \leftarrow \mathsf{SAGA.Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau, (\vec{X}, X_0, Z_0))$

require SAGA.Present did not output $\perp$

$\bar{x}_0, \bar{\nu} \xleftarrow{\$} \mathbb{Z}_p,\ \bar{X}_0 := X_0 + \bar{x}_0 G + \bar{\nu} E,\quad \bar{Z}_0 := Z_0 + \bar{\nu} G$ // decrypts to $\bar{X}_0 - e\bar{Z}_0 = (\bar{x}_0 + x_0)G$

$s \xleftarrow{\$} \mathbb{Z}_p,\ C_a := \vec{a} \cdot \vec{X} + sG$ // Commitment to attributes

$\pi_R \leftarrow \mathsf{NIP}_R.\mathsf{Prove}\Big(\aleph_R := (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a),\ \beth_R := (s, \vec{a}, \vec{\xi}' := \vec{a} \circ \vec{\xi}, \bar{x}_0, \bar{\nu}, \mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi})\Big)$

return $\big(\mathsf{st} := (s, \bar{x}_0, \bar{X}_0, \bar{Z}_0, \vec{a}), \mathsf{creq} := (\mathsf{pres}_{\mathsf{SAGA}}, \aleph_R, \pi_R)\big)$

$\mathsf{Iss}(\mathsf{isk}, \phi, \mathsf{creq})$

parse $(\mathsf{isk}, \mathsf{ipk}, \mathsf{creq})$ **as** $\Big((\mathsf{sk}_{\mathsf{SAGA}}, e), (\mathsf{pk}_{\mathsf{SAGA}}, E), (\mathsf{pres}_{\mathsf{SAGA}}, \aleph_R := (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a), \pi_R)\Big)$

require $\mathsf{NIP}_R.\mathsf{Vf}(\aleph_R, \pi_R) \ \wedge\ \mathsf{SAGA.PresKeyedVf}(\mathsf{isk}, \mathsf{pres}_{\mathsf{SAGA}}, \vec{C})$

$u \xleftarrow{\$} \mathbb{Z}_p,\quad \bar{U} := uG,\quad \bar{V} := u\big((\bar{X}_0 - e\bar{Z}_0) + C_a\big)$ // $= (x_0 + \underline{\bar{x}_0} + \vec{a} \cdot \vec{x} + \underline{s})\bar{U}$

$\pi_I \leftarrow \mathsf{NIP}_I.\mathsf{Prove}(\aleph_I := (E, \bar{U}, \bar{V}, \bar{X}_0, \bar{Z}_0, C_a), \beth_I := (e, u))$

return $\mathsf{bcred} := (\bar{U}, \bar{V}, \pi_I)$

$\mathsf{Obt}_2(\mathsf{st}, \mathsf{bcred})$

parse $(\mathsf{st}, \mathsf{bcred})$ **as** $\big((s, \bar{x}_0, \bar{X}_0, \bar{Z}_0, \vec{a}), (\bar{U}, \bar{V}, \pi_I)\big)$

require $\mathsf{NIP}_I.\mathsf{Vf}(\aleph_I := (E, \bar{U}, \bar{V}, \bar{X}_0, \bar{Z}_0, C_a), \pi_I)$

$\gamma \xleftarrow{\$} \mathbb{Z}_p^*,\quad U := \gamma\bar{U},\quad V := \gamma(\bar{V} - (s + \bar{x}_0)\bar{U})$ // Remove error terms $\underline{\bar{x}_0}, \underline{s}$ from $\bar{V}$

return $\mathsf{cred} := (U, V, \vec{a})$

Fig. 11. Credential issuance protocol of the $\mathsf{mKVAC}_{\mathsf{CMZ}}$.

encryption $(\bar{X}_0, \bar{Z}_0)$ of $(x_0 + \underline{\bar{x}_0})G$ where $x_0 G$ is the real verifier's key, blinded by a random $\underline{\bar{x}_0}$. The issuer decrypts $(x_0 + \underline{\bar{x}_0})G$ from the Elgamal ciphertext and uses it to complete C_a to a blinded credential with $(\bar{U}, \bar{V}) = (\bar{U}, (x_0 + \underline{\bar{x}_0} + \vec{a} \cdot \vec{x} + \underline{s})\bar{U})$. Finally, the user removes the error terms $\underline{\bar{x}_0}$ and $\underline{s}$ from $(\bar{U}, \bar{V})$ to obtain the final MAC on $\vec{a}$. The rest of the protocol deals with checking that both parties properly adhere to these steps.

Well-Formedness of Credential Requests. The issuance of the blinded credential $(\bar{U}, \bar{V})$ only depends on $\bar{X}_0, \bar{Z}_0, C_a$, and the user proves the well-formedness of these values according to NIP_R described below.

$$
\begin{aligned}
\mathsf{NIP}_R := \Big\{ \Big(s, \vec{a}, \vec{\xi}' := \vec{a} \circ \vec{\xi}, \bar{x}_0, \bar{\nu}, \mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}\Big) :\ & \\
\phi(\vec{a}) = \mathsf{true} \ /\!/ \text{ Attributes fulfill policy} & \\
\wedge\ \bar{X}_0 = (C_{n+1} - \xi_{n+1} G_{n+1}) + \bar{x}_0 G + \bar{\nu} E \ /\!/ \text{ Ciphertext valid} & \\
\wedge\ \bar{Z}_0 = (C_{n+2} - \xi_{n+2} G_{n+2}) + \bar{\nu} G \ /\!/ \text{ Ciphertext valid} & \\
\wedge\ C_a = sG + \Big(\sum_{j \in [n]} a_j C_j\Big) - \Big(\sum_{j \in [n]} \xi'_j G_j\Big) \ /\!/ \text{ Commitment valid} & \\
\wedge\ \mathsf{PresWitVf}^{\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}}(\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}) \ /\!/ \text{ SAGA check} \Big\} &
\end{aligned}
$$

First, the user links the underlying $(\vec{X}, X_0, Z_0)$ to a legitimate vpk. This is done by relying on the SAGA presentation of tag τ from vpk. Specifically, SAGA presentation provides an authenticated vector $\vec{C} := (\vec{X}, X_0, Z_0) + \vec{\xi} \circ \vec{G}$ with blinding terms $\vec{\xi}$. The user proves that $\mathsf{PresWitVf}$ holds as part of NIP_R and the issuer checks that $\mathsf{PresKeyedVf}$ holds during the issuance, which completes the SAGA presentation verification. As $\vec{C}$ can be safely handed to the issuer, the user is then tasked to prove the well-formedness of $\bar{X}_0, \bar{Z}_0, C_a$ by relying on a linear relation with public bases and secret exponents as shown in NIP_R.

Note that NIP_R does not enforce the well-formedness of $\vec{\xi} := \vec{a} \circ \vec{\xi}$. This is a deliberate optimization: requiring $\xi_j = a_j \cdot \xi_j$ would introduce non-linear terms to be proven and impose extra complexity in NIP_R. In our security proof, we show that this omission is harmless by relying on the trapdoor $\vec{td}$ of SAGA.

Well-Formedness of Blinded Credentials. To prevent key parameter consistency attacks on user privacy, the issuer proves that NIP_I below holds. This relation shows that *1)* $\bar{U}$ and $\bar{V}$ are consistent with their exponent u, and *2)* $\bar{V}$ is derived from the correct decryption of the Elgamal ciphertext $(\bar{X}_0, \bar{Z}_0)$.

$$\mathsf{NIP}_I := \left\{ (e, u) \ : \ \bar{U} = uG \ \wedge \ E = eG \ \wedge \ \bar{V} = u\left((\bar{X}_0 - e\bar{Z}_0) + C_a\right) \right\}$$

Credential Presentation. The credential presentation is shown in Figure 12. Because the issuance phase yields a valid $\mathsf{CMZ\ KVAC}$, the presentation phase is almost identical to the $\mathsf{CMZ\ KVAC}$'s presentation [CMZ14]. The only significant difference is a modification of the signature-of-knowledge relation $\mathcal{R}_S$ to ensure deniability using the following well-known technique. Namely, the user proves the knowledge of either the vsk component x_1 such that $X_1 = x_1 G$, or a valid CMZ credential for the predicate. This establishes deniability: a malicious verifier can create such a proof by only using x_1 from its secret key, without having to find valid attributes $\vec{a}$ fulfilling ϕ (which might be computationally hard).

$$\begin{aligned} \mathsf{NIP}_S := \Big\{ ((\tilde{\gamma}, \vec{a}, \vec{\gamma}), \bot) \ : \ & X_1 = x_1 G \ \vee \quad /\!/ \text{ Deniability} \\ & \left(Z = \vec{\gamma} \cdot \vec{X} - \tilde{\gamma} H \ \wedge \ \vec{C} = \vec{a} \cdot \tilde{U} + \vec{\gamma} \cdot G \ \wedge \ \phi(\vec{a}) = \mathsf{true}\right) \Big\} \end{aligned}$$

5.2 Security of $\mathsf{mKVAC}_{\mathsf{CMZ}}$

In this section, we state our theorems for the security of $\mathsf{mKVAC}_{\mathsf{CMZ}}$, and sketch the proofs of these theorems. The full proofs of these theorems can be found in Appendix G. We also note that our proof for presentation anonymity could rely on the soundness of NIP_I scheme instead of its simulation-extractability. Similarly, the proof of deniability could rely on the soundness of NIP_I instead of simulation-extractability and the witness indistinguishability of NIP_S instead of its zero-knowledge properties. We omit these extensions to rely on less number of NIP properties for simplicity.

Theorem 4. $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *scheme from Figures 10 to 12 is unforgeable if* NIP_I *and* NIP_S *are zero-knowledge,* NIP_R *and* NIP_S *are complete and simulation-extractable,* DDH *is hard, and* SAGA *and* $\mu\mathsf{CMZ}$ *are correct and unforgeable.*

$\mathsf{Present}(\mathsf{ipk}, \mathsf{vpk}, \mathsf{cred}, \mathsf{ctx}, \phi)$

parse $(\mathsf{vpk}, \mathsf{cred})$ **as** $((\vec{X}, X_0, Z_0, \tau), (U, V, \vec{a}))$

$\gamma, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p^*,\ \vec{\gamma} \xleftarrow{\$} \mathbb{Z}_p^n$

$(\tilde{U}, \tilde{V}) := (\gamma U, \gamma V),\ Z := \vec{\gamma} \cdot \vec{X} - \tilde{\gamma} H,\ C_V := \tilde{V} + \tilde{\gamma} H,\ \vec{C} := \vec{a} \cdot \tilde{U} + \vec{\gamma} \cdot G$

$\pi_S \leftarrow \mathsf{NIP}_S.\mathsf{Prove}(\mathbb{x}_S := (\vec{X}, \tilde{U}),\ \mathbb{w}_S := \left((\tilde{\gamma}, \vec{a}, (\gamma_j)_{j=1}^n), \bot\right), \mathsf{ctx}_{\mathsf{NIP}} := (\mathsf{vpk}, \mathsf{ctx}))$

return $\mathsf{pres} := (\tilde{U}, Z, C_V, \vec{C}, \pi_S)$

$\mathsf{VfPres}(\mathsf{vsk}, \mathsf{ipk}, \mathsf{pres}, \phi)$

parse $(\mathsf{vsk}, \mathsf{pres})$ **as** $((x_0, \vec{x}, \vec{X}), (\tilde{U}, Z, C_V, \vec{C}, \pi_S))$

return $\mathsf{NIP}_S.\mathsf{Vf}(\mathbb{x}_S := (\vec{X}, \tilde{U}), \pi_S, \mathsf{ctx}_{\mathsf{NIP}} := (\mathsf{vpk}, \mathsf{ctx}))$

$\quad \wedge\ \tilde{U} \neq 0\ \wedge\ (Z = x_0 \tilde{U} + \vec{x} \cdot \vec{C} - C_V)$

Fig. 12. Credential presentation and verification of $\mathsf{mKVAC}_{\mathsf{CMZ}}$.

Proof (Sketch). We set the presentation unforgeability extractors to those of the NIP schemes NIP_R and NIP_S, and reduce security to CMZ-MAC unforgeability under the challenge key vsk^* as a result of the following main steps. First, the honest and corrupt credential issuance oracles are restricted to respond only to credential requests with valid vpk values by relying on SAGA unforgeability. This allows us to skip Elgamal decryption during issuance as we can look-up x_0 of each extracted vpk and simulate Elgamal ciphertexts as random values under DDH assumption, thereby leaking no x_0^* of vpk^* as in CMZ MAC. The proof then addresses two challenges: simulating honest presentation queries without the actual credentials, and handling error terms in blinded credentials for corrupt issuance without white-box access to vsk^*. These are resolved using the zero-knowledge properties of NIP_I and NIP_S, along with algebraic manipulations. In the final step, each corrupt issuance query is answered with a CMZ MAC computation, allowing us to build our reduction to CMZ MAC unforgeability.

Theorem 5. $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *has presentation anonymity if* NIP_I *is extractable and* NIP_S *is zero-knowledge.*

Proof (Sketch). To simulate $\mathcal{O}\mathsf{Chall}$ queries without using bit b: we compute π_S of pres via NIP_S zero-knowledge simulator, and simulate $(\tilde{U}, Z, C_V, \vec{C})$ of pres with random $Z, C_V, \vec{C}$ while $\tilde{U}$ is equivocated by using $(x_0 G, \vec{X})$ of the designated verifier of the presentation. An obstacle arises in this strategy if the credential issuance produces an invalid credential without triggerring Obt_2 to abort. Then simulating a valid presentation as above would become distinguishable. We prevent this case by relying on the simulation-extractability of NIP_I.

Theorem 6. $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *scheme from Figures 10 to 12 satisfies the issuance anonymity property if* NIP_I *is simulation-extractable,* NIP_R *is zero-knowledge, and the underlying* SAGA *scheme satisfies privacy.*

Proof (Sketch). Our proof simulates the issuance anonymity game of $\mathsf{mKVAC}_{\mathsf{CMZ}}$ by simulating the credential requests without using the bit b through the following steps. First, both $\mathsf{cred}^{(i)}$ are generated independently of bcred. This is done

by extracting e from π_I under the simulation-extractability of NIP_I and computing both $\mathsf{cred}^{(i)}$ for $i \in \{0,1\}$ directly from the corresponding $a^{(i)}$. Second, the proof π_R are simulated by the zero-knowledge property of NIP_R, thereby revealing no information about bit b. Finally, both $\mathsf{creq}^{(i)}$ are computed by running $\mathsf{SAGA.Present}$ on $\mathsf{vpk}^{(0)}$ and $a^{(0)}$ and randomly choosing $\bar{X}_0, \bar{Z}_0, C_a$. The former is indistinguishable under the SAGA privacy, and the latter is perfectly indistinguishable due to the distributions of $\bar{X}_0, \bar{Z}_0, C_a$. As a result, we simulate the adversary's view in the issuance anonymity game indistinguishably without using the bit b, concluding our proof.

Theorem 7. $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *is deniable for* DenS *in Figure 17 if* NIP_I *is sound, and* NIP_S *is zero-knowledge.*

Proof (Sketch). Our deniability simulator DenS works as follows: Given $\mathsf{vsk} := (x_0, \vec{x})$, it first simulates $(\tilde{U}, Z, C_V, \vec{C})$ of pres with random $Z, C_V, \vec{C}$ while $\tilde{U}$ is equivocated by using vsk. Then it computes the proof π_S of pres by using x_1 as witness for the $X_1 = x_1 G$ part of NIP_S. The indistinguishability of this simulator from the honest run of the $\mathsf{Present}$ algorithms is shown as follows. Regardless of the bit b, $\mathcal{O}\mathsf{Chall}$ can compute π_S via the zero-knowledge simulator. Then $\mathcal{O}\mathsf{Chall}$ also computes $(\tilde{U}, Z, C_V, \vec{C})$ of honestly created presentations as in DenS, so $\mathcal{O}\mathsf{Chall}$ does not use the bit b anymore. The only problem in this strategy is that the adversary may try to trick the challenger to end up with an invalid credential at the end of the credential issuance. This would make honestly created credentials fail, allowing the adversary to trivially distinguish real or simulated proofs. We prevent this case by relying on the simulation-extractability of NIP_I.

6 Applications & Implementation

This section evaluates the applications of mKVAC, and then analyzes its practicality through a proof-of-concept implementation.

Applications of mKVAC. We now sketch how our multi-verifier KVACs can be used to enhance security in closed settings (such as Apple's anonymous rate-limited credentials), and also fits within the EUDI context.

Anonymous Rate-limited Credentials (ARC). A recent internet draft [YW25] proposes *anonymous rate-limited credentials (ARC)* to improve Privacy Pass, which is an IETF standard for single-use anonymous tokens [CDVW24]. In ARC, users obtain a CMZ KVAC on a PRF key and authenticates by revealing a PRF evaluation on a random index $i \in [n]$ where n is the rate-limit, along with a proof of correctness tied to the credential. Verifiers accept only fresh PRF outputs which ensures rate-limiting. ARC improves Privacy Pass by supporting multi-show unlinkability and rate-limiting, but looses public-verifiability (if Privacy Pass is used with blind RSA signatures). The draft reasons their choice by noting

that publicly verifiable credentials, such as BBS, rely on pairings which are not widely supported [YW25]. Another reason for switching from Privacy Pass to ARC is that it enables *verifier-specific rate-limiting*, i.e., each verifier can set their own upper bound. That is, ARC considers to be run with multiple verifiers – which all need to have access to the same secret key when using conventional KVACs. That hiding access information is important, even in an Apple internal setting, is demonstrated through Apple's use of Privacy Pass in the blind authentication to the Private Cloud Compute service [App25], aiming to hide which node server the user is accessing.

Our mKVAC fits that ARC system perfectly and provides better security: Clearly, verification is supposed to be run significantly more often than issuance in this system. Thus, in a conventional KVAC the issuer's key needs to be highly replicated to handle high-throughput authentications, which jeopardizes security of the entire system. Using mKVAC, the issuer secret key is *not* needed for every verification, and instead a more fine-grained key structure can be established. If a verifier key is suspected to be compromised, it can be revoked without impacting the other verifiers.

EUDI (with Intermediaries). In an eID system, the number and set of verifiers might not be that clear and well established as in the ARC context. Using our mKVAC scheme, the user might have to fetch a larger amount of credentials. However, for scenarios where the user wants to repeatedly authenticate to the same set of core services while remaining anonymous, our solution provides the first pairing-free and multi-show unlinkable solution. Further, the implicit "RP authentication" aspect of mKVAC (verifiers must explicitly register with the issuer), is an interesting strengthening over truly publicly verifiable solutions and can counter the risk of over-authentication, where user might get tricked into revealing identifying information to untrusted verifiers. Further, our mKVAC scheme naturally fits a deployment setting where verifiers rely on a smaller set of intermediaries that provide *verification-as-a-service*. In fact, while the EUDI envisions a fully distributed setting, the real-world deployment might end up being much more centralized. Already for simple (ECDSA-based) immunity certificates rolled out during COVID, which were the first wide deployment of user-centric credentials, the verification turned out to be too complex for services: not because of the cryptography, but because the services would have to maintain credential schemas and revocation check-ups for dozens of countries and millions of users. This led to the development of intermediaries that verify the credential presentations and translate them into simpler and standardized tokens [eHe22, GL23]. For such an intermediary-based deployment, our mKVAC fits perfectly. This is also where the deniability aspect of our solution is important, as the intermediaries can then not collect and sell the gathered user information.

Performance evaluation. We implemented and open-sourced [BBL+25] the end-to-end mKVAC prototype in pure Rust, using the Edwards25519 curve (via

	Credential issuance						Credential presentation		
$\lvert\vec{a}\rvert$	Obt_1 (ms)	Iss (ms)	Obt_2 (ms)	creq	bcred	cred	Present (ms)	VfPres (ms)	pres
4	3.41	2.53	0.69	1.1	0.2	0.1	1.09	0.80	0.6
8	4.11	3.27	0.68	1.6	0.2	0.1	1.21	0.85	1.0
32	9.21	8.19	0.69	4.6	0.2	0.1	2.59	1.96	3.2
64	15.27	14.64	0.70	8.6	0.2	0.1	3.70	2.97	6.2

Fig. 13. mKVAC benchmarks: Times are in milliseconds, and payload sizes are in KiB. (i) The three-step issuance pipeline Obt_1, issue_cred (Iss), Obt_2; (ii) presentation (Present) and verification (VfPres). Communication is broken down into the consolidated request payload creq, the blinding payload (bcred), the credential (cred), and the presentation (pres). Payloads are in KiB.

the Arkworks ecosystem [Ark22]) for all group operations and hash-to-scalar computations. All NIZK proofs in the request, issuance, and presentation phases are heuristically instantiated using the Fiat-Shamir transform (Appendix C). All experiments were conducted on a MacBook Pro with an Apple M4 processor and 24 GB RAM. We measured latency for each algorithm in the mKVAC API. Unless otherwise noted, all numbers are for hidden attributes (no selective disclosure), reflecting the default predicate from Appendix C. Our benchmark results are shown in Figure 13. We discuss details in Appendix D.

Acknowledgments. The work by Emad Heydari Beni was supported by Nokia Bell Labs. The work by Anja Lehmann and Cavit Özbay was partially funded by the HPI Research School on Systems Design and by SPRIND, the Federal Agency for Breakthrough Innovation. The authors are solely responsible for the content of this publication; the positions presented here do not reflect the views of SPRIND. The work by Omid Mirzamohammadi was supported by the Flemish Government through the Cybersecurity Research Program (grant number VOEWICS02) and through SolidLab Flanders (Flemish Government, EWI). He also thanks Aysajan Abidin for his valuable comments. Mahdi Sedaghat was partially supported by the Flemish Government through the Cybersecurity Research Program (grant number VOEWICS02), the KU Leuven Global Seed Fund (grant number GSF/25/043) and Sui Academic Research Award.

References

AHIV17. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam, *Ligero: Lightweight sublinear arguments without a trusted setup*, ACM CCS 2017 (Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, eds.), ACM Press, October / November 2017, pp. 2087–2104.

App25. Apple, *Apple Private Cloud Compute*, https://security.apple.com/documentation/private-cloud-compute, 2025.

Ark22. Arkworks Contributors, *Arkworks zkSNARK ecosystem*, https://arkworks.rs, 2022.

BBC+24. Carsten Baum, Olivier Blazy, Jan Camenisch, Jaap-Henk Hoepman, Eysa Lee, Anja Lehmann, Anna Lysyanskaya, René Mayrhofer, Hart Montgomery, Ngoc Khanh Nguyen, et al., *Cryptographers' feedback on the eu digital identity's arf*.

BBDE19. Johannes Blömer, Jan Bobolz, Denis Diemert, and Fabian Eidens, *Updatable anonymous credentials and applications to incentive systems*, ACM CCS 2019 (Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, eds.), ACM Press, November 2019, pp. 1671–1685.

BBDT16. Amira Barki, Solenn Brunet, Nicolas Desmoulins, and Jacques Traoré, *Improved algebraic MACs and practical keyed-verification anonymous credentials*, SAC 2016 (Roberto Avanzi and Howard M. Heys, eds.), LNCS, vol. 10532, Springer, Cham, August 2016, pp. 360–380.

BBL+25. Jan Bobolz, Emad Heydari Beni, Anja Lehmann, Omid Mirzamohammadi, Cavit Özbay, and Mahdi Sedaghat, *Multi-Verifier Keyed-Verification Anonymous Credentials Implementation*, Github repository, 2025, https://github.com/KULeuven-COSIC/mkvac.

BBS04. Dan Boneh, Xavier Boyen, and Hovav Shacham, *Short group signatures*, Advances in Cryptology - CRYPTO 2004, 24th Annual International CryptologyConference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings (Matthew K. Franklin, ed.), Lecture Notes in Computer Science, vol. 3152, Springer, 2004, pp. 41–55.

BEK+21. Jan Bobolz, Fabian Eidens, Stephan Krenn, Sebastian Ramacher, and Kai Samelin, *Issuer-hiding attribute-based credentials*, CANS 21 (Mauro Conti, Marc Stevens, and Stephan Krenn, eds.), LNCS, vol. 13099, Springer, Cham, December 2021, pp. 158–178.

BL13. Foteini Baldimtsi and Anna Lysyanskaya, *Anonymous credentials light*, ACM CCS 2013 (Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, eds.), ACM Press, November 2013, pp. 1087–1098.

Bra95. Stefan Brands, *Off-line electronic cash based on secret-key certificates*, LATIN 1995 (Ricardo A. Baeza-Yates, Eric Goles Ch., and Patricio V. Poblete, eds.), LNCS, vol. 911, Springer, Berlin, Heidelberg, April 1995, pp. 131–166.

CAHLT25. Rutchathon Chairattana-Apirom, Franklin Harding, Anna Lysyanskaya, and Stefano Tessaro, *Server-aided anonymous credentials*, CRYPTO 2025, Part VI (Yael Tauman Kalai and Seny F. Kamara, eds.), LNCS, vol. 16005, Springer, Cham, August 2025, pp. 291–324.

CDDH19. Jan Camenisch, Manu Drijvers, Petr Dzurenda, and Jan Hajny, *Fast keyed-verification anonymous credentials on standard smart cards*, IFIP International Conference on ICT Systems Security and Privacy Protection, Springer, 2019, pp. 286–298.

CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers, *Proofs of partial knowledge and simplified design of witness hiding protocols*, CRYPTO'94 (Yvo Desmedt, ed.), LNCS, vol. 839, Springer, Berlin, Heidelberg, August 1994, pp. 174–187.

CDVW24. Sofia Celi, Alex Davidson, Steven Valdez, and Christopher A. Wood, *Privacy Pass Issuance Protocols*, RFC 9578, June 2024.

Cha85. David Chaum, *Security without identification: Transaction systems to make big brother obsolete*, Communications of the ACM **28** (1985), no. 10, 1030–1044.

CKL+14. Jan Camenisch, Stephan Krenn, Anja Lehmann, Gert Læssøe Mikkelsen, Gregory Neven, and Michael Østergaard Pedersen, *Formal treatment of privacy-enhancing credential systems*, Cryptology ePrint Archive, Report 2014/708, 2014.

CL01. Jan Camenisch and Anna Lysyanskaya, *An efficient system for non-transferable anonymous credentials with optional anonymity revocation*, EUROCRYPT 2001 (Birgit Pfitzmann, ed.), LNCS, vol. 2045, Springer, Berlin, Heidelberg, May 2001, pp. 93–118.

CL02. ———, *Dynamic accumulators and application to efficient revocation of anonymous credentials*, CRYPTO 2002 (Moti Yung, ed.), LNCS, vol. 2442, Springer, Berlin, Heidelberg, August 2002, pp. 61–76.

CL03. ———, *A signature scheme with efficient protocols*, SCN 02 (Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, eds.), LNCS, vol. 2576, Springer, Berlin, Heidelberg, September 2003, pp. 268–289.

CL04. ———, *Signature schemes and anonymous credentials from bilinear maps*, CRYPTO 2004 (Matthew Franklin, ed.), LNCS, vol. 3152, Springer, Berlin, Heidelberg, August 2004, pp. 56–72.

CMZ14. Melissa Chase, Sarah Meiklejohn, and Greg Zaverucha, *Algebraic MACs and Keyed-Verification Anonymous Credentials*, Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (New York, NY, USA), CCS '14, Association for Computing Machinery, 2014, pp. 1205–1216.

CPZ20. Melissa Chase, Trevor Perrin, and Greg Zaverucha, *The Signal private group system and anonymous credentials supporting efficient verifiable encryption*, ACM CCS 2020 (Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, eds.), ACM Press, November 2020, pp. 1445–1459.

CV02. Jan Camenisch and Els Van Herreweghen, *Design and implementation of the idemix anonymous credential system*, ACM CCS 2002 (Vijayalakshmi Atluri, ed.), ACM Press, November 2002, pp. 21–30.

DG23. Heini Bergsson Debes and Thanassis Giannetsos, *RETRACT: Expressive Designated Verifier Anonymous Credentials*, Proceedings of the 18th International Conference on Availability, Reliability and Security (New York, NY, USA), ARES '23, Association for Computing Machinery, 2023, pp. 1–12.

DGS+18. Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda, *Privacy pass: Bypassing internet challenges anonymously*, PoPETs **2018** (2018), no. 3, 164–180.

DKS16. David Derler, Stephan Krenn, and Daniel Slamanig, *Signer-anonymous designated-verifier redactable signatures for cloud-based data sharing*, CANS 16 (Sara Foresti and Giuseppe Persiano, eds.), LNCS, vol. 10052, Springer, Cham, November 2016, pp. 211–227.

eHe22. eHealth Network, *EU Digital COVID Certificate Validation Service.*, https://github.com/eu-digital-green-certificates/dgca-validation-service/tree/2f047cd, 2022.

Eth19. Ethereum Foundation, *Semaphore: A zero-knowledge protocol for anonymous interactions*, https://semaphore.pse.dev/, 2019, Accessed: 2025-08-26.

EU 25. EU Digital Identity Wallet, *European digital identity wallet. architecture and reference framework. version 2.4.0*, 2025.

Eur24. European Parliament and Council, *Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework*, https://eur-lex.europa.eu/eli/reg/2024/1183/oj, 2024, Official Journal of the European Union, L 1183, 30 April 2024, entered into force 20 May 2024.

Eur25. European Commission/EU Digital Identity, *EU Digital Identity Wallet*, https://ec.europa.eu/digital-building-blocks/sites/spaces/EUDIGITALIDENTITYWALLET/pages/694487738/EU+Digital+Identity+Wallet+Home, 2025, Accessed: 2025-09-30.

FG18. Georg Fuchsbauer and Romain Gay, *Weakly secure equivalence-class signatures from standard assumptions*, PKC 2018, Part II (Michel Abdalla and Ricardo Dahab, eds.), LNCS, vol. 10770, Springer, Cham, March 2018, pp. 153–183.

FHS19. Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig, *Structure-preserving signatures on equivalence classes and constant-size anonymous credentials*, Journal of Cryptology **32** (2019), no. 2, 498–546.

Fis05. Marc Fischlin, *Communication-efficient non-interactive proofs of knowledge with online extractors*, CRYPTO 2005 (Victor Shoup, ed.), LNCS, vol. 3621, Springer, Berlin, Heidelberg, August 2005, pp. 152–168.

FL25. Rune Fiedler and Roman Langrehr, *On Deniable Authentication against Malicious Verifiers*, 2025, Publication info: A major revision of an IACR publication in CRYPTO 2025.

FS87. Amos Fiat and Adi Shamir, *How to prove yourself: Practical solutions to identification and signature problems*, CRYPTO'86 (Andrew M. Odlyzko, ed.), LNCS, vol. 263, Springer, Berlin, Heidelberg, August 1987, pp. 186–194.

Fs24. Matteo Frigo and abhi shelat, *Anonymous credentials from ECDSA*, Cryptology ePrint Archive, Report 2024/2010, 2024.

FS25. Matteo Frigo and Abhi Shelat, *The longfellow zero-knowledge scheme*, IETF Internet-Draft draft-google-cfrg-libzk-01, September 2025, Work in Progress.

GKR08. Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum, *Delegating computation: interactive proofs for muggles*, 40th ACM STOC (Richard E. Ladner and Cynthia Dwork, eds.), ACM Press, May 2008, pp. 113–122.

GL23. Tarek Galal and Anja Lehmann, *Privacy-preserving outsourced certificate validation*, Proceedings on Privacy Enhancing Technologies (2023).

Goo25. Google, *Longfellow zk*, https://github.com/google/longfellow-zk, 2025, Open-source zero-knowledge proof library for age assurance (accessed: 2025-09-10).

HIP+24. Scott Hendrickson, Jana Iyengar, Tommy Pauly, Steven Valdez, and Christopher A. Wood, *Rate-Limited Token Issuance Protocol*, Internet-Draft draft-ietf-privacypass-rate-limit-tokens-06, Internet Engineering Task Force, April 2024, Work in Progress.

HS21. Lucjan Hanzlik and Daniel Slamanig, *With a little help from my friends: Constructing practical anonymous credentials*, ACM CCS 2021 (Giovanni Vigna and Elaine Shi, eds.), ACM Press, November 2021, pp. 2004–2023.

ISO21. *Personal identification — iso-compliant driving licence — part 5: Mobile driving licence (mdl) application*, 2021, FDIS / Final Draft International Standard.

KLR23. Julia Kastner, Julian Loss, and Omar Renawi, *Concurrent security of anonymous credentials light, revisited*, ACM CCS 2023 (Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, eds.), ACM Press, November 2023, pp. 45–59.

Ks22. Yashvanth Kondi and abhi shelat, *Improved straight-line extraction in the random oracle model with applications to signature aggregation*, ASIACRYPT 2022, Part II (Shweta Agrawal and Dongdai Lin, eds.), LNCS, vol. 13792, Springer, Cham, December 2022, pp. 279–309.

LR22. Anna Lysyanskaya and Leah Namisa Rosenbloom, *Universally composable Σ-protocols in the global random-oracle model*, TCC 2022, Part I (Eike Kiltz and Vinod Vaikuntanathan, eds.), LNCS, vol. 13747, Springer, Cham, November 2022, pp. 203–233.

Mau09. Ueli M. Maurer, *Unifying zero-knowledge proofs of knowledge*, AFRICACRYPT 09 (Bart Preneel, ed.), LNCS, vol. 5580, Springer, Berlin, Heidelberg, June 2009, pp. 272–286.

MBS+25. Omid Mirzamohammadi, Jan Bobolz, Mahdi Sedaghat, Emad Heydari Beni, Aysajan Abidin, Dave Singelee, and Bart Preneel, *Keyed-Verification Anonymous Credentials with Highly Efficient Partial Disclosure*, 2025, Publication info: Preprint. (To appear in IACR CiC 2025).

Orr24. Michele Orrù, *Revisiting Keyed-Verification Anonymous Credentials*, 2024, Publication info: Preprint. (To appear in ACM CCS 2025).

PZ13. Christian Paquin and Greg Zaverucha, *U-prove cryptographic specification v1.1*, Technical Report Revision 3, Microsoft Corporation, December 2013, Accessed: 2025-09-01.

San20. Olivier Sanders, *Efficient redactable signature and application to anonymous credentials*, PKC 2020, Part II (Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, eds.), LNCS, vol. 12111, Springer, Cham, May 2020, pp. 628–656.

Sch90. Claus-Peter Schnorr, *Efficient identification and signatures for smart cards*, CRYPTO'89 (Gilles Brassard, ed.), LNCS, vol. 435, Springer, New York, August 1990, pp. 239–252.

SKSW22. Yumi Sakemi, Tetsutaro Kobayashi, Tsunekazu Saito, and Riad S. Wahby, *Pairing-friendly curves*, Internet-Draft, draft-irtf-cfrg-pairing-friendly-curves-11, Internet Engineering Task Force, November 2022, Work in Progress.

ST24. Olivier Sanders and Jacques Traoré, *Compact issuer-hiding authentication, application to anonymous credential*, PoPETs **2024** (2024), no. 3, 645–658.

TG23. Lindsey Tulloch and Ian Goldberg, *Lox: Protecting the social graph in bridge distribution*, PoPETs **2023** (2023), no. 1, 494–509.

TZ23. Stefano Tessaro and Chenzhi Zhu, *Revisiting BBS signatures*, EUROCRYPT 2023, Part V (Carmit Hazay and Martijn Stam, eds.), LNCS, vol. 14008, Springer, Cham, April 2023, pp. 691–721.

YW25. Cathie Yun and Christopher A. Wood, *Anonymous Rate-Limited Credentials*, Internet-Draft draft-yun-cfrg-arc-01, Internet Engineering Task Force, August 2025, Work in Progress.

A Extended Related Work

Anonymous Credentials. Over the years, a number of concrete systems have shown how ACs can be realized in practice. Microsoft's U-Prove [PZ13] builds

on blind-signature techniques, while IBM's Idemix [CV02] relies on signature schemes tailored for efficient zero-knowledge proofs. In the blind-signature approach, privacy comes from the algebraic properties of blind Schnorr signatures. In the second approach, the issuer signs a commitment to the user's attributes, and the user can later prove knowledge of this signature while revealing only selected information. Both approaches showed that ACs are possible in practice, but they also come with important costs and restrictions. Baldimtsi and Lysyanskaya [BL13] proposed a one-show anonymous credential, termed "AC light" (ACL). In this setting, users must obtain from the issuer as many credential copies as the number of intended showings. This design, however, entails a tradeoff between privacy and efficiency: either users acquire a sufficiently large batch of ACLs in advance—bounded by an estimate of their lifetime usage—or they request re-issuance once exhausted, thereby leaking information about their credential usage patterns.

SNARK-based Anonymous Credentials. General-purpose zero-knowledge systems have been explored as a way to bring more flexibility to credential design. For example, Frigo and shelat [Fs24, Goo25] proposed a technique that combines Ligero [AHIV17] and the GKR sum-check protocols [GKR08], allowing users to prove possession of valid ECDSA-signed credentials without revealing their device public key over mobile Driving Licence (mDL) data format [ISO21]. This design shows that zero-knowledge can address concrete privacy flaws in existing credential standards like mDL, while remaining compatible with current infrastructure and feasible on smartphones. Nonetheless, it comes with reletively higher computational costs and larger proof sizes compared to specialized credential schemes. Similarly, Semaphore [Eth19] uses zk-SNARKs to build a privacy-preserving identity system for Ethereum, enabling users to prove membership in a group without revealing their identity. While these systems offer great flexibility, they often require more computational resources and larger proofs than traditional credential schemes, making them less suitable for resource-constrained environments.

Anonymous authentication Schemes with Keyed Verification. A major step forward for pairing-free settings came with the introduction of keyed-verification anonymous credentials (KVACs), first proposed by Chase, Meiklejohn, and Zaverucha [CMZ14]. By replacing public-key signatures with algebraic message authentication codes, KVACs avoid pairings while preserving efficient proof techniques. Verification, however, requires knowledge of the issuer's secret key, which prevents public verifiability and makes deployment difficult in scenarios with many verifiers. Orrú [Orr24] revisited the foundations of KVACs and formalized stronger notions such as extractability and one-more unforgeability. His work introduced optimized variants, notably μCMZ and μBBS, that achieve constant-size issuance and statistical anonymity in the algebraic group model. These results not only improve efficiency but also clarify the exact security guarantees offered by pairing-free credentials. Despite these advances, KVACs remain constrained by the requirement that issuer and verifer share a secret key, which

restricts scalability and limits applicability in decentralized or large-scale identity systems.

Another important line of research builds on BBS-style credentials. BBS and BBS+ enable efficient selective disclosure proofs and are naturally publicly verifiable [BBC+24]. Barki, et al. [BBDT16] proposed BBS-based KVACs that reduce reliance on pairings while retaining compactness and flexibility. Orrù [Orr24] further analyzed and refined this line, proposing μBBS as a statistically anonymous, pairing-free alternative. Very recently, Mirzamohammadi et al. [MBS+25] proposed two KVAC constructions with short presentation size: one based on pairing-friendly curves, and another pairing-free variant that, however, the latter comes with credentials of linear size in the number of attributes. These developments highlight an interesting trade-off: these constructions yield compact and efficient credentials, but rely on pairing-friendly curves or stronger algebraic assumptions, while their KVAC counterparts avoid pairings but remain tied to issuer-managed secret keys.

While KVACs assume a symmetric key between the issuer and the verifier, there are previous works that define a keyed verification anonymous authentication schemes where the verifier keys are distinct from the issuer keys. However, none of them fit into our setting. Furthermore, we are not aware of instantiations of them in pairing-free primed order groups. Below we list these works.

Derler et al. proposed *signer-anonymous designated-verifier redactable signatures* [DKS16]. In a nutshell, this primitive extends group signatures to the redactable and designated-verifier setting. This primitive provides anonymous authentication for its signers as they are registered by the group manager. However, this scheme does not allow group manager to certify attributes of users/signers. Furthermore, its anonymity holds only conditionally as the group manager can de-anonymize the users.

Recently, Debes et al. proposed *designated-verifier anonymous credentials* [DG23]. Their scheme allows publicly verifiable credentials which are designated to a verifier in the presentation phase. The motivation of their design is to achieve deniability for anonymous credential schemes.

Sanders and Traore recently proposed an issuer-hiding anonymous credential scheme [ST24]. Issuer hiding ACs allow predicates to set multiple possible issuers to present a credential [BEK+21]. Then a user holding a credential from any of them can authenticate without revealing the exact issuer that they hold a credential from. The authors define a key pair to dedicated to each credential predicate. Then the verifier stores the secret key to verify presentations for this predicate. Their main motivation to define keyed verification is obtaining efficiency in the construction level.

Server-Aided Anonymous Credentials. Server-Aided Anonymous Credentials (SAAC) [CAHLT25] were introduced to overcome the tension between efficiency and public verifiability of KVACs. In this model, a helper server, operated by the issuer, transforms private KVAC presentations into publicly verifiable proofs. The helper is designed to remain oblivious to the user's attributes, so it does not learn sensitive information. This construction allows pairing-free systems to

achieve public verifiability, a property that is otherwise lost in KVACs, and thus makes them more suitable for open deployments such as the European Digital Identity Wallet. The drawback is the reliance on additional infrastructure: the helper server becomes a potential point of failure and increases operational complexity, though in return it enables verifiers to be fully decoupled from issuers. The SAAC framework includes both BBS-based instantiations, which achieve compactness and statistical anonymity under gap assumptions, and DDH-based variants, which avoid gap assumptions at some additional efficiency cost.

In parallel, blind-signature based credentials have remained an attractive option for lightweight deployments. Recent work on anonymous tokens and VOPRF-based credentials shows that such systems are highly efficient for single-use or limited-show scenarios, which explains their adoption by major platforms. However, they lack the generality of multi-show credentials and require re-issuance for repeated use, which may reintroduce linkability or usage leakage without careful system design.

Table 2 provides a comprehensive comparison of various anonymous credential schemes and related systems, including traditional anonymous credentials, SNARK-based anonymous credentials, keyed-verification anonymous credentials (KVAC), server-aided anonymous credentials (SAAC), and our proposed multi-verifier KVAC (mKVAC).

Table 2. Comparison of Anonymous Credential and Related Schemes. The number of attributes is n. Credential size excludes storage of attributes. Showing size depends on the number of disclosed attributes and is given as a close-to-tight upper-bound. Denote $\mathbb{G}$ and $\mathbb{Z}_p$ as the sizes of group elements and scalars, respectively. All security analyses assume the ROM unless otherwise noted. *: Showing requires helper server interaction which can be performed offline and batched. **: Multi-Verifier. AGM: Algebraic Group Model, GGM: Generic Group Model.

Scheme	Public Verifiable	Multi-Show	Pairing Free	Credential Size	Showing Size	Security Assumption	Anonymity Level
Traditional Anonymous Credentials							
CL01/03 [CL01, CL03]	✓	✓	✓	$O(1)$	$O(n)$	Strong RSA/ROM	Computational
ACL [BL13, KLR23]	✓	×	✓	$2\mathbb{G} + 6\mathbb{Z}_p$	$2\mathbb{G} + (n+8)\mathbb{Z}_p$	AGM+DL	Statistical
BBS04/TZ23 [BBS04, TZ23]	✓	✓	×	$1\mathbb{G}_1 + \mathbb{Z}_p$	$2\mathbb{G}_1 + (n+3)\mathbb{Z}_p$	q-SDH	Statistical
Brands/U-Prove [Bra95, PZ13]	✓	×	✓	$O(n)$	$O(n)$	DL	Computational
SNARK-based Anonymous Credentials							
LongFellow-zk [Fs24, Goo25]	✓	✓	✓	$O(n)$	$O(\mathsf{polylog}(n))$	Sum-check & IOP	Statistical
Semaphore [Eth19]	✓	✓	×	$O(\log(n))$	$O(1)$	SNARKs	Computational
Keyed-Verification Anonymous Credentials (KVAC)							
CMZ14 [CMZ14]	×	✓	✓	$2\mathbb{G}$	$(n+2)\mathbb{G} + (2n+2)\mathbb{Z}_p$	GGM	DDH
DGSTV18 [DGS+18]	×	✓	✓	$2\lambda + \mathbb{G}$	4λ	RO + gap-OMCDH	Statistical
BBDT16 [BBDT16]	×	✓	✓	$2\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n+7)\mathbb{Z}_p$	Gap-q-SDH	Statistical
CDDH19 [CDDH19]	×	✓	✓	$(n+1)\mathbb{G}$	$2\mathbb{G} + (n+1)\mathbb{Z}_p$	n-SCDHI	Statistical
CPZ20 (Signal) [CPZ20]	×	✓	✓	$2\mathbb{G} + \mathbb{Z}_p$	$(n+3)\mathbb{G} + 4\mathbb{Z}_p$	GGM	DDH
μCMZ [Orr24]	×	✓	✓	$2\mathbb{G}$	$(n+2)\mathbb{G} + (2n+2)\mathbb{Z}_p$	AGM+3-DL	Statistical
μBBS [Orr24]	×	✓	✓	$1\mathbb{G} + 1\mathbb{Z}_p$	$2\mathbb{G} + (n+4)\mathbb{Z}_p$	AGM+q-DL	Statistical
MBS+25 [MBS+25]	×	✓	✓	$(n+2)\mathbb{G}$	$2\mathbb{G}$	GGM	Statistical
Server-Aided Anonymous Credentials (SAAC)							
SAAC_{BBS} [CAHLT25]	✓	×*	✓	$1\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n+8)\mathbb{Z}_p$	Gap-q-SDH	Statistical
SAAC_{DDH} [CAHLT25]	✓	×*	✓	$4\mathbb{G}$	$(n+6)\mathbb{G} + (4n+11)\mathbb{Z}_p$	DDH	DDH
Multi-Verifier KVAC (This Work)							
mKVAC	✓**	✓	✓	$2\mathbb{G}$	$(n+3)\mathbb{G} + (2n+4)\mathbb{Z}_p$	GGM	Statistical

$\underline{\mathsf{Exp}^{\mathsf{CMZ\text{-}Unf},n}_{\mathcal{A}}(\lambda)}$
$Q := \emptyset,\ \mathsf{pp} \leftarrow \mathsf{CMZ.Setup}(\lambda, n)$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{CMZ.Kg}()$
$(\sigma^*, \vec{m}^*) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Sign}, \mathcal{O}\mathsf{Vf}, \mathcal{O}\mathsf{Help}}(\mathbb{G}, p, G, n, \mathsf{pk})$
return $\mathsf{Vf}(\mathsf{sk}, \sigma^*, \vec{m}^*) \ \wedge\ \vec{m}^* \notin Q$

$\underline{\mathcal{O}\mathsf{Sign}(\vec{m} \in \mathbb{Z}_p^n)}$
$Q := Q \cup \{\vec{m}\}$
return $\mathsf{CMZ.Sign}(\mathsf{sk}, \vec{m})$

$\underline{\mathcal{O}\mathsf{Help}(U_0, \ldots, U_n, V)}$
return $V = \sum_{j=0}^{j=n} x_j U_j$

Fig. 14. CMZ unforgeability game [Orr24].

B Further Preliminaries

Definition 10 (DDH Assumption). DDH *assumption holds if the following advantage is negligible for all* $\lambda \in \mathbb{N}$ *and* $\mathcal{A}$*:*

$$\mathsf{Adv}^{\mathsf{DDH}}_{\mathcal{A}}(\lambda) := \left| \Pr\left[\begin{array}{l} (\mathbb{G}, p, G) \leftarrow \mathsf{GGen}(\lambda) \\ b \leftarrow \{0,1\},\ a_{\mathsf{DDH}}, b_{\mathsf{DDH}}, c_{\mathsf{DDH}} \leftarrow \mathbb{Z}_p \\ A := a_{\mathsf{DDH}} G,\ B := b_{\mathsf{DDH}} G \\ C := (c_{\mathsf{DDH}} \cdot b + a_{\mathsf{DDH}} \cdot b_{\mathsf{DDH}}) G \\ b' \leftarrow \mathcal{A}((\mathbb{G}, p, G), A, B, C) \end{array} : b = b' \right] - 1/2 \right|.$$

Definition 11 (n-DL Assumption). n-DL *assumption holds if the following advantage is negligible for all* $\lambda \in \mathbb{N}$ *and* $\mathcal{A}$*:*

$$\mathsf{Adv}^{n\text{-}\mathsf{DL}}_{\mathcal{A}}(\lambda) := \Pr\left[\begin{array}{l} (\mathbb{G}, p, G) \leftarrow \mathsf{GGen}(\lambda),\ a \xleftarrow{\$} \mathbb{Z}_p \\ \textbf{for } i \in [n] \ :\ A_i := a^i G \\ a' \leftarrow \mathcal{A}((\mathbb{G}, p, G), (A_i)_{i \in [n]}) \end{array} : a = a' \right].$$

Corollary 1. CMZ MAC *from Figure 2 is unforgeable for all* $n = poly(\lambda)$ *with the advantage* $\mathsf{Adv}^{\mathsf{CMZ\text{-}Unf},n}_{\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{3\text{-}\mathsf{DL}}_{\mathcal{A}_{3\text{-}\mathsf{DL}}}(\lambda) + \mathsf{Adv}^{1\text{-}\mathsf{DL}}_{\mathcal{A}_{1\text{-}\mathsf{DL}}}(\lambda) + 3/p$.

NIZK Definitions. Below we present the formal definitions of NIZK properties completeness, zero-knowledge, and simulation-extractability.

We start with the definition of the completeness property. Note that the previous work on Fischlin transform considers overwhelming completeness whereas our definition is for perfect completeness. Any overwhelmingly complete NIZK proof scheme can be turned into a perfectly complete one as follows: The proving algorithm is updated such that if it fails to generate a valid proof, it reveals the witness as the proof. The verification either verifies a valid proof or checks the revealed witness is valid for the statement. This protocol achieves perfect completeness and simulation-extractability is unaffected. The zero-knowledge property of the extended protocol can e shown by relying on the zero-knowledge property and the overwhelming completeness of the initial NIZK proof scheme.

Definition 12 (Completeness). *A* NIP *scheme for relation* $\mathcal{R}$ *has is* complete *if for all* $\lambda \in \mathbb{N}$, $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, $\mathsf{ctx}_{\mathsf{NIP}} \in \{0,1\}^\star$, *and* $\mathsf{pp}_{\mathsf{NIP}} \leftarrow \mathsf{Setup}(\lambda)$, $\pi \leftarrow \mathsf{NIP.Prove}(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}})$*, we have that* $\Pr[\mathsf{NIP.Vf}(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi)] = 1$.

$\underline{\mathsf{Exp}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S}}(\lambda)}$
$b \xleftarrow{\$} \{0,1\}$
if $b = 0$: $\mathsf{pp}_{\mathsf{NIP}} \leftarrow \mathsf{Setup}^{\mathsf{RO}}(\lambda)$
if $b = 1$: $(\mathsf{pp}_{\mathsf{NIP}}, td) \leftarrow \mathsf{S.Setup}^{\mathsf{Pr}}(\lambda)$
$b' \leftarrow \mathcal{A}^{\mathsf{RO},\mathcal{O}\mathsf{Prove}}(\mathsf{pp}_{\mathsf{NIP}})$
return $b = b'$

$\underline{\mathsf{Pr}(x, v)}$
if $\mathsf{ROT}[x] = \perp$: $\mathsf{ROT}[x] := v$

$\underline{\mathcal{O}\mathsf{Prove}(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}})}$
require $\mathcal{R}(\mathbb{x}, \mathbb{w})$
if $b = 0$: $\pi \leftarrow \mathsf{Prove}^{\mathsf{RO}}(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}})$
if $b = 1$: $\pi \leftarrow \mathsf{S.Prove}^{\mathsf{Pr}}(td, \mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}})$
return π

$\underline{\mathsf{RO}(x)}$
if $\mathsf{ROT}[x] = \perp$: $\mathsf{ROT}[x] \xleftarrow{\$} \{0,1\}^{\lambda}$
$Q_{\mathsf{RO}} := Q_{\mathsf{RO}} \cup \{(x, \mathsf{ROT}[x])\}$
return $\mathsf{ROT}[x]$

Fig. 15. Zero-knowledge game.

$\underline{\mathsf{Exp}^{\mathsf{SE}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S},\mathsf{Extract}}(\lambda)}$
$\mathsf{success} := \mathsf{false}$, $Q_s := \emptyset$
$(\mathsf{pp}_{\mathsf{NIP}}, td) \leftarrow \mathsf{S.Setup}^{\mathsf{Pr}}(\lambda)$
$\mathcal{A}^{\mathsf{RO},\mathcal{O}\mathsf{Prove},\mathcal{O}\mathsf{Chall}}(\mathsf{pp}_{\mathsf{NIP}})$
return success

$\underline{\mathcal{O}\mathsf{Prove}(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}})}$
require $\mathcal{R}(\mathbb{x}, \mathbb{w})$
$\pi \leftarrow \mathsf{S.Prove}^{\mathsf{Pr}}(td, \mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}})$
$Q_s := Q_s \cup \{(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi)\}$
return π

$\underline{\mathcal{O}\mathsf{Chall}(\mathbb{x}, \pi, \mathsf{ctx}_{\mathsf{NIP}})}$
if $\mathsf{Vf}(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi) \wedge (\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi) \notin Q_s$:
$\mathbb{w} \leftarrow \mathsf{Extract}(\mathbb{x}, \mathsf{ctx}_{\mathsf{NIP}}, \pi, Q_{\mathsf{RO}})$
if $\neg\mathcal{R}(\mathbb{x}, \mathbb{w})$: $\mathsf{success} := \mathsf{true}$

Fig. 16. Simulation-extractability game.

Definition 13 (Zero-knowledge). *A* NIP *scheme is zero-knowledge if there exists algorithms* $\mathsf{S} := (\mathsf{Setup}, \mathsf{Prove})$ *such that for all PPT adversaries* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S}}(\lambda) = \mathsf{true}\right] - 1/2\right| \leq \mathsf{negl}(\lambda).$$

Definition 14 (Simulation-extractability). *A* NIP *scheme is simulation extractable for a simulator* S *if there exists an algorithm* Extract *such that for all PPT adversaries* $\mathcal{A}$*,* $\mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S},\mathsf{Extract}}(\lambda) := \Pr\left[\mathsf{Exp}^{\mathsf{SE}}_{\mathcal{A},\mathsf{NIP}_{\mathcal{R}},\mathsf{S},\mathsf{Extract}}(\lambda) = \mathsf{true}\right] \leq \mathsf{negl}(\lambda)$.

C Non-interactive zero-knowledge proofs

C.1 Base Schnorr-style proofs

At the heart of our protocols lies (a generalization of) the Schnorr protocol [Sch90, Mau09], applying the Fiat-Shamir transform [FS87] to it (we discuss Fischlin's transform [Fis05, Ks22, LR22] later). This protocol proves knowledge of a homomorphism preimage. For our purposes, let $\varphi : \mathbb{Z}_p^{\ell_{\mathbb{w}}} \to \mathbb{G}^{\ell_{\mathbb{i}}}$ be a homomorphism. Let $\mathsf{H} : \{0,1\}^* \to \mathbb{Z}_p$ be a hash function modeled as a random oracle.

The prover on input $(\mathbb{x}, \mathbb{w}, \mathsf{ctx}_{\mathsf{NIP}})$ first derives homomorphism image $\mathbb{x}_{\text{Schnorr}} \in \mathbb{G}^{\ell_{\text{i}}}$ from $\mathbb{x}$ (this depends on the specific protocol) and secret homomorphism preimage $\mathbb{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_{\text{w}}}$ from $\mathbb{w}$ (again, depending on the protocol). The prover then proceeds as follows.

1. Generate a random $a \xleftarrow{\$} \mathbb{Z}_p^{\ell_{\text{w}}}$ and compute the announcement $A_{\text{Schnorr}} = \varphi(a) \in \mathbb{G}^{\ell_{\text{i}}}$.
2. Compute the challenge $c := \mathsf{H}(\mathsf{ProtName}, \mathsf{ctx}_{\mathsf{NIP}}, \mathbb{x}, A_{\text{Schnorr}}) \in \mathbb{Z}_p$. Here, $\mathsf{ProtName}$ is a protocol-specific constant identifier string to avoid potential cross-protocol attacks (e.g., $\mathsf{ProtName}$ = "AKVAC-Req").
3. Compute the response $s := a + c \cdot \mathbb{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_{\text{w}}}$.
4. Output the proof $\pi := (c, s) \in \mathbb{Z}_p^{1+\ell_{\text{w}}}$.

The verifier on input $(\mathbb{x}, \pi, \mathsf{ctx}_{\mathsf{NIP}})$, where $\pi = (c, s) \in \mathbb{Z}_p^{1+\ell_{\text{w}}}$, proceeds as follows. The verifier derives $\mathbb{x}_{\text{Schnorr}}$ from $\mathbb{x}$ (like the prover; again, the specifics depend on the actual protocol). The verifier then proceeds as follows.

1. Compute the unique accepting announcement $A_{\text{Schnorr}} := \varphi(s) - c \cdot \mathbb{x}_{\text{Schnorr}} \in \mathbb{G}^{\ell_{\text{i}}}$.
2. Accept the proof if and only if $c = \mathsf{H}(\mathsf{ctx}_{\mathsf{NIP}}, \mathbb{x}, A_{\text{Schnorr}})$.

C.2 Application to our protocols

Below, we apply this paradigm to the protocols in this paper. We focus on the $\mathsf{mKVAC}_{\mathsf{CMZ}}$ construction (Section 5) instantiated with the $\mathsf{BBSSAGA}$ (Section 4.2). By default, we will assume that the attribute predicate ϕ is trivial, i.e. $\phi((a_j)_{j=1}^n) = \mathsf{true}$ for all a_j. This means that all attributes are perfectly hidden from the issuer/verifier. We discuss partial disclosure as a meaningful predicate in the relevant parts below.

$\mathsf{BBSSAGA}$ *NIZK proving correct MAC computation.* Consider $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ (Section 4.2). This proof is a straightforward application of the framework above.

Statement. The prover and verifier computes $\mathbb{x}_{\text{Schnorr}} := (X, (Y_1, \ldots, Y_\ell), dA - G_0)$ (where the variables are from $\mathbb{x}_{\mathsf{BBSSAGA}}$ and $\mathsf{pk}_{\mathsf{SAGA}}$).
Witness. The prover sets the witness $\mathbb{w}_{\text{Schnorr}} := (x, y_1, \ldots y_\ell) := \mathbb{w}_{\mathsf{BBSSAGA}}$.
Homomorphism. The homomorphism φ is defined as follows.

$$
\begin{aligned}
&\varphi(x, y_1, \ldots, y_\ell) = \\
&\Big(\\
&\quad xG, /\!\!/ = X \\
&\quad (y_1 G_1, \ldots, y_\ell G_\ell), /\!\!/ = (Y_1, \ldots, Y_\ell) \\
&\quad - xA + \sum_{j \in [\ell]} y_j M_j /\!\!/ = dA - G_0 \\
&\Big)
\end{aligned}
$$

With these values, the prover and verifier can compute and verify the Schnorr-style proofs as described above.

$\mathsf{mKVAC}_{\mathsf{CMZ}}$ *request NIZK.* Consider NIP_R (Section 5). For this proof, we need to handle the non-linear term $d \cdot r$ in $\mathsf{PresWitVf}$. We do so by having the prover commit to d as $ProdCom = dG + \eta H$ and then prove that he can open $rProdCom$ to $d \cdot r$.

The overall protocol NIP_R works as follows.

Statement. The prover computes an additional commitment $ProdCom = dG + \eta H$ for random $\eta \xleftarrow{\$} \mathbb{Z}_p$ and includes $ProdCom$ in the final proof π_R and in the challenge hash $c = \mathsf{H}(ProdCom, \dots)$. Both prover and verifier compute the homomorphism image as $\mathbb{x}_{\text{Schnorr}} := (\bar{X}_0 - C_{n+1}, \bar{Z}_0 - C_{n+2}, C_a, T, ProdCom, 0)$ (where T is from $\mathsf{pres}_{\mathsf{SAGA}}$, $ProdCom$ is chosen by the prover and taken from the final proof by the verifier, and the other variables are from $\mathbb{x}_R$).

Witness. The prover computes the witness $\mathbb{w}_{\text{Schnorr}} := \big(s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{\nu}, \mathsf{wit}_{\mathsf{SAGA}} = (r, d), \xi_1, \dots, \xi_{n+2}, \eta, prod, prod'\big)$, where $\xi'_j = a_j \xi_j$, the ξ_j being as output by the SAGA scheme, and $prod = d \cdot r, prod' = r \cdot \eta$.

Homomorphism. The homomorphism φ is defined as follows.

$$
\begin{aligned}
&\varphi(s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{\nu}, \mathsf{wit}_{\mathsf{SAGA}} = (r, d), \xi_1, \dots, \xi_{n+2}, \eta, prod, prod') = \\
&\big(\\
&\quad - \xi_{n+1} G_{n+1} + \bar{x}_0 G + \bar{\nu} E, /\!/ = \bar{X}_0 - C_{n+1} \\
&\quad - \xi_{n+2} G_{n+2} + \bar{\nu} G, /\!/ = \bar{Z}_0 - C_{n+2} \\
&\quad sG + \Big(\sum_{j \in [n]} a_j C_j\Big) - \Big(\sum_{j \in [n]} \xi'_j G_j\Big), /\!/ = C_a \\
&\quad rX - dC_A + prodG - \sum_{j=1}^{\ell} \xi_j Y_j, /\!/ = T /\!/ \ \mathsf{SAGA} \text{ check} \\
&\quad dG + \eta H /\!/ = ProdCom, /\!/ \text{ product check} \\
&\quad prodG + prod' H - rProdCom /\!/ = 0 /\!/ \text{ product check} \\
&\big)
\end{aligned}
$$

With these values, the prover and verifier can compute and verify the Schnorr-style proofs as described above.

As mentioned above, this description assumes a trivial presentation predicate ϕ. As an important example, partial disclosure can be implemented as follows: Assume ϕ is such that a_1 is disclosed. The prover would omit a_1 from the witness $\mathbb{w}_R$. The C_a in the statement $\mathbb{x}_{\text{Schnorr}}$ is changed to $C_a - a_1 C_1$ (which both prover and verifier can compute knowing the disclosed a_1). The homomorphism φ would omit the term $a_1 C_1$ from the sum. Regarding performance, the change towards partially disclosing a_1 makes the proof π_R smaller by one $\mathbb{Z}_p$ element, corresponding to the witness a_1. Computational costs do not materially change.

$\mathsf{mKVAC}_{\mathsf{CMZ}}$ *issuance NIZK.* Consider NIP_I (Section 5). This proof straightforward to implement. There is a non-linear term $u \cdot e$ in the relation, but since we have uG and eG publicly available, this is easy to handle.

Statement. Prover and verifier compute $\mathbb{x}_{\text{Schnorr}} := (\bar{U}, E, 0, \bar{V})$ (where the variables are from ipk and $\mathbb{x}_I$).
Witness. The prover sets the witness $\mathbb{w}_{\text{Schnorr}} := (e, u, prod)$, where $prod := e \cdot u$ and e, u are from $\mathbb{w}_I$.
Homomorphism. The homomorphism φ is defined as follows.

$$\begin{aligned}&\varphi(e, u, prod) = \\ &\big(\\ &\quad uG, /\!/ = \bar{U} \\ &\quad eG, /\!/ = E \\ &\quad prodG - uE, /\!/ = 0 \\ &\quad u\bar{X}_0 - prod\bar{Z}_0 + uC_a /\!/ = \bar{V} \\ &\big)\end{aligned}$$

With these values, the prover and verifier can compute and verify the Schnorr-style proofs as described above.

$\mathsf{mKVAC}_{\mathsf{CMZ}}$ *presentation NIZK.* Consider NIP_S (Section 5). This proof is an OR proof. We first describe the main parts of the proof and then explain to combine them into an OR proof.

Statement. Prover and verifier compute $\mathbb{x}^{(1)}_{\text{Schnorr}} := (Z, (C_j)_{j=1}^n)$ (where the variables are from $\mathbb{x}_S$) and $\mathbb{x}^{(2)}_{\text{Schnorr}} := X_1$ (from vpk).
Witness. The prover sets the witness $\mathbb{w}^{(1)}_{\text{Schnorr}} := (\tilde{\gamma}, (a_j, \gamma_j)_{j=1}^n)$ from $\mathbb{w}_S$. The witness $\mathbb{w}^{(2)}_{\text{Schnorr}} = \bot$ is unused for honest users.
Homomorphism. The homomorphism $\varphi^{(1)}$ is defined as follows.

$$\begin{aligned}&\varphi^{(1)}(\tilde{\gamma}, (a_j, \gamma_j)_{j=1}^n) = \\ &\quad\big(\\ &\qquad \sum_{j=1}^{n} \gamma_j X_j - \tilde{\gamma} H, /\!/ = Z \\ &\qquad (a_j \tilde{U} + \gamma_j G)_{j=1}^n /\!/ = (C_j)_{j=1}^n \\ &\quad\big)\end{aligned}$$

The homomorphism $\varphi^{(2)}$ is defined as follows.

$$\varphi^{(2)}(x_1) = x_1 G /\!/ = X_1$$

With these values, we can construct the proper OR proof following [CDS94] as follows. We only describe the honest user case here, i.e., the prover knows a witness for the first statement. The prover then proceeds as follows.

1. Derive $\mathbb{x}^{(1)}_{\text{Schnorr}}, \mathbb{x}^{(2)}_{\text{Schnorr}}, \mathbb{w}^{(1)}_{\text{Schnorr}}$ from $\mathbb{x}$ as above.

2. Compute a simulated transcript for the second statement: pick random challenge $c^{(2)} \xleftarrow{\$} \mathbb{Z}_p$ and response $s^{(2)} \xleftarrow{\$} \mathbb{Z}_p$, and set the announcement $A^{(2)}_{\text{Schnorr}}$ to the unique accepting value $A^{(2)}_{\text{Schnorr}} := \varphi^{(2)}(s^{(2)}) - c^{(2)} \cdot \mathbb{x}^{(2)}_{\text{Schnorr}}$.
3. Generate a random $a^{(1)} \xleftarrow{\$} \mathbb{Z}_p^{\ell_{\text{w}}}$ and compute the announcement $A_{\text{Schnorr}} = \varphi^{(1)}(a^{(1)}) \in \mathbb{G}^{\ell_{\text{i}}}$.
4. Compute the challenge $c := \mathsf{H}(\mathsf{ProtName}, \mathsf{ctx}_{\mathsf{NIP}}, \mathbb{x}, A^{(1)}_{\text{Schnorr}}, A^{(2)}_{\text{Schnorr}}) \in \mathbb{Z}_p$.
5. Derive the challenge for the first protocol as $c^{(1)} := c - c^{(2)}$.
6. Compute the first protocol response $s^{(1)} := a^{(1)} + c^{(1)} \cdot \mathbb{w}^{(1)}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_{\text{w}}}$.
7. Output the proof $\pi := (c^{(1)}, c^{(2)}, s^{(1)}, s^{(2)}) \in \mathbb{Z}_p^{2+\ell_{\text{w}}+1}$.

The verifier on input $(\mathbb{x}, \pi, \mathsf{ctx}_{\mathsf{NIP}})$, where $\pi = (c^{(1)}, c^{(2)}, s^{(1)}, s^{(2)}) \in \mathbb{Z}_p^{2+\ell_{\text{w}}+1}$, proceeds as follows. The verifier derives $\mathbb{x}^{(1)}_{\text{Schnorr}}, \mathbb{x}^{(2)}_{\text{Schnorr}}$ from $\mathbb{x}$ as described above. The verifier then proceeds as follows.

1. Compute the unique accepting announcements $A^{(1)}_{\text{Schnorr}} := \varphi^{(1)}(s) - c^{(1)} \cdot \mathbb{x}^{(1)}_{\text{Schnorr}}$ and $A^{(2)}_{\text{Schnorr}} := \varphi^{(2)}(s) - c^{(2)} \cdot \mathbb{x}^{(2)}_{\text{Schnorr}}$.
2. Accept the proof if and only if $c^{(1)} + c^{(2)} = \mathsf{H}(\mathsf{ctx}_{\mathsf{NIP}}, \mathbb{x}, \mathbb{x}, A^{(1)}_{\text{Schnorr}}, A^{(2)}_{\text{Schnorr}})$.

As described for NIP_Rabove, one can implement nontrivial predicates ϕ by adding statements to the proof. To implememnt partial disclosure of a_1, the prover would omit a_1 from $\mathbb{w}_{\text{Schnorr}}$, C_1 in $\mathbb{x}_{\text{Schnorr}}$ becomes $C_1 - a_1\tilde{U}$, and the $a_j\tilde{U}$ term in the homomorphism $\varphi^{(1)}$ is omitted.

C.3 Fischlin transform

We describe the protocols above using the Fiat-Shamir transform. For our security proofs, we require straightline extractability, which Fiat-Shamir does not formally provide. We do note that in practice, Fiat-Shamir is widely used and considered secure, and may be heuristically used to instantiate our proofs. However, for formal completeness, we describe how to apply Fischlin's transform [Fis05, Ks22, LR22] to our protocols. In order to apply the randomized Fischlin transform [Ks22, LR22], the underlying sigma protocol must satisfy strong special-soundness. Strong special soundness requires that there exists an efficient extractor such that given two valid transcripts (R, c, z) and (R, c', z') such that $(c, z) \neq (c', z')$, the extractor outputs a valid witness.

For Schnorr-style protocols, strong special-soundness corresponds to φ being collision-resistant, i.e. it must be hard for the prover to find s such that $\varphi(s) = 0$. $\mathsf{NIP}_{\mathsf{BBSSAGA}}$'s and NIP_I's homomorphisms are injective, hence the corresponding protocols have unique responses and hence strong special-soundness. NIP_S is an OR proof, which has strong special-soundness if the underlying Sigma protocols have strong special soundness. This is the case for our underlying Sigma (Schnorr-style) protocols: φ_1 of NIP_S is collision-resistant assuming the prover does not know the discrete logarithm relation between $\tilde{U}$ and G (and that it is hard to compute). φ_2 is injective. Finally, NIP_R's homomorphism φ is not collision-resistant in general, because the user chooses C_j (the blinded attributes)

and C_A (the blinded MAC). However, forcing the user to Pedersen-commit to the requested attributes $\vec{a}$ and extending the homomorphism φ accordingly will make φ collision-resistant assuming Pedersen commitments are binding. Hence straightforward adaptations to NIP_R are required to apply Fischlin's transform.

Necessity of Straightline-Extractability. We remark that the straightline extractability requirement for NIP_S arises because we choose to rely on CMZ MAC unforgeability in a black-box manner.

Specifically, when reducing to the unforgeability of a signature/MAC from a NIZK proof containing a signature/tag as a witness, rewinding the adversary is problematic: signing queries may differ across branches of the rewinding, so a forgery in one branch may not hold for the reduction that runs all branches. This is mostly a technical issue and can be usually addressed with a white-box reduction to the underlying assumptions of signature/MAC scheme. Previous works deviate in their choice of reliance on a straightline extractor in their presentation as well: some opt for straightline extractors like us [Orr24, CPZ20] whereas some prefer to provide a dedicated reduction to the underlying computational assumption [CAHLT25]. We omit such a reduction here to emphasize the generic nature of our approach, and leave such extensions to future work.

D Additional Evaluation Discussion

We discuss Section 6 in more detail here.

Linear scaling in attribute size. The dominant operations scale *linearly* in the attribute dimension $|\vec{a}|$: Obt_1, Iss, Present, and VfPres. The communication terms that carry per-attribute commitments and proof data (*creq*, *pres*) also increase linearly: for example, pres grows from 0.6 KiB at $|\vec{a}|$=4 to 6.2 KiB at $|\vec{a}|$=64. This matches the design of our relations in Section 5 and the Schnorr-style instantiations in Appendix C.

Issuance vs. presentation. For moderately large attribute sets ($|\vec{a}|$=32), issuance takes on the order of 9.21+8.19 ≈17.4 ms end-to-end ($\mathsf{Obt}_1 + \mathsf{Iss}$), while a single present/verify is ≈ 2.59 ms / 1.96 ms, respectively. For the largest tested set ($|\vec{a}|$=64), issuance is ≈30 ms end-to-end, while present/verify is ≈3.7 ms / 3.0 ms. This is consistent with mKVAC 's design goal: each verifier key requires only a single issuance (non-interactive from the user's perspective), after which the user can perform many efficient presentations without relying on a helper server.

SAGA *overhead.* The consolidated request payload creq grows linearly with $|\vec{a}|$ (e.g., 1.1 KiB at $|\vec{a}|$=4, 8.6 KiB at $|\vec{a}|$=64), while the credential size itself is always constant (0.1 KiB). This reflects the cost of cryptographically authenticating the verifier key while keeping it hidden from the issuer during issuance, as required by our model in Section 4. The overhead remains small and predictable.

Practicality. Overall, the measurements show that mKVAC is highly practical for deployment. Even with dozens of attributes, issuance costs are only a few tens of milliseconds, and presentations are in the low millisecond range — easily fast enough for interactive or real-time use. Communication overheads are modest: even at $|\vec{a}|$=64, the largest payload is just 8.6 KiB, well within what modern networks can handle efficiently. This makes mKVAC suitable for privacy-preserving credential systems with realistic attribute sizes, while also showing promising scalability to larger vectors.

E Comparison to Existing Presentation Anonymity Definitions.

Presentation anonymity definitions differ in three aspects. One is whether the adversary may choose the issuer public key. This is permitted in our model as done by [CAHLT25, ST24, CKL+14], while others assume honestly generated keys [Orr24, CMZ14, CDDH19, FHS19, San20, HS21].

A second difference is how indistinguishability is achieved. We use a left-or-right oracle, requiring that presentations of different credentials in the same anonymity set be indistinguishable [FHS19, San20, HS21, ST24]. Simulator-based definitions [Orr24, CAHLT25, CKL+14, CMZ14, CDDH19], by contrast, generate a simulated presentation from vpk and ϕ that is indistinguishable from a real one. These are particularly useful for certain predicates, e.g. predicates involving encrypted attributes, where the simulator can simulate presentations without learning the plaintext – something not possible by just relying on a left-or-right oracle.

The third aspect is whether presentation anonymity holds if credentials of honest users are revealed to the adversary. This is crucial in practice, since compromising a credential should not allow linking past presentations. Simulator-based models do not provide this guarantee, and some left-or-right definitions also avoid revealing credentials [FHS19, San20, HS21]. Our model always reveals issued credentials, so anonymity relies only on the randomness of the presentation algorithm, similar to Sanders and Traoré [ST24].

F Deferred Content of Section 3

Definition 15 (Correctness). *A scheme* mKVAC *as in Definition 2 is* correct *if for all security parameters $\lambda \in \mathbb{N}$, all possible groups $(\mathbb{G}, p, G) \in [\mathsf{GGen}(1^\lambda)]$, all attribute length parameters n, and all attribute vectors $\vec{a} \in \mathbb{Z}_p^n$:*

$$\Pr\left[\begin{array}{l}\mathsf{Setup}(\lambda, n) \to \mathsf{pp},\\ \mathsf{IKg}(\mathsf{pp}) \to (\mathsf{isk}, \mathsf{ipk}), \mathsf{VKg}(\mathsf{isk}) \to (\mathsf{vsk}, \mathsf{vpk}),\\ \mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}, \vec{a}, \phi) \to (\mathsf{creq}, \mathsf{st}),\\ \mathsf{Iss}(\mathsf{isk}, \phi, \mathsf{creq}) \to \mathsf{bcred}, \mathsf{Obt}_2(\mathsf{st}, \mathsf{bcred}) \to \mathsf{cred},\\ \mathsf{Present}(\mathsf{ipk}, \mathsf{vpk}, \mathsf{cred}, \mathsf{ctx}, \phi) \to \mathsf{pres} :\\ \mathsf{VfPres}(\mathsf{vsk}, \mathsf{ipk}, \mathsf{pres}, \mathsf{ctx}, \phi) = \mathsf{true}\end{array}\right] = 1.$$

G Security proofs of $\mathsf{mKVAC}_{\mathsf{CMZ}}$

G.1 Proof of unforgeability

Theorem 4 (restated). $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *scheme from Figures 10 to 12 is unforgeable if* NIP_I *and* NIP_S *are zero-knowledge,* NIP_R *and* NIP_S *are complete and simulation-extractable,* DDH *is hard, and* SAGA *and* μCMZ *are correct and unforgeable.*

Proof ($\mathsf{mKVAC}_{\mathsf{CMZ}}$ *unforgeability.).* We set the extractors $\mathsf{E.I}$ and $\mathsf{E.P}$ to run the knowledge extractors of NIP_R and NIP_S, respectively, and output the extracted attributes. We prove Theorem 4 by using a sequence of game hops Game_i. The event that the adversary $\mathcal{A}$ wins in Game_i is denoted as $\mathbf{W}_i$. Game_0 is identical to the original unforgeability game.
$\underline{\mathsf{Game}_1}$: This game hop computes some NIP proofs by running their zero-knowledge simulators. Specifically,

- $\pi_I$when answering $\mathcal{O}\mathsf{CCIssue}$ queries
- $\pi_S$when answering $\mathcal{O}\mathsf{HCShow}$ queries

are simulated by their respective zero-knowledge simulators. Note that in Game_0, given correctness of the SAGA scheme, the game always uses valid witnesses to compute these proofs. These changes are indistinguisable by relying on the zero-knowledge properties of NIP_I and NIP_S, so $\Pr[\mathbf{W}_0] \leq \Pr[\mathbf{W}_1] + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathrm{ZK,I}},\mathsf{NIP}_I}(\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathrm{ZK,S}},\mathsf{NIP}_S}(\lambda)$.
$\underline{\mathsf{Game}_2}$: This game hop aborts if $\mathsf{E.I}$ fails to extract a valid witness from a valid π_R. If Game_2 aborts with non-negligible probability, then we can easily build an adversary $\mathcal{A}_{\mathsf{SE},R}$ against the simulation-extractability of NIP_R. Thus, we conclude that $\Pr[\mathbf{W}_1] \leq \Pr[\mathbf{W}_2] + \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE},R},\mathsf{NIP}_R}(\lambda)$.
$\underline{\mathsf{Game}_3}$: This game hop ensures that any adversarially chosen verifier keys vpk correspond to keys generated by the game. Intuitively, this is guaranteed by unforgeability of the SAGA scheme. To formally define this guarantee, Game_3 introduces a book-keeping set VPKSet which is initially empty. Then Game_3 updates VPKSet whenever it runs VKg algorithm. Specifically, when the challenge $\mathsf{vpk}^* := (X_1^*, \ldots, X_n^*, X_0^*, Z_0^*, \tau^*)$ is created, VPKSet is updated as $\mathsf{VPKSet} := \mathsf{VPKSet} \cup \{\mathsf{vpk}^*\}$. Similarly, for each $\mathcal{O}\mathsf{VKg}$ query, the output vpk is added to $\mathsf{VPKSet} := \mathsf{VPKSet} \cup \{\mathsf{vpk}\}$.

We additionally define the following notation:

- During a $\mathcal{O}\mathsf{CCIssue}$ query, let $(\aleph_R, \beth_R)$ be the statement and extracted witness for the adversary's π_R.
- During a $\mathcal{O}\mathsf{HCIssue}(\mathsf{vpk}, a, \phi)$ query, let $\aleph_R, \beth_R$ be as computed by Obt_1.

In either case, we write $\aleph_R =: (C_1, \ldots, C_{n+2}, \bar{X}_0, \bar{Z}_0, C_a)$ and $\beth_R =: \big(s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{\nu}, \mathsf{wit}_{\mathsf{SAGA}}, \xi_1, \ldots, \xi_{n+2}\big)$. We further denote the effective (unblinded) SAGA message $X_j := C_j - \xi_j G_j$ for $j \in [n]$, $X_0 := C_{n+1} - \xi_{n+1} G_{n+1}$, $Z_0 := C_{n+2} - \xi_{n+2} G_{n+2}$.

Game_3 aborts if during a $\mathcal{O}\mathsf{CCIssue}$ or $\mathcal{O}\mathsf{HCIssue}$ query, $(\aleph_R, \beth_R) \in \mathcal{R}_R$ but there is no entry of the form $(X_1, \ldots, X_n, X_0, Z_0, \cdot)$ in VPKSet.

Transition $\mathsf{Game}_2 \to \mathsf{Game}_3$*:* We prove that if Game_3 aborts, we can build a SAGA forger $\mathcal{R}_{\mathsf{UF-CoMA}}$ which runs Game_3 identically except the following changes. In $\mathcal{O}\mathsf{VKg}$, instead of running SAGA.MAC on $(X_1 := x_1G, \ldots, X_n := x_nG, X_0 := (x_0 + e \cdot \nu)G, Z_0 := \nu G)$, $\mathcal{R}_{\mathsf{UF-CoMA}}$ makes an $\mathcal{O}\mathsf{MAC}((x_1, \ldots, x_n, x_0 + e \cdot \nu, \nu))$ query. Similarly, in $\mathcal{O}\mathsf{HCIssue}$ and $\mathcal{O}\mathsf{CCIssue}$, instead of running $\mathsf{SAGA.PresKeyedVf}(\mathsf{isk}, \mathsf{pres}_{\mathsf{SAGA}}, \mathsf{ctx}, C_1, \ldots, C_{n+2})$, $\mathcal{R}_{\mathsf{UF-CoMA}}$ makes an $\mathcal{O}\mathsf{PresKeyedVf}(\mathsf{pres}_{\mathsf{SAGA}}, \mathsf{wit}_{\mathsf{SAGA}}, (C_j, \xi_j)_{j \in [n+2]})$ query, where $\mathsf{wit}_{\mathsf{SAGA}}$ and ξ_j are obtained from $\mathbb{w}_R$.

If any $\mathcal{O}\mathsf{HCIssue}$ or $\mathcal{O}\mathsf{CCIssue}$ query triggers the abort condition of Game_3, $\mathcal{R}_{\mathsf{UF-CoMA}}$ outputs its final $\mathsf{UF-CoMA}$ forgery $(\mathsf{pres}_{\mathsf{SAGA}}, \mathsf{wit}_{\mathsf{SAGA}}, (C_j, \xi_j)_{j \in [n+2]})$. By definition, this is a valid forgery: $\mathsf{SAGA.PresKeyedVf}(\mathsf{pres}_{\mathsf{SAGA}}, \mathsf{wit}_{\mathsf{SAGA}}, (C_j, \xi_j)_{j \in [n+2]}) = \mathsf{true}$, as guaranteed by $(\mathbb{x}_R, \mathbb{w}_R) \in \mathcal{R}_R$, and we have not queried for a SAGA tag on $(X_1, \ldots, X_n, X_0, Z_0)$, as $(X_1, \ldots, X_n, X_0, Z_0) \notin \mathsf{VPKSet}$. Thus, we conclude that $\Pr[\mathbf{W}_2] \leq \Pr[\mathbf{W}_3] + \mathsf{Adv}^{\mathsf{UF-CoMA}}_{\mathcal{R}_{\mathsf{UF-CoMA}}, \mathsf{SAGA}}(\lambda)$.

$\underline{\mathsf{Game}_4}$: This game introduces a random dummy attribute vector a^{rnd}, which willl later be used to answer $\mathcal{O}\mathsf{HCShow}$ queries in Game_5. More specifically, at the start of the game, Game_4 samples a random attribute vector $a^{\mathrm{rnd}} \xleftarrow{\$} \mathbb{Z}_p^n$. At the end of Game_4, if the extractor $\mathsf{E.P}$ outputs $a^* = a^{\mathrm{rnd}}$, then Game_4 aborts.

Since a^{rnd} is not used anywhere and hence perfectly hidden from $\mathcal{A}$ and E, the probability for the new abort condition is at most p^{-n}, so $\Pr[\mathbf{W}_3] \leq \Pr[\mathbf{W}_4] + p^{-n}$.

$\underline{\mathsf{Game}_5}$: This game answers $\mathcal{O}\mathsf{HCShow}$ queries using a MAC σ^{rnd} on a^{rnd} instead of the honest user's actual credential cred_i. More specifically, Game_5 handles $\mathcal{O}\mathsf{HCIssue}(i, \mathsf{ctx}, \phi)$ queries with the following additional behavior:

- It parses vpk_i as $(X_1, \ldots, X_n, X_0, Z_0, \tau)$.
- Because Game_3 aborts otherwise, we know that $(X_1, \ldots, X_n, X_0, Z_0, \cdot) \in \mathsf{VPKSet}$ and hence $X_1, \ldots, X_n, X_0, Z_0$ were generated by the game.
- Thus, Game_5 is able to look up corresponding values $\mathsf{vsk}_i = (x_1, \ldots, x_n, x_0), \nu$ such that $X_j = x_jG$ for $j \in [n]$ and $(X_0, Z_0) = (x_0G + \nu E, \nu G)$ is a ciphertext of x_0G.
- It computes a MAC $\sigma^{\mathrm{rnd}} := (U, V) \leftarrow \mathsf{CMZ.Sign}(\mathsf{sk} := (x_0, \ldots, x_n), a^{\mathrm{rnd}})$ on a^{rnd}.
- It presents σ^{rnd} on a^{rnd} instead of cred_i on a_i, i.e. it computes $\mathsf{pres} \leftarrow \mathsf{Present}(\mathsf{ipk}^*, \mathsf{vpk}, \mathsf{cred}^{\mathrm{rnd}} := (\sigma^{\mathrm{rnd}}, a^{\mathrm{rnd}}), \mathsf{ctx}, \phi)$ instead of $\mathsf{Present}(\mathsf{ipk}^*, \mathsf{vpk}, \mathsf{cred}_i, \mathsf{ctx}, \phi)$.

This modification does not change the adversary's view. This is because the output $\mathsf{pres} = (\tilde{U}, Z, C_V, C_1, \ldots, C_n, \pi_S)$ of $\mathsf{Present}(\mathsf{ipk}^*, \mathsf{vpk}, \mathsf{cred} = (U, V, a), \mathsf{ctx}, \phi)$ is effectively independent of (U, V, a): $\tilde{U}, C_V, C_1, \ldots, C_n$ are uniformly independently random in $\mathbb{G}$ and Z is uniquely determined by these values and vsk to satisfy the verification equation. π_S is simulated (cf. Game_1) using only $\mathbb{x}_S$, which is independent of (U, V, a).

Additionally, since this means that honest users' cred_i are now unused, Game_5 stops computing honest users' credentials during $\mathcal{O}\mathsf{HCIssue}$

queries. Specifically, for $\mathcal{O}\mathsf{HCIssue}$ queries, Game_5 does not run Iss or Obt_2. Instead, it just checks that $\mathsf{SAGA.Present}$ does not output $\perp$ and that $\mathsf{SAGA.PresKeyedVf}(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pres}_{\mathsf{SAGA}}, \mathsf{ctx}, C_1, \ldots, C_{n+2})$. If the checks succeed, it adds $(\mathsf{vpk}, \mathsf{cred} := \perp, a)$ to $\mathcal{HC}$. It is easy to see that in the previous game (Game_4), all other $\mathsf{Obt}_1, \mathsf{Iss}, \mathsf{Obt}_2$ checks are guaranteed to succeed (given that both user and issuer are honest, $\mathsf{NIP}_R, \mathsf{NIP}_I$ are complete, and the SAGA is correct), hence checking $\mathsf{pres}_{\mathsf{SAGA}}$ is sufficient to model the same behavior as Game_4.

Overall, neither change impacts the adversary's view or the win condition, hence $\Pr[\mathbf{W}_4] = \Pr[\mathbf{W}_5]$.

$\underline{\mathsf{Game}_6}$: This game hop changes how the blinded credential values $\bar{U}, \bar{V}$ are computed when answering $\mathcal{O}\mathsf{CCIssue}$ queries.

For an $\mathcal{O}\mathsf{CCIssue}$ query with $\mathsf{creq} := (\mathsf{pres}_{\mathsf{SAGA}}, \not{\alpha}_R := (C_1, \ldots, C_{n+2}, \bar{X}_0, \bar{Z}_0, C_a), \pi_R)$, we are using the notation of Game_3, i.e. some variables below correspond to (extracted) witness values from π_R. Similar to Game_5, the new Game_6 is able to look up secret values $\mathsf{vsk}_i = (x_1, \ldots, x_n, x_0), \nu$ corresponding to vpk such that $X_j = x_j G$ for $j \in [n]$ and $(X_0, Z_0) = (x_0 G + \nu E, \nu G)$ is a ciphertext of $x_0 G$. This is possible because Game_3 aborts if $(X_1, \ldots, X_n, X_0, Z_0, \cdot) \notin \mathsf{VPKSet}$.

In Game_5, the (blinded) credential response $(\bar{U}, \bar{V})$ is computed as

$$\bar{U} := uG \qquad\qquad \bar{V} := u\left((\bar{X}_0 - e\bar{Z}_0) + C_a\right)$$

for a random $u \in \mathbb{Z}_p$.

In Game_6, we instead compute $(\bar{U}, \bar{V})$ as follows. Game_6 computes a fresh MAC $(\bar{U}, V') \leftarrow \mathsf{CMZ.Sign}(\mathsf{sk} := (x_0, \ldots, x_n), (a_1, \ldots, a_n))$. Then it computes $\bar{V}$ as

$$\bar{V} := V' + \left(\bar{x}_0 + s + \sum_{j \in [n]} td_j(a_j \xi_j - \xi'_j)\right) \bar{U}$$

by using the trapdoor information td_j from SAGA such that $G_j = td_j G$. The distribution of $(\bar{U}, \bar{V})$ is identical to Game_5, so $\Pr[\mathbf{W}_6] = \Pr[\mathbf{W}_5]$. Intuitively, while Game_5 takes the *blinded request* and applies the *MAC key* (hidden in ciphertext (X_0, Z_0)) to it, Game_6 instead takes a *valid MAC* and applies the *(extracted) blinding terms* to it.

$\underline{\mathsf{Game}_7}$: This game changes how the Elgamal ciphertext (X_0^*, Z_0^*) in the honest verifier public key vpk^* is computed. Instead of computing the ciphertext honestly as $(x_0^* G + \nu^* E, \nu^* G)$, it is computed as $(\alpha^* G, \beta^* G)$ for random $\alpha^*, \beta^* \in \mathbb{Z}_p$.

Transition $\mathsf{Game}_6 \to \mathsf{Game}_7$*:* This change is indistinguishable under the DDH assumption. Specifically, if an adversary can distinguish this change then we can build a DDH distinguisher $\mathcal{R}_{\mathrm{DDH}}$ as follows. Given a DDH challenge $(G, A_{\mathrm{DDH}}, B_{\mathrm{DDH}}, C_{\mathrm{DDH}})$, we set $E := A_{\mathrm{DDH}}$, and $(X_0^*, Z_0^*) := (B_{\mathrm{DDH}}, x_0^* G + C_{\mathrm{DDH}})$. Note that the value e (the discrete logarithm of E and the issuer's credential-issuance secret) is unused in Game_6: since Game_5, $\mathcal{O}\mathsf{HCIssue}$ does not create a credential anymore and in particular does not use e; since Game_6,

$\mathcal{O}\mathsf{CCIssue}$ uses extracted blinding terms to reply instead of using e. If C_{DDH} is the real DH solution, then (X_0^*, Z_0^*) is set as in Game_6 (proper ElGamal ciphertext of $x_0^* G$). Otherwise, (X_0^*, Z_0^*) is set as random group elements. Thus, we conclude that $\Pr[\mathbf{W}_6] \leq \Pr[\mathbf{W}_7] + \mathsf{Adv}^{DDH}_{\mathcal{R}_{\mathrm{DDH}}}(\lambda)$.

$\underline{\mathsf{Game}_8}$: This game aborts if the presentation knowledge extractor fails. More precisely, Game_8 aborts if the game's winning condition is fulfilled but the knowledge extractor of NIP_S (run within $\mathsf{E.P}$) fails to produce a valid witness $\mathbb{w}_S$. Note that if the winning condition is fulfilled, then $(\mathsf{ctx}^*, \phi^*) \notin \mathcal{HP}$, hence we have never simulated a proof π_S^* concerning $\mathsf{ctx}_{\mathsf{NIP}} := (\mathsf{vpk}^*, \mathsf{ctx}^*)$ with the predicate ϕ^*. As a consequence, we can build an efficient simulation-extractability adversary $\mathcal{A}_{SE,S}$ against NIP_S that wins whenever the abort condition of Game_8 occurs, so $\Pr[\mathbf{W}_7] \leq \Pr[\mathbf{W}_8] + \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE},S},\mathsf{NIP}_S}(\lambda)$.

We will conclude our proof by showing that if $\mathcal{A}$ wins in Game_8 with non-negligible probability, then we can build an efficient $\mathsf{CMZ\text{-}Unf}$ adversary $\mathcal{A}_{\mathsf{CMZ}}$. $\mathcal{A}_{\mathsf{CMZ}}$ simulates $\mathcal{A}$'s view by running Game_8 except the following changes. $\mathcal{A}_{\mathsf{CMZ}}$ receives $X_1^*, \ldots, X_n^*$ from its $\mathsf{CMZ\text{-}Unf}$ challenger and incorporates them into the verifier key vpk^*. Note that vsk^* is unknown to $\mathcal{A}_{\mathsf{CMZ}}$. To cope with that, when simulating oracles of Game_8, $\mathcal{A}_{\mathsf{CMZ}}$ makes the following changes.

- When answering $\mathcal{O}\mathsf{CCIssue}$ queries involving vsk^*, instead of running $\mathsf{CMZ.Sign}(\mathsf{vsk}^*, a)$ (cf. Game_6), it makes a query $\mathcal{O}\mathsf{Sign}(a)$ to its own oracle.
- When answering $\mathcal{O}\mathsf{HCShow}$ queries involving vsk^*, instead of running $\mathsf{CMZ.Sign}(\mathsf{vsk}^*, a^{\mathrm{rnd}})$ (cf. Game_5), it makes a query $\mathcal{O}\mathsf{Sign}(a^{\mathrm{rnd}})$ to its own oracle.
- When answering $\mathcal{O}\mathsf{VfPres}$ queries, instead of checking the validity of $(\tilde{U}, Z, C_V, C_1, \ldots, C_n)$ by using vsk^* in $\mathsf{mKVAC}_{\mathsf{CMZ}}.\mathsf{VfPres}$ (cf. Figure 12), it makes an $\mathcal{O}\mathsf{Help}$ query.

When $\mathcal{A}$ outputs its forgery pres^*, $\mathcal{A}_{\mathsf{CMZ}}$ extracts $\mathbb{w}_S^* := \left(\left(\tilde{\gamma}^*, (a_j^*, \gamma_j^*)_{j=1}^n\right), x_1^*\right)$ as in Game_8. For the following, assume that the adversary wins Game_8.

If $x_1^* G = X_1^*$, then $\mathcal{A}_{\mathsf{CMZ}}$ has learned part of the CMZ secret key and can easily output a CMZ forgery by taking any valid CMZ tag $(U, V = x_0 U + \sum_j m_j x_j U)$ on $(m_1, m_2, \ldots, m_n)$ and computing the valid tag $(U, V + r x_1 U)$ on $(m_1 + r, m_2, \ldots, m_n)$ for any $r \in \mathbb{Z}_p$. With this insight, $\mathcal{A}_{\mathsf{CMZ}}$ can easily win its forgery game.

Otherwise, if $x_1^* G \neq X_1^*$, then the $\left(\tilde{\gamma}^*, (a_j^*, \gamma_j^*)_{j=1}^n\right)$ part of the witness $\mathbb{w}_S^*$ must be satisfying $\mathcal{R}_S$. In this case, $\mathcal{A}_{\mathsf{CMZ}}$ computes its forgery as $\left(\sigma^* := (\tilde{U}^*, C_V^* - \tilde{\gamma}^* H), a^* = (a_1^*, \ldots, a_n^*)\right)$. Notice that σ^* is a valid $\mu\mathsf{CMZ}$ tag on a^*. Furthermore, the forgery is non-trivial: $\mathcal{A}_{\mathsf{CMZ}}$ only made $\mathcal{O}\mathsf{Sign}$ queries for $a \in \mathcal{CC}$ and for a^{rnd}. We know that $a^* \notin \mathcal{CC}$ from the winning condition and we know $a^* \neq a^{\mathrm{rnd}}$ because otherwise the game would have aborted (cf. Game_4)

Hence whenever $\mathcal{A}$ wins Game_8, $\mathcal{A}_{\mathsf{CMZ}}$ is able to win its CMZ unforgeability game. It follows that $\Pr[\mathbf{W}_8] \leq \mathsf{Adv}^{\mathsf{CMZ\text{-}Unf},n}_{\mathcal{A}_{\mathsf{CMZ}}}(\lambda)$ and we conclude that overall,

$$\begin{aligned}\mathsf{Adv}^{\mathsf{Unf}}_{\mathsf{mKVAC},\mathcal{A},\mathsf{E}}(\lambda) \leq\ & \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathrm{ZK,I}},\mathsf{NIP}_I}(\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathrm{ZK,S}},\mathsf{NIP}_S}(\lambda) \\ & + \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE},R},\mathsf{NIP}_R}(\lambda) + \mathsf{Adv}^{\mathsf{UF-CoMA}}_{\mathcal{R}_{\mathsf{UF-CoMA}},\mathsf{SAGA}}(\lambda) \\ & + \mathsf{Adv}^{\mathsf{DDH}}_{\mathcal{R}_{\mathrm{DDH}}}(\lambda) + \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE},S},\mathsf{NIP}_S}(\lambda) + \mathsf{Adv}^{\mathsf{CMZ},n}_{\mathcal{A}_{\mathsf{CMZ}}}(\lambda) + \frac{1}{p^n}.\end{aligned}$$

□

G.2 Proof of presentation anonymity

Theorem 5 (restated). $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *has presentation anonymity if* NIP_I *is extractable and* NIP_S *is zero-knowledge.*

Proof. $\underline{\mathsf{Game}_1}$: This game changes how $\mathcal{O}\mathsf{Obt}_2$ behaves. When $\mathsf{cred} \neq \bot$ for a call, this game also runs the knowledge extractor of NIP_I on π_I to obtain a valid $\mathbb{w}_I := (e, \cdot)$. If the extractor fails to achieve this, then Game_1 aborts.

It is worth to note for the rest of the proof that e is uniquely determined by the group element E, so Game_1 does not have to handle the consistency of extracted e values from multiple calls. Furthermore if the abort condition of Game_1 occurs non-negligibly, then we can build a simulation-extractability adversary $\mathcal{A}_{\mathsf{SE}}$ against NIP_I, so $\left|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]\right| \leq \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE}},\mathsf{NIP}_I}(\lambda)$.
$\underline{\mathsf{Game}_2}$: This game changes how $\mathcal{O}\mathsf{LoR}$ computes π_S which is part of pres_b. Game_2 computes π_S by running the zero-knowledge simulator of NIP_S. Obviously, this change is indistinguishable by $\mathcal{A}$ or else we can build an efficient zero-knowledge adversary $\mathcal{A}_{\mathsf{ZK}}$ such that $\left|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]\right| \leq \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathrm{ZK}},\mathsf{NIP}_S}(\lambda)$.
$\underline{\mathsf{Game}_3}$: Next we change how $\tilde{U}, C_j, C_V, Z$ of $\mathsf{pres}^{(b)}$ are computed in $\mathcal{O}\mathsf{LoR}$ calls. For each call $\mathsf{vpk} = \mathsf{vpk}^{(0)} = \mathsf{vpk}^{(1)}$ be parsed as $(\tau, X_1, \ldots, X_n, X_0, Z_0)$ and let $\tilde{X}_0 := X_0 - eZ_0$. Game_3 sets

$$\tilde{U} := r_0 G, \qquad\qquad C_j := r_j G,$$

for random $r_0, \ldots, r_n, r_r \xleftarrow{\$} \mathbb{Z}_p$ and $C_V \xleftarrow{\$} \mathbb{G}$ is sampled randomly. Then equivocated Z is computed as

$$Z := r_0 \tilde{X}_0 + \sum_{j \in [n]} r_j X_j - C_V.$$

π_S is simulated as before. This change is perfectly indistinguishable. First, notice that the distribution of $\tilde{U}, C_j, C_V$ in the orignal $\mathsf{Present}$ algorithm are uniformly random due to the re-randomization factors $\gamma, \gamma_j, \gamma_r, \tilde{\gamma}$, respectively. Similarly, Game_4 simulates these values uniformly by relying on r_0, r_j, r_r.

Furthermore, by relying on the knowledge extractor from $\mathcal{O}\mathsf{Obt}_2$, we know that cred_b is valid $\mathsf{CMZ\ MAC}$ for the secret key $(\tilde{x}_0, x_1, \ldots, x_n)$ for both $b = 0$ and $b = 1$. This means that for a fixed $\tilde{U}, C_j, C_V$, there is a unique valid Z such

that $Z = x_0\tilde{U} + \sum_{j\in[n]} x_j C_j - C_V$. One can observe that the simulated Z in Game_3 also satisfies this equation.

In Game_3, $\mathcal{O}\mathsf{LoR}$ does not use the challenge bit b, so the adversary's chance to guess b is not better than a random guess and $\Pr[\mathbf{W}_3] = 1/2$. Finally, we conclude that,

$$\mathsf{Adv}^{\mathsf{PAnon}}_{\mathcal{A},\mathsf{mKVAC}_{\mathsf{CMZ}}}(\lambda) \leq \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE}},\mathsf{NIP}_I}(\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathsf{ZK}},\mathsf{NIP}_S}(\lambda).$$

□

G.3 Proof of issuance anonymity

Theorem 6 (restated). $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *scheme from Figures 10 to 12 satisfies the issuance anonymity property if* NIP_I *is simulation-extractable,* NIP_R *is zero-knowledge, and the underlying* SAGA *scheme satisfies privacy.*

Proof ($\mathsf{mKVAC}_{\mathsf{CMZ}}$ *Issuance Anonymity).* We prove Theorem 6 by using a sequence of game hops Game_i. Let $\mathbf{W}_i$ denote the event that the adversary $\mathcal{A}$ wins in Game_i. Game_0 corresponds to the original issuance anonymity game.
$\underline{\mathsf{Game}_1}$: In this game, by the simulation-extractability property of π_I, we attempt to extract the same value e from both $\mathsf{bcred}^{(b)}$ and $\mathsf{bcred}^{(1-b)}$. If this extraction fails, we abort. Otherwise, this ensures that

$$\mathsf{bcred}^{(b)} := (\bar{U}_b, \bar{V}_b, \pi_I^{(b)}) \quad \text{and} \quad \mathsf{bcred}^{(1-b)} := (\bar{U}_{1-b}, \bar{V}_{1-b}, \pi_I^{(1-b)})$$

are well-formed. In particular, for $i \in \{0,1\}$,

$$\bar{U}_i := u_i G, \qquad \bar{V}_i := u_i\big((\bar{X}_0 - e\bar{Z}_0) + C_{a^{(i)}}\big).$$

If we compute cred based on bcred, we obtain

$$U_i := \gamma_i u_i G, \qquad V_i := \gamma_i u_i\big(\bar{V}_i - (s+\bar{x}_0)\bar{U}_i\big) = \gamma_i u_i\Big(\sum_{j\in[n]} a_j^{(i)} X_j + X_0 - eZ_0\Big),$$

where γ_i is chosen uniformly at random.

This shows that both credentials can in fact be computed without relying on bcred. Using the extracted value e, we can directly sign both attribute sets $a^{(i)}$ for $i \in \{0,1\}$ and generate the corresponding credentials as

$$\gamma \xleftarrow{\$} \mathbb{Z}_p^*, \qquad U_i := \gamma G, \qquad V_i := \gamma\Big(\sum_{j\in[n]} a_j^{(i)} X_j + X_0 - eZ_0\Big),$$

$$\mathsf{cred}_{(i)} := (U_i, V_i, a^{(i)}).$$

The credentials computed in this way are statistically indistinguishable from those generated using bcred. Hence, the only distinguishing gap between this

game and the original game arises from breaking the simulation-extractability of NIP_I, and therefore

$$\left|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]\right| \leq \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A},\mathsf{NIP}_I}(\lambda).$$

$\underline{\mathsf{Game}_2}$: creq consists of three parts $(\mathsf{pres}_{\mathsf{SAGA}}, \aleph_R, \pi_R)$. By the zero-knowledge property of NIP_R, we can replace π_R with a simulated proof, which does not reveal any information about the bit b to the adversary. Hence, the adversary's advantage in distinguishing b is bounded as

$$\left|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]\right| \leq \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_R}(\lambda).$$

$\underline{\mathsf{Game}_3}$: In this game, we compute both creq using the attribute set $a^{(0)}$. Formally, for $i \in \{0,1\}$ we set

$$(\mathsf{creq}^{(i)}, \mathsf{st}_i) \leftarrow \mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}^{(i)}, a^{(0)}, \phi).$$

Note that the attribute vector a is only used in the component C_a of the statement $\aleph_R$ within creq, where

$$\aleph_R \coloneqq (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a).$$

The components $\bar{X}_0, \bar{Z}_0, C_a$ do not reveal any information about the bit b, since they are generated as

$$s, \bar{x}_0, \bar{v} \xleftarrow{\$} \mathbb{Z}_p^*,$$

$$\bar{X}_0 \coloneqq X_0 + \bar{x}_0 G + \bar{v} E, \qquad \bar{Z}_0 \coloneqq Z_0 + \bar{v} G, \qquad C_a \coloneqq \sum_{j \in [n]} a_j X_j + sG,$$

and are therefore indistinguishable from uniformly random group elements.

Consequently, replacing $a^{(i)}$ with $a^{(0)}$ when computing C_a for both creq does not affect the adversary's view. Thus, this game is indistinguishable from the previous one:

$$\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_2].$$

$\underline{\mathsf{Game}_4}$: Now, the adversary's distinguishing power is limited to the following values:

$$L_b \coloneqq (\mathsf{pres}^{(b)}_{\mathsf{SAGA}}, C_1^{(b)}, \ldots, C_{n+3}^{(b)}), \qquad L_{1-b} \coloneqq (\mathsf{pres}^{(1-b)}_{\mathsf{SAGA}}, C_1^{(1-b)}, \ldots, C_{n+3}^{(1-b)}),$$

which are directly related to the privacy of the underlying SAGA scheme.

In this game, we compute both creq using the public key $\mathsf{vpk}^{(0)}$. Formally, for $i \in \{0,1\}$ we set

$$(\mathsf{creq}^{(i)}, \mathsf{st}_i) \leftarrow \mathsf{Obt}_1(\mathsf{ipk}, \mathsf{vpk}^{(0)}, a^{(0)}, \phi).$$

Hence, both L_b and L_{1-b} are computed using $\mathsf{vpk}^{(0)}$.

If there exists an adversary $\mathcal{A}$ that can distinguish this game from the previous one, then we can use $\mathcal{A}$ to break the privacy of the underlying SAGA

$\mathsf{DenS}(\mathsf{ipk}, \mathsf{vsk}, \mathsf{ctx}, \phi)$

parse vsk **as** $(x_0, \ldots, x_n, x_r, X_0, \ldots, X_n, X_r)$

$\tilde{U}, C_V, C_1, \ldots, C_n \xleftarrow{\$} \mathbb{G},\ Z := x_0\tilde{U} + \sum_{j\in[n]} x_j C_j - C_V$

$\mathbb{x}_S := (X_1, \ldots, X_n, \tilde{U}),\ \mathbb{w}_S := (\bot, x_1)$

$\pi_S \leftarrow \mathsf{NIP}_S.\mathsf{Prove}(\mathbb{x}_S, \mathbb{w}_S, (\mathsf{vpk}, \mathsf{ctx}))$

return $\mathsf{pres} := (\tilde{U}, Z, C_V, C_1, \ldots, C_n, \pi_S)$

Fig. 17. Description of DenS for $\mathsf{mKVAC}_{\mathsf{CMZ}}$.

scheme. The SAGA adversary works as follows: The SAGA challenger provides $(\mathsf{pres}^{(b)}_{\mathsf{SAGA}}, \vec{C}^{(b)})$. The SAGA adversary attaches fresh random group elements $(\bar{X}_0, \bar{Z}_0, C_a)$ together with a simulated proof π_I in order to compute $\mathsf{creq}^{(b)}$. It then generates $\mathsf{creq}^{(1-b)}$ using vpk_0, fresh random elements $(\bar{X}_0, \bar{Z}_0, C_a)$, and another simulated proof. Finally, the SAGA adversary forwards everything to $\mathcal{A}$ and outputs whatever $\mathcal{A}$ outputs.

If $\mathcal{A}$ can distinguish between Game_4 and Game_3 with non-negligible advantage, then the SAGA adversary can also distinguish the bit b in the SAGA privacy game with non-negligible probability, contradicting the assumed privacy of the SAGA scheme. Therefore, the adversary's distinguishing advantage between these two games is bounded by

$$\left|\Pr[\mathbf{W}_4] - \Pr[\mathbf{W}_3]\right| \leq \mathbf{Adv}^{\mathsf{PRIV}}_{\mathsf{SAGA},\mathcal{A}}(\lambda).$$

In the final game, the bit b is no longer used and can therefore be sampled even after the adversary's interaction. Consequently, the adversary's advantage in guessing b is exactly $1/2$. Thus, we obtain

$$\mathsf{Adv}^{\mathsf{IAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{IAnon}}_{\mathcal{A},\mathsf{mKVAC}}(\lambda) = \mathsf{true}\right] - \tfrac{1}{2}\right| \leq \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A},\mathsf{NIP}_I}(\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_R}(\lambda) + \mathbf{Adv}^{\mathsf{PRIV}}_{\mathsf{SAGA},\mathcal{A}}(\lambda),$$

which is negligible. □

G.4 Proof of deniability

Theorem 7 (restated). $\mathsf{mKVAC}_{\mathsf{CMZ}}$ *is deniable for* DenS *in Figure 17 if* NIP_I *is sound, and* NIP_S *is zero-knowledge.*

Proof. $\underline{\mathsf{Game}_1}$: This game ensures that cred is a valid credential on a so that its presentations will not be trivially distinguishable from valid presentations of DenS. Namely, additional to checking that π_I holds, Game_1 also checks if $(x_0 + \sum_{j\in[n]} x_j a_j)U = V$. If this does not hold, Game_1 sets $\mathsf{cred} = \bot$. Notice that if π_I is valid but the extra condition introduced in Game_1 does not hold,

then the adversary breaks the soundness of NIP_I and we can build a soundness adversary $\mathcal{A}_{\mathsf{Snd}}$, so $\Pr[\mathbf{W}_1] \leq \Pr[\mathbf{W}_0] + \mathsf{Adv}^{\mathsf{Snd}}_{\mathcal{A}_{\mathsf{Snd}},\mathsf{NIP}_I}(\lambda)$.
$\underline{\mathsf{Game}_2}$: This game changes how π_S is computed in $\mathcal{O}\mathsf{Chall}$. For both $b = 0$ and $b = 1$, when $\mathsf{NIP}_S.\mathsf{Prove}$, Game_2 simulates π_S. We can build a zero-knowledge adversary $\mathcal{A}_{\mathsf{ZK}}$ against NIP_S if an efficient adversary can distinguish Game_1 and Game_2, so $\Pr[\mathbf{W}_2] \leq \Pr[\mathbf{W}_1] + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathsf{ZK}},\mathsf{NIP}_S,\mathsf{S}}(\lambda)$.
$\underline{\mathsf{Game}_3}$: This game changes how $\tilde{U}, C_V, C_1, \ldots, C_n, Z$ are computed when $b = 0$. Game_3 computes these value identically to DenS. The proof π_S is simulated as before. This change is perfectly indistinguishable as the distribution of output pres is unchanged, $\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_2]$.

In Game_3, the behavior of $\mathcal{O}\mathsf{Chall}$ is independent of the challenge bit b. Thus, Game_3 leaks no information about b to $\mathcal{A}$ and $\mathcal{A}$ can guess it only randomly: $\Pr[\mathbf{W}_3] \leq 1/2$. As a result, we conclude that

$$\mathsf{Adv}^{\mathsf{Den}}_{\mathcal{A},\mathsf{mKVAC}_{\mathsf{CMZ}},\mathsf{DenS}}(\lambda) \leq \mathsf{Adv}^{\mathsf{Snd}}_{\mathcal{A}_{\mathsf{Snd}},\mathsf{NIP}_I}(\lambda) + \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A}_{\mathsf{ZK}},\mathsf{NIP}_S,\mathsf{S}}(\lambda).$$

□

H Deferred Content of Section 4

Definition 16 (Correctness). *A scheme* SAGA *as in Definition 7 is* correct *if for all security parameters* $\lambda \in \mathbb{N}$*, all possible groups* $(\mathbb{G}, p, G) \in [\mathsf{GGen}(1^\lambda)]$*, all message length parameters* ℓ*, and all message vectors* $\vec{M} \in \mathbb{G}^\ell$*:*

$$\Pr\left[\begin{array}{l} \mathsf{Setup}(\lambda, \ell, \mathbb{G}, p, G) \to (\mathsf{pp}_{\mathsf{SAGA}}, \vec{G}, \vec{td}), \\ \mathsf{Kg}(\mathsf{pp}_{\mathsf{SAGA}}) \to (\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pk}_{\mathsf{SAGA}}), \mathsf{MAC}(\mathsf{sk}_{\mathsf{SAGA}}, \vec{M}) \to \tau, \\ \mathsf{Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau, \vec{M}) \to (\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}, \vec{\xi}, \mathsf{wit}_{\mathsf{SAGA}}) : \\ \forall j : \ C_j = M_j + \xi_j G_j \wedge \mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau, \vec{M}) \\ \wedge \ \mathsf{PresWitVf}^{\mathsf{pres}_{\mathsf{SAGA}}, \vec{C}}(\mathsf{wit}_{\mathsf{SAGA}}, \vec{\xi}) \ \wedge \ \mathsf{PresKeyedVf}(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pres}_{\mathsf{SAGA}}, \vec{C}) \end{array}\right] = 1.$$

I Failed attempts at BBS-based SAGA

Recall that BBS signatures on a message $m_1 \in \mathbb{Z}_p$ are in the form of $\sigma := (A := (x + d)^{-1}(G_0 + m_1 Y_1), d)$ for a key pair $(\mathsf{sk} := x, \mathsf{pk} := (X := xG, Y_1))$ and $d \xleftarrow{\$} \mathbb{Z}_p$.

First Attempt to Sign Group Elements. Consider our scheme to sign a single group element for simplicity. Given $Y_1 = y_1 G$, the original scheme computes $A := (x+d)^{-1}(G_0 C)$ where $C := m_1 y_1 G$. However, when we wish to create and verify tags on $M_1 := m_1 G$, C must be computed without knowing m_1. Based on this intuition, we extend the BBS secret key as $\mathsf{sk} := (x, y_1)$, and when signing a group element M_1 — or when verifying a tag on M_1 — C is computed as $C := y_1 M_1$.

Unfortunately this first version turns out to be insecure: Let an adversary obtain a tag on an arbitrary M_1 as $\sigma := (A := (x+d)^{-1}(G_0 + y_1 M_1), d)$. Then the adversary can forge a tag on $M^* := M_1 + X + dG$ as $A^* := (A + Y_1)$ for the same d. This tag is valid since

$$\begin{aligned}(x+d)A^* &= (x+d)A + (x+d)Y_1 \\ &= G_0 + y_1 M_1 + (x+d)Y_1 \\ &= G_0 + y_1 M_1 + y_1(X + dG) \\ &= G_0 + y_1(M_1 + X + dG) = G_0 + y_1 M^*\end{aligned}$$

Fixing the Scheme. The attack above occurs due to the switch in the message space. Notice that the same attack strategy would not work in BBS MAC on field elements as one has to know the discrete logarithm $m^* = m_1 + x + d$ of M^* which requires to compute the secret key x of the signer.

Intuitively, we prevent the above attack by changing the structure of the public key Y_1. Instead of computing Y_1 with base G, it is computed with a random generator G_1 as $Y_1 := y_1 G_1$. This prevents the above attack as now the forgery message requires to compute $M^* := M_1 + g_1 X + dG$ where g_1 is the discrete logarithm of $G_1 := g_1 G$.

J MAC unforgeability definition for SAGA

In this section, we define unforgeability with respect to the MAC component of the SAGA scheme, i.e. it is hard to produce a MAC accepted by MACVerify except by querying an open message MAC oracle $\mathcal{O}$MAC. This property will be used to as an intermediate step in the proof of unforgeability of a full SAGA scheme (see Theorems 1 and 2).

Definition 17 (MAC unforgeability under Chosen open Message Attack). *A* SAGA *scheme satisfies MAC unforgeability under chosen open message attack (UF-CoMA) if for all PPT adversaries $\mathcal{A}$, the advantage*

$$\mathbf{Adv}^{\mathit{UF\text{-}CoMA\text{-}MAC}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) := \Pr[\mathsf{Exp}^{\mathsf{UF\text{-}CoMA\text{-}MAC}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) = 1]$$

is negligible in λ.

K Proofs for BBSSAGA

Theorem 1 (restated). *If* $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ *is zero-knowledge, then* BBSSAGA *(Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 17) in GGM.*

$\underline{\mathsf{Exp}^{\mathsf{UF\text{-}CoMA\text{-}MAC}}_{\mathsf{SAGA},\mathcal{A}}(\lambda)}$
$(\mathbb{G}, p, G) \leftarrow \mathsf{GGen}(1^\lambda)$
$(\mathsf{pp}_{\mathsf{SAGA}}, (G_j, td_j)_{j=1}^{\ell}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, \mathbb{G}, p, G)$
$(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pk}_{\mathsf{SAGA}}) \leftarrow \mathsf{Kg}(\mathsf{pp}_{\mathsf{SAGA}})$
$(\tau, \vec{M}) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{MAC}, \mathcal{O}\mathsf{MACVerify}}(\mathsf{pp}_{\mathsf{SAGA}}, \mathsf{pk}_{\mathsf{SAGA}})$
$b := \mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau, \vec{M})$
return $b = 1 \wedge \vec{M} \notin \mathcal{Q}$

$\underline{\mathcal{O}\mathsf{MAC}(\alpha_1, \ldots, \alpha_\ell)}$
for $j \in [\ell]$: $M_j := \alpha_j \cdot G$
$\vec{M} := (M_1, \ldots, M_\ell)$
$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\vec{M}\}$
return $\mathsf{MAC}(\mathsf{sk}_{\mathsf{SAGA}}, \vec{M})$

$\underline{\mathcal{O}\mathsf{MACVerify}(\tau', \vec{M}')}$
return $\mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau', \vec{M}')$

Fig. 18. MAC unforgeability game for SAGA. The adversary has access to $\mathcal{O}$MAC and $\mathcal{O}$MACVerify oracles.

Proof. We define a sequence of game hops. Let $\mathcal{A}$ be a (generic) adversary against the MAC unforgeability of BBSSAGA as defined in Definition 17 in the generic group model (i.e. both $\mathcal{A}$ and the game receive random encodings of group elements and use oracles to execute group operations). Let $\mathbf{W}_i$ denote the event that the adversary $\mathcal{A}$ wins in Game_i. Game_0 is identical to the original MAC unforgeability game.

$\underline{\mathsf{Game}_1}$: The tag consists of three components $(A, d, \pi_{\mathsf{BBSSAGA}})$. We begin by switching to a modified game in which the adversary receives simulated NIZK proofs π_{BBSSAGA} for its MAC oracle queries. By the zero-knowledge property of the NIZK proof system, these simulated proofs do not leak any information about the secret keys $(x, \vec{y})$. Hence, the adversary's distinguishing advantage between the modified game and the original game is bounded by $\mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathsf{BBSSAGA}}}(\lambda)$. Consequently, the adversary accepts the MAC oracle tags and does not gain any useful information from the NIZK proofs. Importantly, after this switch, the secret keys $(x, \vec{y})$ only appear "in the exponent," which enables us to treat them as formal variables in the polynomial-based execution of the GGM.

$$\left|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]\right| \leq \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathsf{BBSSAGA}}}(\lambda)$$

$\underline{\mathsf{Game}_2}$: This game is identical to the previous one, except that we abort if the responses to at least two distinct MAC queries contain the same value d, since such a collision would allow the adversary to forge a MAC easily, or if $d = -x$. The probability of this event is negligible: for q MAC queries, the collision probability is

$$1 - \frac{p!}{p^q(p-q)!} \approx \frac{q^2}{2p},$$

which is negligible in the security parameter λ, as $p \gg q$. We have,

$$\left|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]\right| \leq \frac{q^2}{2p} + \frac{q}{p}.$$

$\underline{\mathsf{Game}_3}$: This game is identical to the previous one, except that the game chooses all q values d_i $(i \in [q])$ at the start of the game. We replace the generator G with

$f(x)G$. Let H denote the generator in Game_2. In Game_3, we set $G := f(x)H$, where $f(x) := \prod_{i=1}^{q}(x + d)$. We further set $G_0 := g_0 G$ (for random $g_0 \xleftarrow{\$} \mathbb{Z}_p$), and all other computations referencing the generator H in Game_2 are now using the new generator G.

In particular, the i-th MAC query is answered as

$$A_i := \frac{1}{(x + d_i)} \left(g_0 G + \sum_{j \in [\ell]} y_j m_{i,j} G \right)$$

(where $m_{i,1}, \ldots, m_{i,\ell} \in \mathbb{Z}_p$ is the message for the ith query). The public key is $\vec{G} := t\vec{d}G$, meaning $\vec{G} := f(x)t\vec{d}H$. This change prepares for the next game hop by removing divisions $1/(x + d_i)$: A_i can be computed without division. The adversary's winning probability does not change because $f(x) \neq 0$ (as enforced in Game_2). Hence

$$\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_2].$$

$\underline{\mathsf{Game}_4}$: This game is identical to the previous one, except that we replace the generic group model with a polynomial-based execution, as usual in GGM proofs. This means that instead of randomly encoded group elements, the GGM is now based on randomly encoded polynomials in the ring

$$\mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^{\ell}, x].$$

As a result, Game_4 now treats $G, g_0, (td_j, y_j)_{j=1}^{\ell}, x$ as polynomial variables instead of choosing random scalar values in $\mathbb{Z}_p$ for them. The Game_2 generator H in Game_3 corresponds to the polynomial 1, the Game_3 generator G corresponds to the polynomial $f(x)$ (we simply write $G := f(x)$). The group operation $B + B'$ corresponds to polynomial addition $b + b'$, where b is the polynomial corresponding to B and b' is the polynomial corresponding to B'. Similarly, scalar multiplication aB in the group corresponds to scalar multiplication $a \cdot B$ in the polynomial ring.

The $\mathsf{MACVerify}(\tau' = (A', d'), \vec{M}')$ oracle stops computing group operations and instead directly checks the polynomial equation $(x + d')A' = g_0 + \vec{y} \cdot \vec{M}'$ over the ring $\mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^{\ell}, x]$. (note that we are interpreting A', M'_j as polynomials corresponding to the randomly encoded group elements supplied by the adversary to the oracle).

Game_4 produces the same distribution of encoded group elements as Game_3 unless there are two different encoded polynomials $g \neq g' \in \mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^{\ell}, x]$ that evaluate to the same scalar element when $g_0, (td_j, y_j)_{j=1}^{\ell}, x$ are instantiated with random $\mathbb{Z}_p$ elements. By the Schwartz-Zippel lemma, for any pair $g \neq g'$, such a collision happens with probability at most t/p, where t is the degree of $g - g'$. If the game overall makes at most q_{GGM} group oracle queries, then applying the union bound over all pairs g, g' gives us

$$\left|\Pr[\mathbf{W}_4] - \Pr[\mathbf{W}_3]\right| \leq \frac{(q_{\mathrm{GGM}})^2 t}{p},$$

with $t := q + 1$. This is because MAC responses A_i have degree at most q, since f is degree q. $\mathsf{MACVerify}$ multiplies arbitrary polynomials with y_j, resulting in the maximum degree $q + 1$.

In the remainder of this proof, we show that the probability to create a forgery in Game_4 is zero.

Suppose the adversary outputs a forgery consisting of a tag-message pair (A^*, d^*) and $(M_1^*, \ldots, M_\ell^*)$ fulfilling the verification equation

$$(x + d^*)A^* = g_0$$

We argue that such a forgery can only be valid if the forged pair corresponds to one of the messages previously queried to the oracle together with its returned tag. Since the definition of the game requires the forgery to be on a fresh message (i.e., one not in the query set), this case is ruled out. Therefore, the adversary cannot produce a valid forgery.

Now consider an execution of Game_4. Suppose the adversary makes at most q queries. For the i-th queried message $(m_{i,1}, \ldots, m_{i,\ell})$, where $i \in [q]$, the MAC oracle returns the pair (A_i, d_i), where

$$A_i = \frac{1}{(x + d_i)} \left(g_0 G + \sum_{j \in [\ell]} y_j m_{i,j} G \right) = \frac{f(x)}{(x + d_i)} \left(g_0 + \sum_{j \in [\ell]} y_j m_{i,j} \right)$$

due to $G = f(x)$. The list of group elements the adversary sees prior to the forgery is (in terms of polynomials)

$$\big(g_0 G, (td_j G, y_j td_j G)_{j \in [\ell]},\ xG,\ (A_i)_{i \in [q]}\big).$$

where $g_0 G$ corresponds to G_0, the polynomial $td_j G$ corresponds to $G_j := td_j$, the polynomial $y_j tdG$ corresponds to $Y_j := y_j G_j$, and xG corresponds to $X := xG$. Note that there are no constant terms because we do not give H to the adversary. (Additionally, the adversary gets simulated proofs π_{BBSSAGA}, which we omit because they do not play a role in this proof; the adversary can compute these proofs itself using the ZK simulator).

A forged MAC (A^*, d^*) on a message $(M_1^*, \ldots, M_\ell^*)$ can be expressed as a linear combination of these elements, using coefficients from a vector $\vec{\alpha}$:

$$A^* = \alpha_a G + \alpha_{a,g_0} g_0 G + \sum_{j \in [\ell]} (\alpha_{a,td_j}\, td_j G + \alpha_{a,y_j} y_j td_j G) + \alpha_{a,x} xG + \sum_{i \in [q]} (\alpha_{a,a_i} A_i)$$

$$\forall k \in [\ell] : M_k^* = \alpha_{m_k} G + \alpha_{m_k,g_0} g_0 G + \sum_{j \in [\ell]} (\alpha_{m_k,td_j}\, td_j G + \alpha_{m_k,y_j} y_j td_j G) + \alpha_{m_k,x} xG + \sum_{i \in [q]} (\alpha_{m_k,a_i} A_i)$$

Let

$$f(x) := \prod_{i \in [q]} (x + d_i) \quad \text{and} \quad f_k(x) := \frac{f(x)}{x + d_k},$$

for each $k \in [q]$. Then:

$$A^* = \alpha_a G + \alpha_{a,g_0} g_0 G + \sum_{j\in[\ell]} (\alpha_{a,td_j}\, td_j G + \alpha_{a,y_j} y_j td_j G) + \\ \alpha_{a,x} xG + \sum_{i\in[q]} (\alpha_{a,a_i} f_i(x)(g_0 G + \sum_{j\in[\ell]} y_j m_{i,j} G))$$

$$\forall k \in [\ell] : M_k^* = \alpha_{m_k} G + \alpha_{m_k,g_0} g_0 G \\ + \sum_{j\in[\ell]} (\alpha_{m_k,td_j}\, td_j G + \alpha_{m_k,y_j} y_j td_j G) + \alpha_{m_k,x} xG \\ + \sum_{i\in[q]} (\alpha_{m_k,a_i} f_i(x)(g_0 G + \sum_{j\in[\ell]} y_j m_{i,j} G))$$

A successful forgery must satisfy the following equation:

$$(x + d^*)A^* = g_0 G + \sum_{k\in[\ell]} y_k M_k^*$$

In the right-hand side, there appear monomials of the form $y_k y_j td_j$, $y_k y_j$, $y_k g_0$, and $y_k td_j$ for $j \neq k$, while no such terms occur in the left-hand side. Therefore, it must hold that

$$\alpha_{m_k,a_i} = \alpha_{m_k,y_j} = \alpha_{m_k,g_0} = \alpha_{m_k,td_j} = 0$$

for all $j, k \in [\ell]$ with $j \neq k$, and for all $i \in [q]$. Moreover, every monomial on the right-hand side contains either y_k or g_0. Hence, no term of the form td_j, G, or xG appears, which implies that

$$\alpha_{a,td_j} = \alpha_a = \alpha_{a,x} = 0 \qquad \text{for all } j \in [\ell].$$

Thus, we obtain:

$$\alpha_{a,g_0}(x + d^*) f(x) g_0 + \sum_{j\in[\ell]} (\alpha_{a,y_j}(x + d^*) f(x) y_j td_j) \\ + \sum_{i\in[q]} (\alpha_{a,a_i}(x + d^*) f_i(x)(g_0 + \sum_{j\in[\ell]} y_j m_{i,j})) \\ = f(x) g_0 + \sum_{k\in[\ell]} y_k (\alpha_{m_k,g_k} f(x) g_k + \alpha_{m_k} f(x) + \alpha_{m_k,x} f(x) x) \quad (1)$$

using $G = f(x)$. Coefficients of g_0 must be equal on both sides, so:

$$\alpha_{a,g_0}(x + d^*) f(x) + \sum_{i\in[q]} (\alpha_{a,a_i}(x + d^*) f_j(x)) = f(x)$$

Since the left-hand side is divisible by $(x + d^*)$ and the right-hand side contains $f(x) = \prod_{i\in[q]}(x+d_i)$, we must have $d^* \in \{d_1, \ldots, d_q\}$. Without loss of generality, assume $d^* = d_1$. Comparing degrees, the left-hand side contains the monomial $\alpha_{a,g_0}x^{q+1}$, whereas the right-hand side does not; hence

$$\alpha_{a,g_0} = 0.$$

Therefore, we have:

$$\alpha_{a,a_1}f(x) + \sum_{i\in[q]\setminus\{1\}}(\alpha_{a,a_i}(x + d_1)f_i(x)) = f(x)$$

$$\rightarrow \sum_{i\in[q]\setminus\{1\}}(\alpha_{a,a_i}(x + d_1)f_i(x)) = (1 - \alpha_{a,a_1})f(x)$$

Observe that the left-hand side has a double root at d_1, whereas the right-hand side has only a single root at d_1. This forces $\alpha_{a,a_1} = 1$ and, consequently, $\alpha_{a,a_i} = 0$ for all $i \in [q] \setminus \{1\}$. Returning to Equation (1), we obtain:

$$\sum_{j\in[\ell]}(\alpha_{a,y_j}(x + d_1)f(x)y_jtd_j) + f(x)(g_0 + \sum_{j\in[\ell]} y_jm_{1,j})$$
$$= f(x)g_0 + \sum_{k\in[\ell]} y_k(\alpha_{m_k,g_k}f(x)g_k + \alpha_{m_k}f(x) + \alpha_{m_k,x}f(x)x)$$

By replacing k with j on the right-hand side and cancelling the term $f(x)g_0$ present on both sides, we obtain:

$$\rightarrow \sum_{j\in[\ell]} \alpha_{a,y_j}(x + d_1)f(x)y_jtd_j + f(x)y_jm_{1,j}$$
$$= \sum_{j\in[\ell]} \alpha_{m_j,td_j}f(x)y_jtd_j + \alpha_{m_j}f(x)y_j + \alpha_{m_j,x}f(x)xy_j \quad (2)$$

The coefficients of y_jtd_j must match on both sides, so for each $j \in [\ell]$ we have:

$$\alpha_{a,y_j}(x + d_1)f(x) = \alpha_{m_j,td_j}f(x).$$

Since the left-hand side has a double root at d_1 while the right-hand side has only a single root, we conclude that $\alpha_{a,y_j} = \alpha_{m_j,td_j} = 0$ for all $j \in [\ell]$. Returning to Equation (2), we obtain:

$$\rightarrow \sum_{j\in[\ell]} f(x)y_jm_{1,j} = \sum_{j\in[\ell]} \alpha_{m_j}f(x)y_j + \alpha_{m_j,x}f(x)xy_j.$$

Hence, for each $j \in [\ell]$, we have

$$\rightarrow m_{1,j} = \alpha_{m_j} + \alpha_{m_j,x}x.$$

Therefore, we have

$$\alpha_{m_j} = m_{1,j} \quad \text{and} \quad \alpha_{m_j,x} = 0.$$

As a result, the only possible forgery is

$$A^* = A_1, \qquad \forall k \in [\ell] : M_k^* = m_{1,k}G = M_{1,k}.$$

This is not a valid forgery because the messsage has been queried before. Therefore, the advantage of the adversary in the original MAC unforgeability game is bounded as follows:

$$\mathsf{Adv}^{\text{UF-CoMA-MAC}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{ZK}}_{\mathcal{A},\mathsf{NIP}_{\mathsf{BBSSAGA}}}(\lambda) + \frac{q^2}{2p} + \frac{q}{p} + \frac{(q_{\text{GGM}})^2(q+1)}{p},$$

Theorem 2 (restated). *If* BBSSAGA *(Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 17), then* BBSSAGA *is presentation unforgeable (Definition 8).*

Proof. We reduce the SAGA unforgeability game to the MAC unforgeability game. To simulate the MAC oracle, we answer each MAC query using the oracle from the MAC unforgeability game.

To simulate a query $\mathcal{O}\mathsf{PresKeyedVf}(\mathsf{pres}'_{\mathsf{SAGA}}, \mathsf{wit}'_{\mathsf{SAGA}}, \vec{C}', \vec{\xi}')$, we first parse (r', d') from $\mathsf{wit}'_{\mathsf{SAGA}}$ and (C'_A, T') from $\mathsf{pres}'_{\mathsf{SAGA}}$, and check whether

$$T' = r'X - d'C'_A + d'r'G - \vec{\xi}' \cdot \vec{Y}.$$

If the check fails, we return 0 and abort. Otherwise, we continue by computing

$$\tau' = C'_A - r'G, \qquad M'_j = C'_j - \xi'_j G_j \quad \text{for } j \in [\ell].$$

We then query $\mathcal{O}\mathsf{MACVerify}(\tau', M'_1, \ldots, M'_\ell)$ and use its output as the answer to $\mathcal{O}\mathsf{PresKeyedVf}$.

When the adversary outputs $(\mathsf{pres}_{\mathsf{SAGA}}, \mathsf{wit}_{\mathsf{SAGA}}, (C_j, \xi_j)_{j=1}^{\ell})$, we first parse (r, d) from $\mathsf{wit}_{\mathsf{SAGA}}$ and (C_A, T) from $\mathsf{pres}_{\mathsf{SAGA}}$, and check whether

$$T = rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}.$$

If the check fails, we return 0 and abort. Otherwise, we continue by computing

$$\tau = C_A - rG, \qquad M_j = C_j - \xi G_j \quad \text{for } j \in [\ell].$$

We then submit $(\tau, M_1, \ldots, M_\ell)$ as the adversary's forgery in the MAC unforgeability game and output the bit b returned by the MAC game. Therefore, the adversary's advantage in this game is exactly the same as in the MAC unforgeability game, i.e.,

$$\mathsf{Adv}^{\mathsf{UF\text{-}CoMA}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) = \mathsf{Adv}^{\text{UF-CoMA-MAC}}_{\mathsf{SAGA},\mathcal{A}}(\lambda),$$

which is negligible. □

Theorem 3 (restated). BBSSAGA *(Figure 9) satisfies the privacy property (Definition 9) if* $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ *is simulation-extractable.*

Proof. If $\mathsf{pres}_{\mathsf{SAGA}}$ equals $\perp$ for either $b = 0$ or $b = 1$, then $\mathsf{Exp}^{\mathsf{PRIV}\text{-}b}$ returns 0 in both cases. Otherwise, assuming the simulation-extractability of $\mathsf{NIP}_{\mathsf{BBSSAGA}}$ holds, both tags are well-formed. In particular, for $b \in \{0, 1\}$, by parsing $(A^{(b)}, d^{(b)})$ from $\tau^{(b)}$, we have

$$A^{(b)} = (x + d^{(b)})^{-1}\Big(G_0 + \sum_{j\in[\ell]} y_j M_j^{(b)}\Big).$$

After running Present, the tuple $(C_A^{(b)}, T^{(b)})$ in $\mathsf{pres}_{\mathsf{SAGA}}^{(b)}$ and $(C_1^{(b)}, \ldots, C_\ell^{(b)})$ are returned. Because of the random values $r^{(b)}$ in $\mathsf{wit}_{\mathsf{SAGA}}^{(b)}$ and $(\xi_1^{(b)}, \ldots, \xi_\ell^{(b)})$, each of $C_A^{(b)}$ and $C_1^{(b)}, \ldots, C_\ell^{(b)}$ is distributed as a uniformly random group element:

$$C_A^{(b)} = A^{(b)} + r^{(b)}G,$$
$$\vec{C}^{(b)} = \vec{M}^{(b)} + \vec{\xi}^{(b)} \circ \vec{G}.$$

Moreover, $T^{(b)}$ must satisfy

$$T^{(b)} = xC_A^{(b)} - G_0 - \sum_{j=1}^{\ell} y_j C_j^{(b)}.$$

Thus, apart from the fact that $T^{(b)}$ always satisfies the above relation, the adversary sees only uniformly random group elements in both experiments. Consequently, there is no information to distinguish the two experiments for different bits b, which concludes the proof. Therefore, the advantage of an adversary in the privacy game of the SAGA is at most the advantage of an adversary in the simulation-extractability game, i.e.,

$$\mathsf{Adv}^{\mathrm{priv}}_{\mathsf{SAGA},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{SE}}_{\mathcal{A}_{\mathsf{SE}},\mathsf{NIP}_{\mathsf{BBSSAGA}}}(\lambda),$$

which is negligible.

□

L A SAGA based on CPZ

In this section, we describe how to instantiate SAGA using CPZ KVACs [CPZ20]. CPZ is a full-fledged KVAC on group elements already. Thus, it is actually more powerful than needed to instantiate SAGA. Namely, its unforgeability holds even without knowing the discrete logarithms of the signed group elements. Furthermore, the presetation of a CPZ-based SAGA is shorter than our BBS-based instantiation. However, the efficiency of CPZ KVACs come with a caveat: they only provide computational anonymity under DDH assumption. This means that the issuance anonymity of $\mathsf{mKVAC}_{\mathsf{CMZ}}$ instantiated with CPZ-based SAGA

would only hold under the DDH assumption. This leaves room for *harvest now, deanonymize later* attacks when DDH problem becomes easy to solve e.g. with quantum computers.

In Figure 19, we describe CPZSAGA, a SAGA instantiation based on CPZ KVACs [CPZ20]. When signing a CPZSAGA tag, additional to computing a CPZ tag (A, B, d), a CPZSAGA signer also issues a NIP proof π_{CPZ}which shows the well-formedness of the issued tag according to the relation

$$\begin{aligned}
&\mathcal{R}_{\mathsf{CPZ}} := \Big\{ (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, ..., \beta_\ell) : \\
&C_W = \omega G_\omega + \tilde{\omega} G_{\tilde{\omega}} \wedge \\
&I = G_B - (\alpha_0 G_{\alpha_0} + \alpha_1 G_{\alpha_1} + \beta_1 G_1 + ... + \beta_\ell G_\ell) \wedge \\
&B = \omega G_\omega + \alpha_0 A + \alpha_1 dA + \sum_{j \in [\ell]} \beta_j M_j \Big\}.
\end{aligned}$$

This proof ensures us that a malicious signer cannot link the presentations of a user by issuing malformed credentials. The presentation of CPZSAGA and PresWitVf are identical the CPZ KVAC presentation in which all attributes are hidden.

$\underline{\mathsf{Setup}(1^\lambda, 1^\ell, \mathbb{G}, p, G)}$

$G_\omega, G_{\tilde{\omega}}, G_{\alpha_0}, G_{\alpha_1}, G_B \xleftarrow{\$} \mathbb{G}$

for $j \in [\ell]$: $td_j \xleftarrow{\$} \mathbb{Z}_p,\ G_j := td_j G$

return $(\mathsf{pp}_{\mathsf{SAGA}} := (\mathbb{G}, p, G, G_\omega, G_{\tilde{\omega}}, G_{\alpha_0}, G_{\alpha_1}, G_B), (G_j, td_j)_{j=1}^{\ell})$

$\underline{\mathsf{Kg}(\mathsf{pp}_{\mathsf{SAGA}}, \mathbb{G}, p, G)}$

$(\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, ..., \beta_\ell) \xleftarrow{\$} \mathbb{Z}_p^{\ell+4}$

$C_W := \omega G_\omega + \tilde{\omega} G_{\tilde{\omega}},\ I := G_B - (\alpha_0 G_{\alpha_0} + \alpha_1 G_{\alpha_1} + \beta_1 G_1 + ... + \beta_\ell G_\ell)$

return $(\mathsf{sk}_{\mathsf{SAGA}} := (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, ..., \beta_\ell), \mathsf{pk}_{\mathsf{SAGA}} := (C_W, I))$

$\underline{\mathsf{MAC}(\mathsf{sk}_{\mathsf{SAGA}}, M_1, \ldots, M_\ell)}$

parse $\mathsf{sk}_{\mathsf{SAGA}}$ **as** $(\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, ..., \beta_\ell)$

$A \xleftarrow{\$} \mathbb{G},\quad d \xleftarrow{\$} \mathbb{Z}_p$

$B := \omega G_\omega + (\alpha_0 + \alpha_1 d)A + \sum_{i \in [\ell]} \beta_i M_i$

$\mathbb{x}_{\mathsf{CPZ}} := (A, B, d, M_1, \ldots, M_\ell)$

$\mathbb{w}_{\mathsf{CPZ}} := (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, ..., \beta_\ell)$

$\pi_{\mathsf{CPZ}} \leftarrow \mathsf{NIP}_{\mathsf{CPZ}}.\mathsf{Prove}(\mathbb{x}_{\mathsf{CPZ}}, \mathbb{w}_{\mathsf{CPZ}})$

return $\tau := (A, B, d, \pi_{\mathsf{CPZ}})$

$\underline{\mathsf{Present}(\mathsf{pk}_{\mathsf{SAGA}}, \tau, M_1, \ldots, M_\ell)}$

parse τ **as** $(A, B, d, \pi_{\mathsf{CPZ}})$

parse $\mathsf{pk}_{\mathsf{SAGA}}$ **as** (C_W, I)

$\mathbb{x}_{\mathsf{CPZ}} := (A, B, d, M_1, \ldots, M_\ell)$

require $\mathsf{NIP}_{\mathsf{CPZ}}.\mathsf{Vf}(\mathbb{x}_{\mathsf{CPZ}}, \pi_{\mathsf{CPZ}})$

$\xi \leftarrow \mathbb{Z}_p,\ \xi_0 := -d\xi,\ J := \xi I$

$C_{\alpha_0} := \xi G_{\alpha_0} + A,\ C_{\alpha_1} := \xi G_{\alpha_1} + dA$

$C_B := \xi G_B + B$

for $j \in [\ell]$: $C_j := \xi G_j + M_j$

return $(\mathsf{pres}_{\mathsf{SAGA}} := (J, C_{\alpha_0}, C_{\alpha_1}, C_B), (C_j, \xi)_{j=1}^{\ell},$
$\mathsf{wit}_{\mathsf{SAGA}} := (\xi_0, d))$

$\underline{\mathsf{PresWitVf}^{\mathsf{pres}_{\mathsf{SAGA}} = (J, C_{\alpha_0}, C_{\alpha_1}, C_B), C_1, \ldots, C_\ell}(\mathsf{wit}_{\mathsf{SAGA}} = (\xi_0, d), \xi, \ldots, \xi)}$

return $J = \xi I \ \wedge\ C_{\alpha_1} = dC_{\alpha_0} + \xi_0 G_{\alpha_0} + \xi G_{\alpha_1}$

$\underline{\mathsf{PresKeyedVf}(\mathsf{sk}_{\mathsf{SAGA}}, \mathsf{pres}_{\mathsf{SAGA}} = (J, C_{\alpha_0}, C_{\alpha_1}, C_B), C_1, \ldots, C_\ell)}$

return $C_B = \omega G_\omega + \alpha_0 C_{\alpha_0} + \alpha_1 C_{\alpha_1} + \sum_{j \in [\ell]} \beta_j C_j$

$\underline{\mathsf{MACVerify}(\mathsf{sk}_{\mathsf{SAGA}}, \tau = (A, B, d, \pi_{\mathsf{CPZ}}), M_1, \ldots, M_\ell)}$

return $B = \omega G_\omega + (\alpha_0 + \alpha_1 d)A + \sum_{j \in [\ell]} \beta_j M_j$

Fig. 19. Description of CPZSAGA.